AlgDat eksamensnotater

Av Jørgen Skontorp

Notater basert på forelesning av Øystein Klemetsdal og André R. Brodtkorb, egen kildekode og Ulf Uttersrud sitt kompendium (https://www.cs.hioa.no/~ulfu/appolonius/index.html)

Innhold

AlgDat eksamensnotater....1
- Algoritmer....4
  - Maksmetode og bytt....4
  - Algoritmers effektivitet....4
    - Dominerende operasjoner....5
  - Algoritmeanalyse....5
  - Tabellintervaller....6
  - Gjennomsnittlig og verste tilfellet....7
  - Analyse av gjennomsnittlig effektivitet....7
  - Big O cheat sheet....7
- Søking....9
  - Gjennomsnittlig antall sammenlikninger....9
  - Usortert søk....10
  - Sortert søk (lineærsøk)....10
  - Binærsøk....13
    - Binærsøk algoritme....14
    - Binærsøk kode....14
- Forskyvinger og rotasjoner....15
- Leksikografisk permutasjon....18
- Sortering....19
  - Partisjonering....19
    - Partisjoneringskode....21
    - Effektivitet....21
  - Quicksort (Kvikksortering)....22
    - Quicksort illustrasjon....23
    - Quicksort kode....24
    - Oppsummering - quicksort:....24
    - Quick search....25
  - Merge (Fletting)....26
    - Flettekode....27
  - Merge sort....28
    - Merge sort illustrasjon....29
    - Merge sort kode....30
  - Selection sort (Utvalgssortering)....31
    - Kode....31
    - Selection sort illustrasjon....32
    - Effektivitet....33
    - Oppsummering - utvalgssortering:....33
  - Insertion sort (Innsettingssortering)....34
    - Insertion sort kode....35
    - Algoritmeanalyse....35
    - Insertion sort illustrasjon....36
    - Oppsummering innsettingssortering....37
  - Shell sort....38
    - Kode....38
    - Shell sort illustrasjon....39
    - Oppsummering - shellsortering....39
  - Bubble sort....40
    - Bubble sort illustrasjon....40
    - Algoritmeanalyse....41
    - Oppsummering - boblesortering:....41
- Generics og java....42
  - Omslagsklasser....43
  - Subtyper....43
  - Ternary if....44
  - Lambda....44
  - Fra-til kontroll....44
  - Komperator interface kode....45
- Rekursjon....47
- Multidimensjonale tabeller....47
- Lineære datastrukturer....47
- Lenkede lister....48
  - Enkeltlenket liste....49
    - Legg inn bakerst....49
    - Legg inn på indeks....50
    - Finn node....50
    - Hent node....50
    - Oppdater verdi....50
    - Fjern node ved indeks....51
    - Indre iteratorklasse....52
    - Array vs. enkeltlenket liste....53
  - Dobbeltlenket liste....54
    - Kode....54
    - Sjekke om liste er tom....55
    - Legg inn node....55
    - Legge inn node med indeks....56
    - Finn indeks til node....57
    - Oppdater verdi basert på indeks....57
    - Fjerne node basert på verdi....58
    - Fjern node basert på indeks....59

Nullstill liste....59
Finn node basert på indeks....60
Antall noder av gitt verdi....60
Effektivitet av lenkede lister....61
Køer....62
Stakker (stack)....62
First in - first out (FIFO)....62
FIFO som enkeltlenket liste....63
Last in - first out (LIFO)....63
Sirkulær kø....64
Lenket kø....66
Lenket kø kode....67
Toveiskø (deque)....68
Lenket toveiskø....68
Sirkulær toveiskø....69
Prioriteringskø....70
Prioriteringskø med usortert tabell....70
Prioriteringskø med sortert tabell....71
Prioriteringskø kode....72
Binære trær....73
Begreper....73
Nodeposisjon....75
Nodeposisjon og binære tall....75
Binærtre kode....76
Typer binære trær....77
Perfekt tre....77
Komplett tre....77
Fullt tre....77
Turneringstre....77
Traverseringer....78
Nivåorden (bredde først)....78
Dybde først....80
Preorden....81
Inorden....82
Postorden....83
Speilvendt og omvendt orden....84
Traversering uten rekursjon....85
Binære søketrær....86
Innlegging....87
Som sortert array....87
Fjerning av node....88
Generelt for fjerning med to barn:....89
Binærtre kode....90
Kopiere....90
Legg inn....91
Antall og tom....91
Finn node....91
Finnes og hent....92
Oppdater....92
Inneholder....92
Posisjon....92
Fjern....93
Nullstill....93
Minimums og maksimumstrær....94
Kode for å sjekke minimumstre....94
Heap....95
Binære heaper....95
Innlegging i minimumsheap....95
Uttak fra minimumsheap....96
Effektivitet - legge inn/uttak....97
Heap som prioriteringskø....97
Heap som turneringstre....98
Heapsortering....98
Andrés kildekode for heapsort....101
Komprimering....103
Huffmantrær....103
Kanonisk huffmantre....104
Huffmankomprimering - koding/dekoding 105
LZW-komprimering....106
LZW algoritmen....106
LZW-komprimering eksempel....107
LZW-dekoding....108
Hashing (hashmap)....109
Hash innlegging....110
Hash søking....110
Hash fjerning....110
Hash kode....111
Balanserte trær....112
Selvbalanserende trær....113
2-3-4 trær....113
Røde og svarte trær....114
Røde og sorte trær eksempel: 1-8....115
Grafer....116
Begreper....116
Urettet graf....116
Rettet graf....117
Grafer i minnet....118
Som tekstfil....118
Grafer som matriser....118
Dijkstras algoritme....119

Algoritmer

En algoritme (eng: algorithm) er en beskrivelse av de operasjonene som skal til for å løse en gitt oppgave - dvs. en oppskrift.

An algorithm is a well-ordered collection of unambiguous and effectively computable operations that, when executed, produces a result and halts in a finite amount of time.

En datastruktur (eng: data structure) er den måten en samling dataverdier er organisert. Vårt fag heter Algoritmer og datastrukturer fordi våre algoritmer vanligvis er nøye knyttet til en datastruktur. Hvis vi feks skal lete etter en bestemt verdi i en samling verdier, er det lett å forstå at letemetoden vil være avhengig av hvordan verdiene er organisert. Har verdiene en bestemt «orden»? Eller ligger de tilfeldig fordelt? Vi vil se at jo mer «orden» det er, jo mer effektive letemetoder kan vi normalt lage.

Maksmetode og bytt

```
public static int maks(int[] a) {
    int m = 0;               // indeks til største verdi
    int maksverdi = a[0];    // største verdi

    for (int i = 1; i < a.length; i++) if (a[i] > maksverdi)
    {
      maksverdi = a[i];     // største verdi oppdateres
      m = i;                // indeks til største verdi oppdateres
    }
    return m;   // returnerer indeks/posisjonen til største verdi

  } // maks

 public static void bytt(int[] a, int i, int j) {

    int temp = a[i]; a[i] = a[j]; a[j] = temp;
  }
```

Algoritmers effektivitet

En algoritme er en beskrivelse av de arbeidsoperasjonene som skal til for å løse en bestemt oppgave.

En algoritmes tidsforbruk vil normalt være avhengig av de dataverdiene som algoritmen arbeider med. Hvis en bruker tidsforbruket til å si noe om effektiviteten, må en gjøre mange testkjøringer med datasett som er forskjellige både i innhold og størrelse. Målinger av tidsforbruk er imidlertid godt egnet til å sammenligne forskjellige algoritmer (og forskjellige implementasjoner) for samme oppgave. Det bør imidlertid være signifikant forskjell i tidsforbruket før vi kan si at en algoritme (eller en implementasjon) er mer effektiv enn en annen.

En mer objektiv (dvs. uavhengig av ytre faktorer) måte å vurdere en algoritmes effektivitet på er å se på de arbeidsoperasjonene som utføres i algoritmen. Da kunne vi si at en algoritme er effektiv hvis den relativt sett (dvs. med hensyn på oppgavens størrelse) utfører «få arbeidsoperasjoner».

Vi kan kalle flg. operasjoner for grunnleggende arbeidsoperasjoner:

- en tilordning - en variabel får en verdi
- en tabelloperasjon - en tabellverdi aksesseres
- en sammenligning - to verdier sammenlignes
- en regneoperasjon - f.eks. en addisjon av to tall

Dominerende operasjoner

Det er ikke vanlig å ta med alle operasjonene når en vurderer en algoritmes effektivitet. Det er ofte slik at én eller noen av operasjonene er viktigere, mer sentrale eller mer kostbare enn de andre. Slike operasjoner kalles dominerende. En algoritmes orden Resultatet av opptellingen (antall ganger den dominerende operasjonen utføres) avhenger av oppgavestørrelsen

Algoritmeeffektivitet - oppsummering:

- En algoritme er en beskrivelse av de arbeidsoperasjonene som skal til for å løse en bestemt oppgave.

- Den har vanligvis en dominerende operasjon - f.eks. det å sammenligne.

- Vi kan «telle opp» antallet ganger den dominerende operasjonen utføres.

- Antallet kan ofte uttrykkes som en funksjon av oppgavens størrelse n. Denne funksjonen definerer algoritmens orden.

- Algoritmens orden er en målestokk for dens effektivitet.

- Jo «mindre» en algoritmes orden er, jo mer effektiv er den. F.eks. er lineær orden generelt mer effektivt enn kvadratisk orden.

- Maks-metoden har lineær orden (orden n). Den er effektiv. Det er umulig å finne den største i en usortert tabell med færre enn n - 1 sammenligninger.

Algoritmeanalyse

Algoritmeanalyse (eng: analysis of algorithms) handler om å studere algoritmers egenskaper ved hjelp av matematiske og statistiske teknikker. Når vi skal vurdere effektiviteten til en algoritme skiller vi normalt mellom gjennomsnittlig effektivitet og effektiviteten i det mest ugunstige tilfellet.

Tabellintervaller

Halvåpent intervall: Parametrene fra og til er grensene for det halvåpne tabellintervallet **a[fra:til>**. Det består av elementene i a fra og med fra og til (men ikke med) til.

Det må stilles bestemte krav til fra og til for at intervallet a[fra:til> skal ha mening:

- Posisjonen/indeksen fra kan ikke være negativ siden 0 er første lovlige indeks.
- Posisjonen til kan ikke være større enn a.length, dvs. tabellens lengde. Tabellen i
- Posisjonen fra kan ikke være større enn til, dvs vi kan ikke ha fra til høyre for til.

Vi kaller a[fra:til> et lovlig (halvåpent) tabellintervall hvis 1, 2 og 3 er oppfylt. Vi sier at det er tomt (men lovlig) hvis fra = til. På samme måte som i mengdelære der den tomme mengden har mening, kan det gi mening å ha et tomt intervall.

Lukket intervall: I noen situasjoner er det mest naturlig at en tabellmetode arbeider i et lukket tabellintervall. La v og h betegne intervallets venstre og høyre endepunkt. Et lukket tabellintervall betegnes med **a[v:h]** og består av elementene i a fra og med indeks v til og med indeks h.

Det må stilles krav til intervallendepunktene v og h for at a[v:h] skal bli et lovlig intervall:

- Venstre endepunkt v kan ikke være negativt.
- Høyre endepunkt h kan ikke være større enn eller lik a.length.
- Vi må ha at $h >= v - 1$.
- Hvis kravene 1, 2 og 3 er oppfylt, kaller vi a[v:h] et lovlig (lukket) tabellintervall. Vi får et spesialtilfelle hvis $h = v - 1$, dvs. hvis h ligger rett til venstre for v. Da er a[v:h] et tomt (men lovlig) tabellintervall. Hvis $v = h$ inneholder a[v:h] nøyaktig én verdi.

Legg merke til at det lukkede intervallet a[v:h] blir lik det halvåpne intervallet a[$v : h + 1$>, og at det halvåpne intervallet a[*fra*:*til*> blir lik det lukkede intervallet a[*fra* : *til* − 1].

Huskeregel: En tabellmetode som skal arbeide med et halvåpent tabellintervall, skal ha parameternavnene fra og til i sin signatur:

```
. . . . metode1(int[] a, int fra, int til, . .)  // a[fra:til>
```

Hvis den skal arbeide med et lukket tabellintervall, skal den ha parameternavnene v (v står for venstre) og h (h står for høyre) i sin signatur:

```
. . . . metode2(int[] a, int v, int h, . .)     // a[v:h]
```

Gjennomsnittlig og verste tilfellet

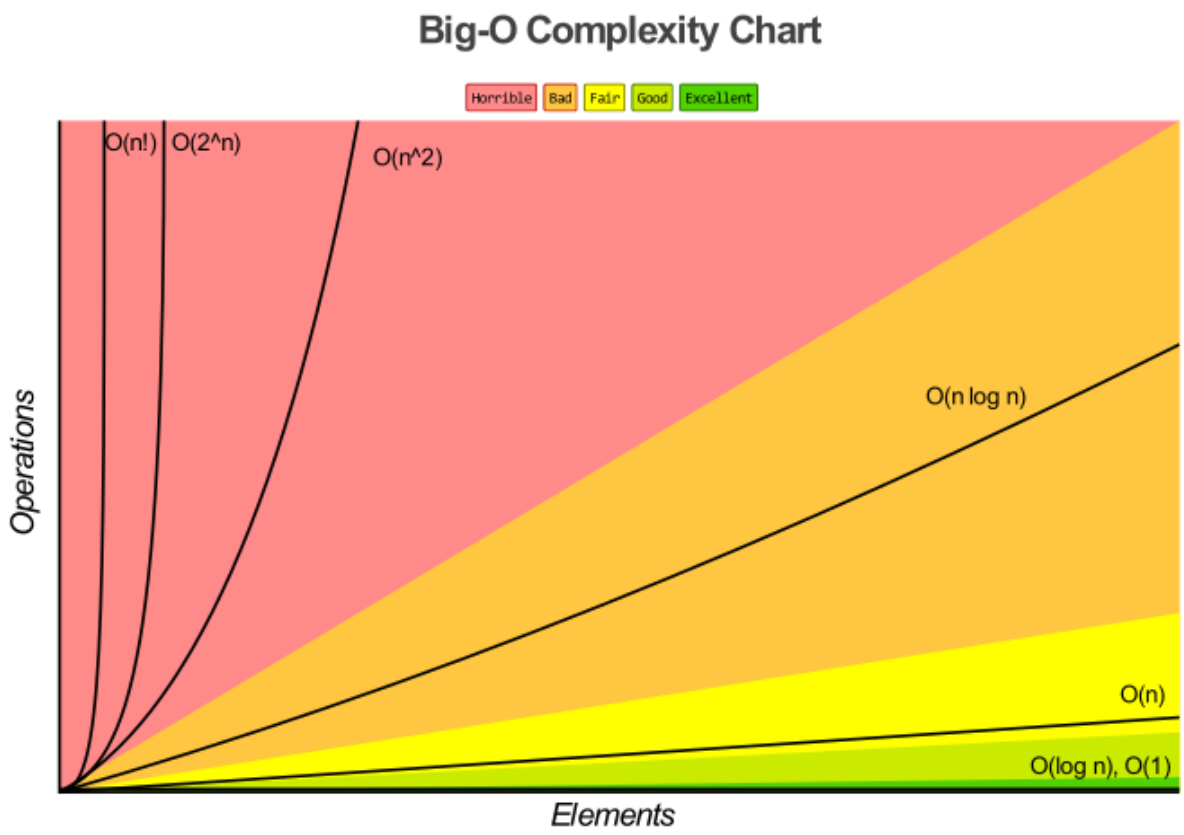
Når det gjelder effektivitet skal vi skille mellom følgende tre tilfeller:

- Den gjennomsnittlige effektiviteten (eng: average case)
- Effektiviteten i det mest ugunstige eller verste tilfellet (eng: worst case)
- Effektiviteten i det beste tilfellet (eng: best case)

Analyse av gjennomsnittlig effektivitet

Kort om hva som kjennetegner de ulike Big O. Feks $O(n^2)$ som er n-1,n-2, etc. Hvordan finne dette?

Big O cheat sheet



Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

Søking

Det å finne en verdi i en samling verdier er en av de mest grunnleggende oppgavene i databehandling.

Gjennomsnittlig antall sammenlikninger

Søkealgoritme	**Det gjennomsnittlige antallet sammenligninger**					
Navn	**Formel**	**n = 10**	**n = 100**	**n = 10.000**	**n = 1.000.000**	**Orden**
Lineærsøk	$(n + 1)/2 + 2$	7,5	52,5	≈ 5 000	≈ 500 000	n
Kvadratrotsøk	$\sqrt{n} + 2$	5,2	12	102	1002	$\sqrt{n}$
Binærsøk, 1. versjon	$2 \cdot \log_2(n+1) - 3$	4,8	10,3	23,6	37	$\log_2(n)$
Binærsøk, 2. versjon	$1{,}5 \cdot \log_2(n+1) - 1$	4,8	9,0	18,9	29	$\log_2(n)$
Binærsøk, 3. versjon	$\log_2(n) + 1$	4,4	7,7	14,3	21	$\log_2(n)$

Usortert søk

Hvis tabellen er usortert, er det ingen annen måte å gjøre det på enn å se på én og én verdi. Da vil vi, hvis tabellen inneholder den vi leter etter, før eller senere finne den. Hvis vi har sett på alle verdiene uten å finne den, kan vi konkludere med at den ikke er der.

Flg. metode returnerer posisjonen til søkeverdien hvis den ligger i tabellen, og returnerer –1 hvis den ikke er der. Hvis det er flere forekomster av søkeverdien, returneres posisjonen til den første av dem (fra venstre). Tabellen og søkeverdien inngår som parametere:

```
public static int usortertsøk(int[] a, int verdi){  // tabell og søkeverdi
    for (int i = 0; i < a.length; i++)  // går gjennom tabellen
      if (verdi == a[i]) return i;      // verdi funnet - har indeks i

    return -1;                          // verdi ikke funnet
  }
```

Det finnes ingen mer effektiv teknikk enn dennne koden for å søke i en usortert tabell. La n være tabellengden. Hvis verdien ikke er der, vil if (verdi == a[i]) utføres n ganger. Men hva hvis den er der? Vi antar at alle verdiene er forskjellige og har samme sannsynlighet for å bli etterspurt. Hvis verdi ligger først, trengs én sammenligning, to hvis den ligger nest først, osv. Tilsammen $1 + 2 + \cdots + n = n(n + 1)/2$ stykker og dermed blir det $n(n + 1)/2n = (n + 1)/2$ som gjennomsnitt. Søkemetoden er derfor av orden n.

Sortert søk (lineærsøk)

Hvis tabellen er sortert (f.eks. stigende), kan vi forbedre søkealgoritmen. Den mest optimale teknikken kalles binærsøk.

Vi kan slik som i koden for usortert søk, se på én og én tabellverdi og returnere posisjonen så fort vi finner den vi leter etter. Hvis ikke, kan vi, siden tabellen er sortert, avbryte letingen så fort vi kommer til en verdi som er for stor. Ta som eksempel at vi leter etter 31 i flg. tabell:

3	8	10	12	14	16	21	24	27	30	32	33	34	37	40
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Her stopper vi på 32. Vi kunne nå returnere –1 som et signal om at vi ikke fant 31. Men her er det mulig å slå «to fluer i en smekk». En negativ returverdi kan signalisere at verdien ikke er der. Men det vil også være fordelaktig å kunne rapportere hvilken posisjon/indeks verdien ville ha hatt dersom den hadde ligget der, dvs. verdiens innsettingspunkt. Det er der den måtte ha blitt satt inn for å få bevart sorteringen. Her er det posisjon eller indeks 10.

Innsettingspunkt: En verdi som ikke ligger i en (stigende) sortert tabell, ville hvis den hadde ligget der, hatt en bestemt posisjon p. Denne posisjonen kalles verdiens innsettingspunkt (eng: insertion point).

Gitt en verdi som ikke er i tabellen. Hvis den er mindre enn den minste, dvs. mindre enn den som ligger lengst til venstre, blir dens innsettingspunktet p lik 0. Dermed kan vi ikke bruke –p som returverdi siden –0 og 0 er det samme. Returverdien skal alltid være negativ for verdier som ikke ligger i tabellen. Vi returnerer isteden –(p + 1). Den blir alltid negativ (også når p er 0). La videre k = –(p + 1). Da vil p = –(k + 1). Med andre ord kan vi, hvis søkeverdien ikke ligger i tabellen, beregne innsettingspunktet ved hjelp av returverdien.

```
public static int lineærsøk(int[] a, int verdi)
  {
    if (a.length == 0 || verdi > a[a.length-1])
      return -(a.length + 1);  // verdi er større enn den største

    int i = 0; for( ; a[i] < verdi; i++);  // siste verdi er vaktpost

    return verdi == a[i] ? i : -(i + 1);   // sjekker innholdet i a[i]
  }
```

Gitt int[] a = {3,8,10,12,14,16,21,24,27,30,32,33,34,37,40}; så returnerer metoden 10 når det søkes etter 32. Det stemmer siden 32 har indeks/posisjon 10. Returverdien blir –11 når det søkes etter 31. En negativ returverdi forteller at den søkte verdien ikke ligger i tabellen. Tallet 31 hører hjemme mellom 30 og 32. Innsettingspunktet blir derfor lik posisjonen til 32, dvs. 10. Men vi kan beregne oss frem til det ved hjelp av returverdien. Generelt gjelder at hvis k er en negativ returverdi, blir innsettingspunktet lik –(k + 1). I eksemplet: –(–11 + 1) = 10.

Med tanke på effektivitet er lineærsøk litt bedre enn usortertsøk for verdier som ikke ligger i tabellen. Den går i gjennomsnitt kun halvveis gjennom tabellen for å komme til første verdi som er for stor. Det er andre teknikker som er vesentlig mer effektive, men lineærsøk er likevel viktig. Den er god nok for små tabeller. Men det er selve idéen som er viktig siden den også vil virke for datastrukturer som kan «leses» kun én vei (f.eks. en pekerkjede).

Det er også mulig å forbedre lineærsøk, f.eks. ved å gjøre en serie «hopp» under søkingen. La tabellen a inneholde 100 verdier. Da kan vi f.eks. se på hver 10-ende verdi. Når vi passerer stedet der søkeverdien hører hjemme, vet vi at den vil måtte befinne seg blant de 10 siste vi hoppet over. Dermed kan vi lete etter søkeverdien blant dem. Bruker vi en «hopplengde» lik kvadratroten av tabellens lengde, vil algoritmen bli av orden kvadratrot. Da kalles algoritmen for **kvadratrotsøk**:

```
 public static int kvadratrotsøk(int[] a, int verdi) {

    return lineærsøk(a,(int)Math.sqrt(a.length),verdi);
  }
```

Linærintervallsøk kan brukes hvis man vet intervallet som søkeverdien befinner seg i. Det vil selvfølgelig være mer effektivt til å søke. Metoden under returnerer et nytt array med tallene innenfor intervallet.

```
public static int[] lineærIntervallsøk(int[] a, int fraverdi, int tilverdi) {

    if (a.length == 0 || fraverdi > a[a.length - 1])
      return new int[0];  // returnerer en tom tabell

    int fra = 0;
    while (a[fra] < fraverdi) fra++;

    int til = a.length;

    if (tilverdi <= a[a.length - 1]) {
      til = fra;
      while (a[til] < tilverdi) til++;
    }

    // intervallet a[fra:til> inneholder de søkte verdiene
    return Arrays.copyOfRange(a, fra, til);
  }
```

```
  public int sortertSøk(int[] a, int value) {
    for (int i = 0; i < a.length; i++) {
        if(a[i] == value) {
            return i;
    }
    return -1;
  }

  public int sortertSøk2(int[] a, int value) {      //Mer effektiv - "hopper" over verdier
    int step-length = 2;      //Steglengde kan varieres etter størrelse. Bruker 2 i dette tilfelle
    for (int i = 0; i < a.length; i = i+2) {
        if(a[i] >= value) {
            return i;
    }
    return -1;
  }
```

Binærsøk

Binærsøk baserer seg på konseptet splitt og hersk. Det går raskere å søke i halvparten enn i hele tabellen. Vi kan hoppe inn midt i en tabell. Hvis verdien ikke er der, vil sorteringen avgjøre på hvilken side vi skal lete videre. Da får vi halvert søkeområdet. Idéen kalles binærsøk.

Vi skal generelt søke etter en verdi i et lukket intervall a[v:h] der v står for venstre og h for høyre endepunkt. I tillegg har vi midten m = (v + h)/2. Vi kan tenke på én av to måter:

1. Verdien ligger enten på midten eller på en av sidene a[v:m–1] eller a[m+1:h]. Søkeområdet blir dermed delt i tre - ett element og to lukkede delintervaller.

2. Alternativt kan vi nøye oss med å dele søkeområdet i to deler. Dvs. vi avgjør om den søkte verdien ligger i a[v:m] eller i a[m+1:h].

Vi starter med 1. måte. Flg. tabellintervall a[v:h] med 15 verdier er gitt. Anta at vi skal finne verdien 30. Fra starten av er hele søkeområdet markert med grå:

3	8	10	14	14	16	21	24	27	30	32	33	34	37	40
v							m							h

Midten er gitt ved m = (v + h)/2. Tabellen viser at a[m] er lik 24. Vår søkeverdi 30 er større. Den må da, hvis den er der, ligge til høyre for m. Vi setter v = m + 1 og fortsetter:

3	8	10	14	14	16	21	24	27	30	32	33	34	37	40
								v			m			h

På nytt er m = (v + h)/2. Søkeverdien 30 er heller ikke nå lik a[m] (dvs. 33), men derimot mindre. Dermed må den ligge til venstre for m. Nå settes h = m – 1:

3	8	10	14	14	16	21	24	27	30	32	33	34	37	40
								v	m	h				

Nå ser vi at søkeverdien 30 er lik a[m]. Vi kan avslutte søkingen og returnere posisjonen.

Binærsøk algoritme

1. Start med tabellintervallet a[v:h] og søkeverdien verdi.
2. Finn midten m = (v + h)/2.
3. Hvis verdi == a[m] , er vi ferdige!
4. Hvis verdi > a[m] , settes v = m + 1 og hvis ikke, settes h = m – 1.
5. Hvis v <= h, gå til 2. Hvis v > h, er a[v:h] tom og verdi er ikke i tabellen.

Med v > h blir intervallet a[v:h] tomt. Det betyr at verdi ikke er i tabellen. Men da blir v innsettingspunktet til verdi.

Binærsøk kode

```
  public static int binærsøk(int[] a, int fra, int til, int verdi) {
    int v = fra, h = til - 1;  // v og h er intervallets endepunkter

    while (v <= h) {    // fortsetter så lenge som a[v:h] ikke er tom
      int m = (v + h)/2;      // heltallsdivisjon - finner midten
      int midtverdi = a[m];   // hjelpevariabel for midtverdien

      if (verdi > midtverdi) v = m + 1;        // verdi i a[m+1:h]
      else if (verdi < midtverdi) h = m - 1;   // verdi i a[v:m-1]
      else return m;                           // funnet


    }

    return -(v + 1);    // ikke funnet, v er relativt innsettingspunkt
  }


  public static int binærsøk(int[] a, int verdi) {  // søker i hele a
    return binærsøk(a,0,a.length,verdi);  // bruker metoden over
  }
```

<u>Basert på metode 2 beskrevet over:</u>

```
public static int binærsøk(int[] a, int fra, int til, int verdi){

 int v = fra, h = til - 1;  // v og h er intervallets endepunkter

    while (v < h) {  // obs. må ha v < h her og ikke v <= h
      int m = (v + h)/2;  // heltallsdivisjon - finner midten

      if (verdi > a[m]) v = m + 1;   // verdi må ligge i a[m+1:h]
      else  h = m;                   // verdi må ligge i a[v:m]
    }
    if (h < v || verdi < a[v]) return -(v + 1);  // ikke funnet
    else if (verdi == a[v]) return v;            // funnet
    else  return -(v + 2);                       // ikke funnet
  }
```

Forskyvinger og rotasjoner

Gitt følgende tabell:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Tabellforskyvninger. En forskyvning (eng: array shift) på f.eks. 3 enheter mot høyre i tabellen over, vil føre til at de tre siste verdiene forsvinner:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

Tabellinnholdet er forskjøvet tre enheter mot høyre

I tabellen over har ikke de tre første rutene noe innhold. Men de hører til tabellen og må derfor ha et innhold. Vi kunne f.eks. tenke oss at verdiene er 0. Men det er kanskje mer naturlig å kreve at de skal inneholde den verdien som opprinnelig lå først. Dvs. slik:

1 1 1 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

Det er fylt på fra venstre med verdien som opprinnelig lå først

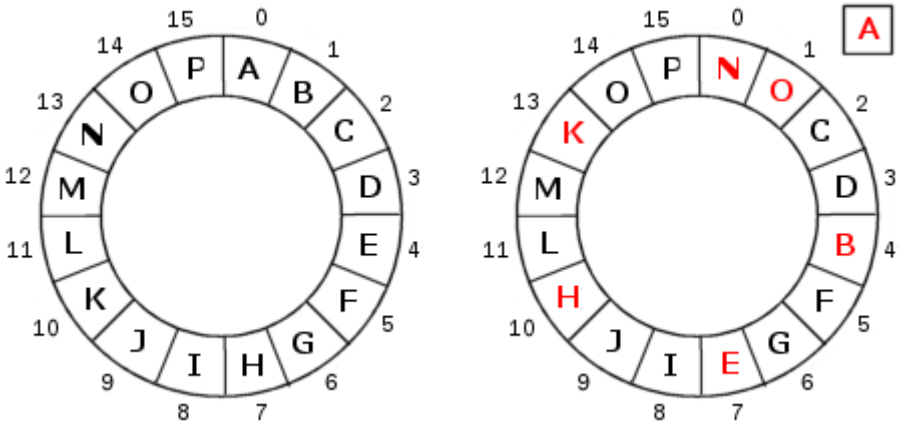
Dette svarer til bitskyvning av bitene i et heltall. Der er det to typer bitforskyving mot høyre: >> og >>>. Operatoren >> kalles en fortegnsbevarende shiftoperator. Det betyr at det fylles på fra venstre med den biten som opprinnnelig lå lengst til venstre. Mens >>> fyller alltid på 0-biter. Setningen: int k = -1; gjør at k får 32 1-biter. Setningen: k >>= 3; forskyver bitene i k tre enheter mot høyre. Men siden den første biten er 1, fylles det på med 1-biter. Det betyr at k fortsatt er lik −1.

Kodeeksempel:

```
public static void rightshift(int[] a, int d) {
   if (d < 0) throw new IllegalArgumentException("Negativ parameter!");

   if (a.length <= 1) return;              // a har maksimalt ett element
   d %= a.length;                          // d < a.length

   for (int i = a.length - 1; i >= d; i--) a[i] = a[i - d];  // forskyver
   for (int i = 1; i < d; i++) a[i] = a[0];  // kopierer inn a[0]
 }
```

En rotasjon i en tabell er en forskyvning på en slik måte at de elementene som skyves ut kommer inn i samme rekkefølge i den andre enden av tabellen. Hvis vi tenker oss at tabellen bøyes til en sirkel slik at indeks 0 og tabellens siste indeks kommer ved siden av hverandre, kan vi se på dette som en rotasjon. Vi kan tenke på dette som et «lykkehjul». Indeksene holdes fast, mens «hjulet» kan snurre både med og mot klokken.

En rotasjon på f.eks. 8 enheter får vi ved å rotere hjulet med klokken slik at bokstaven A kommer ned til indeks 8. En rotasjon på f.eks. −5 enheter får vi ved å rotere hjulet 5 enheter mot klokken slik at A kommer til indeks 11. Dette kunne vi også ha fått til ved å rotere 11 enheter med klokken. Det kommer av at en rotasjon på k enheter mot klokken er det samme som en rotasjon på 16 − k enheter med klokken der 16 er tabellens lengde.



Koden over er effektiv nok hvis tabellen skal roteres få enheter. Men med mange enheter mot høyre (med klokken) eller få enheter mot venstre (mot klokken), er den mindre effektiv. F.eks. utføres en rotasjon på én enhet mot venstre som en rotasjon på a.length − 1 enheter mot høyre. Da må hjelpetabellen ha samme lengde og dermed svært mye kopiering. Det er imidlertid mulig å lage kode der hjelpetabellen maksimalt blir halvparten av a.length. Men målet er å lage en rotasjonsmetode som ikke bruker en hjelpetabell.

Anta at vi skal rotere tabellen under (med 16 verdier) 10 enheter mot høyre:

A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Vi starter med å snu rekkefølgen på elementene i tabellen. Det gir oss flg. resultat:

P	**O**	**N**	**M**	**L**	**K**	**J**	**I**	**H**	**G**	**F**	**E**	**D**	**C**	**B**	**A**
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Så snur vi rekkefølgen på de 10 første og deretter på de 16 - 10 = 6 siste:

G	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**A**	**B**	**C**	**D**	**E**	**F**
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Tabellen er rotert 10 enheter mot høyre

Tabellen har blitt rotert 10 enheter mot høyre. For å innse at dette alltid gir korrekt resultat er det enklest å tenke motsatt vei. Anta derfor at vi starter med resultatet, dvs. med at tabellen er blitt rotert 10 enheter mot høyre. De 6 siste elementene er da de som opprinnelig lå først. Snur vi disse kommer de som de 6 siste i en snudd tabell. Snur vi de 10 første kommer disse som de 10 første i en snudd tabell. Hvis vi så avslutter med å snu tabellen, vil vi dermed få den opprinnelige tabellen. Denne argumentasjonen gjelder generelt, dvs. ved rotasjon på et bestemt antall enheter i en vilkårlig tabell.

Vi snur en tabell eller et tabellinetvall ved hjelp av en serie ombyttinger. Da kan det være praktisk med en egen ombyttingsmetode.

Kodeeksempel:

```
 public static void bytt(char[] a, int i, int j) {
    char temp = a[i]; a[i] = a[j]; a[j] = temp;
  }

public static void rotasjon(char[] a, int d) {             // 2. versjon
    int n = a.length;
    if (n < 2) return;                                     // tomt eller en verdi

    if ((d %= n) < 0) d += n;                              // motsatt vei?

    for (int v = 0, h = n - 1; v < h; Tabell.bytt(a, v++, h--));  // snur a[a:n>
    for (int v = 0, h = d - 1; v < h; Tabell.bytt(a, v++, h--));  // snur a[0:d>
    for (int v = d, h = n - 1; v < h; Tabell.bytt(a, v++, h--));  // snur a[d:n>
  }
```

Leksikografisk permutasjon

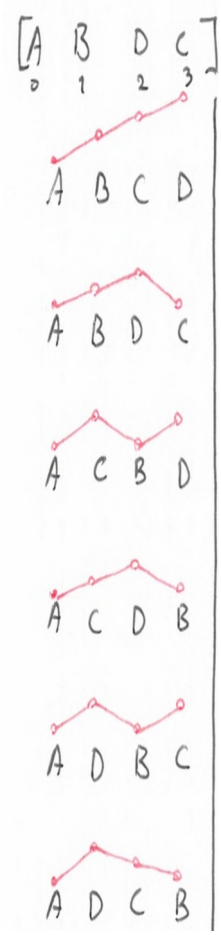
Gitt [ABCD]. Hvis ABCD hadde vært et ord i ordboka, hva hadde neste ord med de samme bokstavene vært?

Da ville vi fått [ABDC] fordi de to første bokstavene er like, mens ved å stokke om får vi at D kommer etter C (2. posisjon). Fortsetter vi videre må vi bytte om posisjonen før og får [ACBD]. Neste der igjen blir [ACDB]. Da får vi følgende 12 permutasjoner av [ABCD]:

[ABCD], [ABDC], [ACBD], [ACDB], [ADBC], [ADCB],
[BACD], [BADC], [BCAD], [BCDA], [BDAC], [BDCA]

Totalt kan vi få 4! permutasjoner = 24

Hvis vi gir en verdi til bokstavene, hvor A = 1, B = 2, osv. så kan vi tegne en strek som viser "verdien" av hver bokstav (figur til høyre)



Algoritme: Neste permutasjon i leksikografisk sortering.

1) Begynn bakerst og finn første element som ikke er større enn det bak	Knekkpunkt A D C B
2) Finn det minste av elementene til høyre som er større enn dette	A D C B
3) Bytt om de to elementene	B D C A
4) Reverser rekkefølgen til høyre for knekkpunkt	B A C D

Definisjon:

Gitt to permutasjoner av 1 til n

$p = [p_0, p_1, p_2...., p_n]$
$q = [q_0, q_1, q_2...., q_n]$

1) p og q <u>er like</u> hvis p_i og q_i for alle i
2) La k være minste indeks, slik at $p_k \neq q_k$. Da er $p < q$ leksikografisk hvis $p_k < q_k$

Hack

a <-> 0, B <-> 1, ... -> [ADCB] = 0321
Neste permutasjon tallet du kan lage med de samme siffrene som er større. 1023 = [BACD]

Sortering

Om sortering og ulike sorteringsalgoritmer.

Partisjonering

Å partisjonere en tabell betyr å omorganisere og ordne den i deler etter bestemte kriterier.

Eks:

Flg. tabell inneholder kun tallene 0 og 1:

0	0	1	0	0	1	0	1	1	0	0	1	1	1	0	1	0	1	1	0

Vi ønsker å «partisjonere» den slik at alle 0-ene kommer i første halvdel og dermed alle 1-erne i andre halvdel. Målet er å gjøre det med minst mulig innsats.

Todelt partisjonering: Målet er å finne en algoritme som omorganiserer en tabell slik at alle verdier som er mindre enn en skilleverdi kommer først.

La f.eks. 10 være skilleverdi. To indekser v og h starter i hver sin ende av tabellen:

8	3	15	13	1	9	20	3	18	2	6	25	14	8	20	16	5	21	11	14
v																			h

Det starter ved at v flyttes mot høyre så lenge som tilhørende tabellverdi er mindre enn 10. Da vil v stoppe ved 15. Så flyttes h mot venstre så lenge som tilhørende tabellverdi er større enn eller lik 10. Da vil h stoppe ved 5. De verdiene som v og h har passert får hvit bakgrunn:

8	3	15	13	1	9	20	3	18	2	6	25	14	8	20	16	5	21	11	14
		v														h			

Tabellen er nå (og vil inntil videre være) tredelt. Venstre del (hvit bakgrunn) har verdier som er mindre enn 10 og høyre del (også hvit bakgrunn) verdier som er større enn (eller lik) 10. Den midterste delen (grå bakgrunn) inneholder de «ukjente» verdiene, dvs. de som ennå ikke er undersøkt. Indeksene v og h skal hele tiden ligge i hver sin ende av den «ukjente» delen.

I tabellen over har indeksen *v* stoppet på en verdi (15) som er større enn 10 og *h* på en verdi (5) som er mindre enn 10. Hvis de to bytter plass, vil 5 høre til den venstre «hvite» delen og 15 til de som er større enn (eller lik) 10. Men da må også *v* og *h* flyttes videre:

8	3	5	13	1	9	20	3	18	2	6	25	14	8	20	16	15	21	11	14
			v												h				

Nå står allerede v på en verdi (13) som er større enn 10 og må stå der. Men h må flyttes videre mot venstre til første verdi som er mindre enn (eller lik) 10:

8	3	5	13	1	9	20	3	18	2	6	25	14	8	20	16	15	21	11	14
			v										h						

Verdiene til v og h (13 og 8) bytter plass, v og h flyttes én posisjon:

8	3	5	8	1	9	20	3	18	2	6	25	14	13	20	16	15	21	11	14
				v								h							

Vi fortsetter til den «ukjente» (grå) delen er tom. Da vil v høre til den første av de som er større enn (eller lik) 10 og h til den siste av de som er mindre enn 10:

8	3	5	8	1	9	6	3	2	18	20	25	14	13	20	16	15	21	11	14
								h	v										

Legg merke til at det i tabellen over ikke er noen spesiell rekkefølge på tallene i venstre del (de mindre enn 10) og heller ingen spesiell rekkfølge på de i høyre del.

Spesialtilfeller: Hvis alle er mindre enn skilleverdi, vil v havne utenfor når den flyttes mot høyre. Omvendt vil h havne utenfor til venstre hvis ingen er mindre enn skilleverdi.

Partisjoneringskode

```
 private static int parter0(int[] a, int v, int h, int skilleverdi) {
    while (true)                                    // stopper når v > h
{
      while (v <= h && a[v] < skilleverdi) v++;   // h er stoppverdi for v
      while (v <= h && a[h] >= skilleverdi) h--;  // v er stoppverdi for h

      if (v < h) bytt(a,v++,h--);                 // bytter om a[v] og a[h]
      else  return v;  // a[v] er nåden første som ikke er mindre enn skilleverdi
    }
  }
```

De to while-løkkene over inneholder v <= h. Hvis alle verdiene i a[v:h] er mindre enn skilleverdi, vil v stoppe på h + 1. Omvendt vil h stoppe på v − 1 hvis ingen av dem er mindre enn skilleverdi. Hvis vi ikke har noen av disse tilfellene, vil første ombytting gjøre at a[v] er mindre enn og a[h] større enn eller lik skilleverdi. De to blir da stoppverdier eller vaktposter. Dermed kan vi fjerne v <= h i resten av algoritmen.

En vilkårlig tabellverdi, f.eks. a[indeks], kan være skilleverdi i parter0(). To ombyttinger (en før og en etter) får den inn på rett sortert plass

```
  private static int sParter0(int[] a, int v, int h, int indeks) {
    bytt(a, indeks, h);           // skilleverdi a[indeks] flyttes bakerst
    int pos = parter0(a, v, h - 1, a[h]);  // partisjonerer a[v:h - 1]
    bytt(a, pos, h);              // bytter for å få skilleverdien på rett plass
    return pos;                   // returnerer posisjonen til skilleverdien
  }
```

Det er ingen ekstra sammenligninger i sParter0(). Hvis intervallet a[v:h] har n verdier, blir det n − 1 stykker i parter0() siden kallet skjer på a[v:h-1]. Det blir to ekstra ombyttinger. Det kan vises at hvis sParter0() brukes på en permutasjon av tallene fra 1 til n og med en tilfeldig indeks, vil gjennomsnittlig antall ombyttinger bli eksakt 2 + (n − 2)/6 = (n + 10)/6.

Effektivitet

La tabellen inneholde en vilkårlig permutasjon av tallene fra 1 til n og la s være et av dem. Da vil antallet tall blant de s − 1 første som ikke er mindre enn s (dvs. større enn eller lik s), være det samme som antallet ombyttinger med s som skilleverdi i partisjoneringsalgoritmen.

Det gjennomsnittlige antallet ombyttinger i algoritmen for partisjonering, der gjennomsnittet tas over alle permutasjoner av tallene fra 1 til n og over alle skilleverdier s fra 1 til n, er eksakt lik (n2 − 1)/6n.

En verdi i en tabell står på rett sortert plass hvis alle elementene til venstre for den er mindre enn eller lik og alle elementene til høyre for den er større enn eller lik.

Quicksort (Kvikksortering)

Kvikksortering er en systematisk anvendelse av sParter0(). Først velges en tabellverdi som skilleverdi. Et kall på sParter0() får den inn på rett sortert plass k. Dette gjentas på hver side av k. Osv. Det beste er en skilleverdi som gjør de to sidene noenlunde like i størrelse. Men med en ukjent tabell er det ingen annen mulighet enn å velge vilkårlig. F.eks. den første, den midterste eller den siste. En bedre strategi er å ta den «midterste» av de tre.

Eks:

Vi velger den på midten av a[v:h] (m = (v + h)/2) som skilleverdi:

13	2	8	10	16	9	15	4	18	14	12	11	7	5	3	6	17	1	20	19
v									m										h

bruker vi verdien på midten, dvs. a[m] der m = (v + h)/2, som skilleverdi

Metodekallet sParter0(a,v,h,(v + h)/2) vil gjøre at tabellen blir slik:

13	2	8	10	1	9	6	4	3	5	12	11	7	14	18	15	17	16	20	19
v													k						h

Tallene i a[v:k − 1] er mindre enn 14 og de i a[k + 1:h] større enn 14

De «grå» delene a[v:k−1] og a[k+1:h] er separate deler. Hvis a[v:k−1] partisjoneres mhp. midtverdien (6), vil dens verdier fortsatt ligge til venstre for 14. Hvis a[k+1:h] partisjoneres mhp. dens midtverdi (17) blir det tilsvarende. Gjør vi begge deler, får vi flg. resultat:

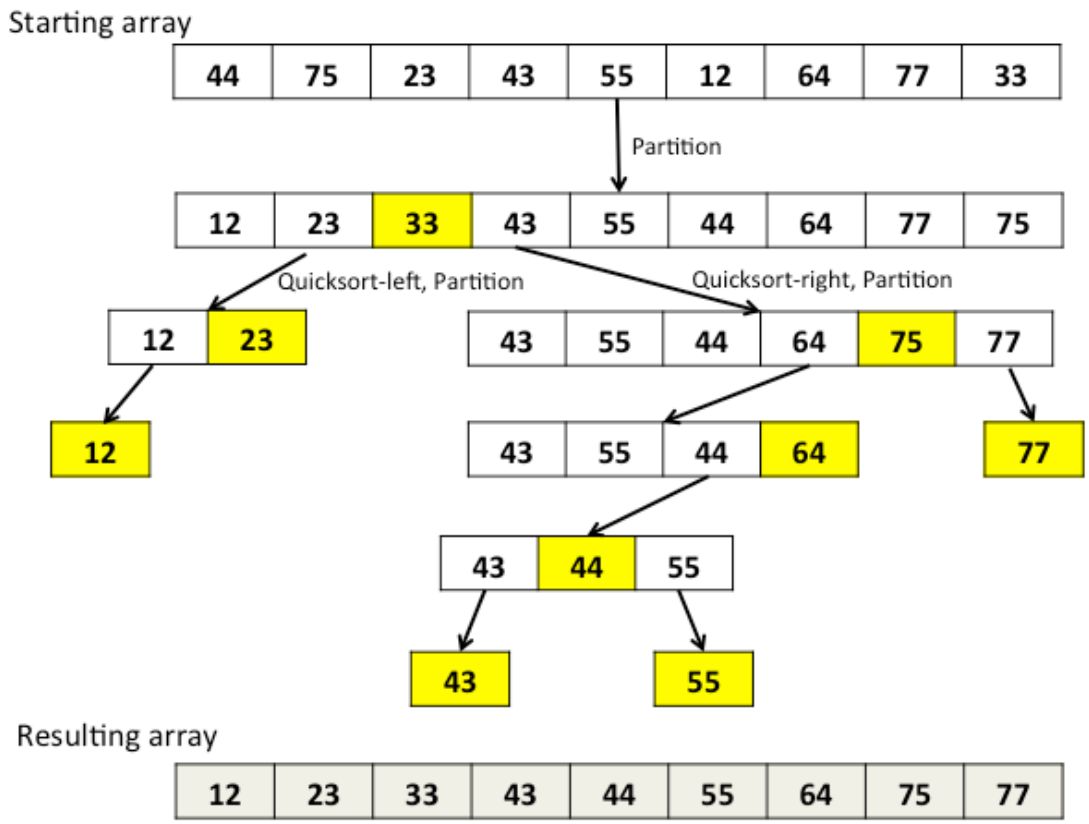
5	2	3	4	1	6	7	10	8	13	12	11	9	14	16	15	17	18	20	19

Tallene 6, 14 og 17 ligger alle nå på rett sortert plass

Dette fortsetter ved at sParter0() kalles på hver av de fire «grå» delene. Osv. Hvis et intervall (en «grå» del) er tomt eller har kun ett element, stopper vi siden det er sortert.

Kvikksortering kaller seg selv to ganger. Det kalles rekursjon. Metodekallene stopper når vi får v >= h, og det vil før eller senere skje siden kallene utføres på intervaller som er kortere enn a[v:h]. Det ene kan i verste fall være tomt. Det andre er dermed bare én kortere enn a[v:h]. Det gjør at metoden i verste fall blir av kvadratisk orden. Men i gjennomsnitt vil lengden til intervallene a[v:k−1] og a[k+1:h] ikke skille seg så mye. Det gjør at kvikksortering i gjennomsnitt er av orden n log2(n).

Quicksort illustrasjon



Quicksort kode

```
  private static void kvikksortering0(int[] a, int v, int h)
  {
    if (v >= h) return;  // a[v:h] er tomt eller har maks ett element
    int k = sParter0(a, v, h, (v + h)/2);  // bruker midtverdien
    kvikksortering0(a, v, k - 1);     // sorterer intervallet a[v:k-1]
    kvikksortering0(a, k + 1, h);     // sorterer intervallet a[k+1:h]
  }

public static void kvikksortering(int[] a)   // sorterer hele tabellen
  {
    kvikksortering0(a, 0, a.length - 1);
  }
```

Oppsummering - quicksort:

- **Navn:** Kvikksortering (eng: quick sort) ble «oppfunnet» allerede i 1959 av briten Tony Hoare og er kjent som den beste (mest effektive) generelle sorteringsteknikken. I java.util er det en optimalisert versjon (a dual-pivot quicksort) som brukes for standardtypene (byte, short, int, long, float, double, char). Men for generiske typer brukes (i java.util) imidlertid flettesortering (eng: merge sort).

- **Idé og effektivitet:** Den kan ses på som en anvendelse av partisjonering, dvs. å dele en tabell (eller et tabellintervall) i to deler mhp. en skilleverdi (eng: pivot). Et enkelt valg av skilleverdi kan være den første eller siste verdien. Men det vil føre til at den får orden n^2 hvis tabellen allerede er sortert. Da er det bedre å velge den midterste. Men i optimaliserte versjoner gjøres valget på en «lurere» måte. Men uansett vil det være mulig (men ikke enkelt) å konstruere en tabell som gjør at den får orden n^2. Men i gjennomsnitt er den garantert av orden n log(n). En mulig optimaliseringsteknikk er å skifte til innsettingssortering for små intervaller (lengde mindre enn 47 brukes i java.util). Den er en «på plass»-algoritme (eng: in place), dvs. at den ikke trenger noen hjelpetabell.

- **Ulemper:** Den optimaliserte versjonen vil, som nevnt over, kunne være av kvadratisk orden i (svært) spesielle tilfeller. Hvis en trenger en teknikk som alltid er n log(n), må en velge noe annet - f.eks. heapsortering som alltid er av orden n log(n), men som ikke er fullt så god som kvikksortering i gjennomsnitt. Et mulig valg er å kombinere de to slik det er gjort i introsort. En (liten) ulempe er at den ikke er stabil, dvs. like verdier får ikke nødvendigvis samme innbyrdes rekkefølge etter sorteringen som de hadde før.

- **Konklusjon:** Dette vil normalt være førstevalget når en trenger en sorteringsalgoritme.

Quick search

Vi kan også bruke partisjonering til å finne den m-te minste verdien i en (usortert) tabell, dvs. den verdien som ville ha kommet på plass nr m hvis vi hadde sortert tabellen. F.eks. er den 0-te minste verdien det samme som den minste og den 1-te minste verdien det samme som den andre eller nest minste verdien.

Kvikksøk bruker idéen i kvikksortering. En partisjonering med midtverdien som skilleverdi, vil gjøre at den havner på rett sortert plass (indeks k). Hvis det er lik indeks m, er vi ferdige. Hvis ikke må vi lete videre på venstre side hvis m < k og på høyre side hvis m > k. Flg. metode flytter om på på verdiene i tabellen a slik at den m-te verdien til slutt havner på indeks m. Hvis tabellen a har lengde n, vil kvikksøk får orden n siden sParter0() kalles kun én gang i hver iterasjon.

```
public static int kvikksøk(int[] a, int m) {

  int v = 0, h = a.length - 1;  // intervallgrenser

  while (true) {
    int k = sParter0(a,v,h,(v + h)/2);
    if (m < k) h = k - 1;
    else if (m > k) v = k + 1;
    else return k;
  }
}
```

Medianen til en samling verdier er den som ville ha havnet på midten etter en sortering. Hvis antallet er odde, er dette veldefinert. Hvis ikke, defineres medianen som gjennomsnittet av de to verdiene på hver side av «midten». Vi kan bruke kvikksøk() til å finne medianen

Merge (Fletting)

Å flette (eng: merge) betyr å forene. F.eks. å tvinne sammen flere enheter til en felles enhet. Vi kan flette hår eller vi kan flette kvister til en kurv. Hvis en vei med to felter i samme retning snevres inn til kun ett felt, er det mest «rettferdig» at trafikken «flettes sammen».

I databehandling er det aktuelt å flette to eller flere sekvensielle enheter (f.eks. tegnstrenger, tabeller, lister eller filer) til en felles enhet av samme type. En enkel form for fletting av to enheter er at annenhver verdi kommer fra den ene og annenhver fra den andre. Hvis det er ulikt antall, må vi ha en regel for de som er «til overs». De kan f.eks. legges inn bakerst

Sortert fletting: Anta at vi har to sekvensielle datastrukturer, f.eks. to tabeller, som begge er sortert. En viktig oppgave er da å kunne flette dem sammen til en datastruktur som er sortert. Da blir det normalt feil bruke regelen om «annenhver verdi». Vi må gjøre det på en annen måte. Se på flg. eksempel med to heltallstabeller

a							**b**									
1	3	4	6	9	9	11	2	3	5	6	7	8	9	10	12	14
i							j									

c

k

Vi har to sorterte tabeller a og b med henholdsvis 7 og 10 verdier. De skal flettes sammen og legges i en tredje tabell c slik at c blir sortert. Den har plass til de 7 + 10 = 17 verdiene. Første posisjon (posisjon 0) i a, b og c er markert med henholdsvis i, j og k.

Algoritme: Sammenlign a[i] og b[j]. Hvis a[i] er mindre enn eller lik b[j], kopieres den over i posisjon k i c og både i og k økes med 1. Hvis ikke, dvs. hvis b[j] er minst, kopieres den over i posisjon k i c og både j og k økes med 1. Etter fem «runder» får vi:

a							**b**									
1	3	4	6	9	9	11	2	3	5	6	7	8	9	10	12	14
			i						j							

c

1	2	3	3	4
				k

Det er ofte aktuelt å kunne flette sammen to sorterte tabellintervaller, f.eks. de to halvåpne intervallene a[0:m> og b[0:n>. Flg. metode gjør dette og returnerer antallet. Tabellen c må være stor nok, dvs. ha plass til minst m + n verdier.

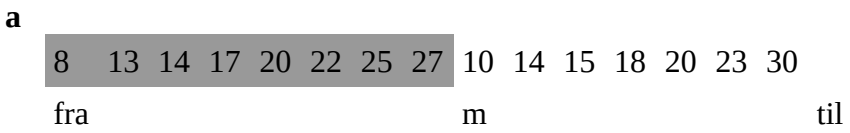
Flettekode

```
public static int flett(int[] a, int m, int[] b, int n, int[] c) {
  int i = 0, j = 0, k = 0;
  while (i < m && j < n) c[k++] = a[i] <= b[j] ? a[i++] : b[j++];

  while (i < m) c[k++] = a[i++];   // tar med resten av a
  while (j < n) c[k++] = b[j++];   // tar med resten av b

  return k;   // antallet verdier som er lagt inn i c
}

public static int flett(int[] a, int[] b, int[] c) {

  return flett(a, a.length, b, b.length, c);
}
```

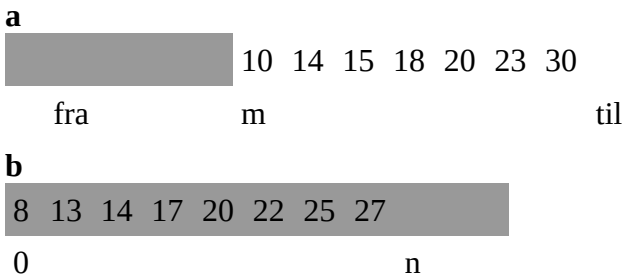
a[i] <= b[j] blir utført m + n – 1 ganger hvis tallene i a og b er slik at a[i] <= b[j] blir sann/usann annenhver gang. F.eks. hvis a = { 1, 3, 5, 7, 9 } og b = { 2, 4, 6, 8, 10 }. Hvis siste verdi i a er mindre enn første i b, vil a[i] <= b[j] bli utført m ganger, og hvis omvendt, n ganger. Normalt et sted mellom min(m , n) og m + n – 1. Det betyr at algoritmen har orden m + n i gjennomsnitt.

Merge sort

Fletting er en viktig teknikk. F.eks. er fletting den essensielle delen i flettesortering (eng: merge sort). Idéen er at en tabell kan sorteres ved at dens to halvdeler sorteres hver for seg og at de så flettes sammen. Dette gjentas rekursivt for hver av de to delene. I flettesortering er det sorterte nabointervaller som flettes sammen. Se på flg. eksempel:

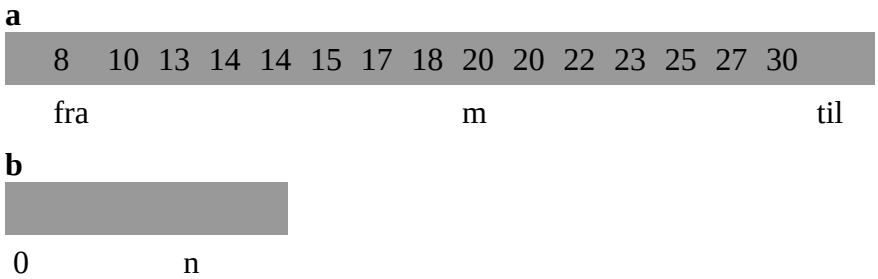


Intervallene a[fra:m> (hvit del med tall) og a[m:til> (den grå delen) skal flettes sammen. Først kopieres a[fra:m> over i b. Den må minst ha plass til n verdier der n = m – fra:



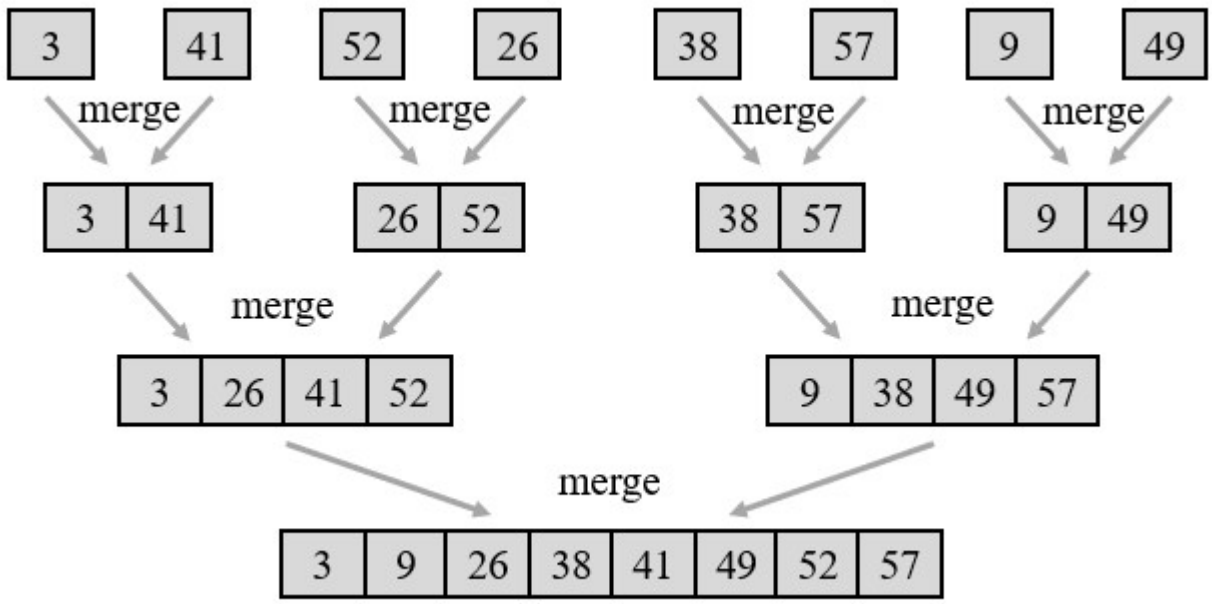
Intervallet a[fra:m> er kopiert over i b[0:n>.

Neste skritt er å flette sammen b[0:n> og a[m:til>. Legg spesielt merke til at hvis vi i flettingen blir ferdig med verdiene i b[0:n> først, trenger vi ikke gjøre noe med det som ligger igjen bakerst i a[m:til>. De ligger der de skal ligge:



Det er ofte aktuelt å kunne flette sammen to sorterte tabellintervaller, f.eks. de to halvåpne intervallene a[0:m> og b[0:n>. Tabellen c må være stor nok, dvs. ha plass til minst m + n verdier.

Merge sort illustrasjon



Merge sort kode

fletter b[0:n> og a[m:til> over i a[fra:til>

Iterativt:

```
private static void flett(int[] a, int[] b, int fra, int m, int til) {
    int n = m - fra;                // antall elementer i a[fra:m>
    System.arraycopy(a,fra,b,0,n);  // kopierer a[fra:m> over i b[0:n>

    int i = 0, j = m, k = fra;      // løkkevariabler og indekser

    while (i < n && j < til)        // fletter b[0:n> og a[m:til> og
    {                               // legger resultatet i a[fra:til>
      a[k++] = b[i] <= a[j] ? b[i++] : a[j++];
    }

    while (i < n) a[k++] = b[i++];  // tar med resten av b[0:n>
  }
```

Rekursivt:

```
private static void flettesortering(int[] a, int[] b, int fra, int til) {
    if (til - fra <= 1) return;   // a[fra:til> har maks ett element
    int m = (fra + til)/2;        // midt mellom fra og til

    flettesortering(a,b,fra,m);   // sorterer a[fra:m>
    flettesortering(a,b,m,til);   // sorterer a[m:til>

    if (a[m-1] > a[m]) flett(a,b,fra,m,til);  // fletter a[fra:m> og a[m:til>
  }
```

I koden over deles a[fra:til> på midten og metoden kalles (rekursjon) først på a[fra:m> og så på a[m:til>. Etterpå vil de være sortert og kan flettes sammen. Legg merke til setningen: if (a[m-1] > a[m]). Intervallet a[fra:til> er allerede sortert hvis den siste i a[fra:m> er mindre enn eller lik den første i a[m:til>. Midtdelingen gjør at dette blir av orden n log2(n) og dermed en effektiv metode.

Hel tabell:

```
 public static void flettesortering(int[] a)
  {
    int[] b = Arrays.copyOf(a, a.length/2);   // en hjelpetabell for flettingen
    flettesortering(a,b,0,a.length);          // kaller metoden over
  }
```

Selection sort (Utvalgssortering)

Det er enkelt å finne den minste (eller den største) verdien i en tabell. Utvalgssortering (eng: selection sort) går ut på å gjøre det gjentatte ganger.

1. Første oppgave er å finne den minste verdien. Den byttes med verdien i posisjon 0 slik at den minste verdien kommer først i tabellen

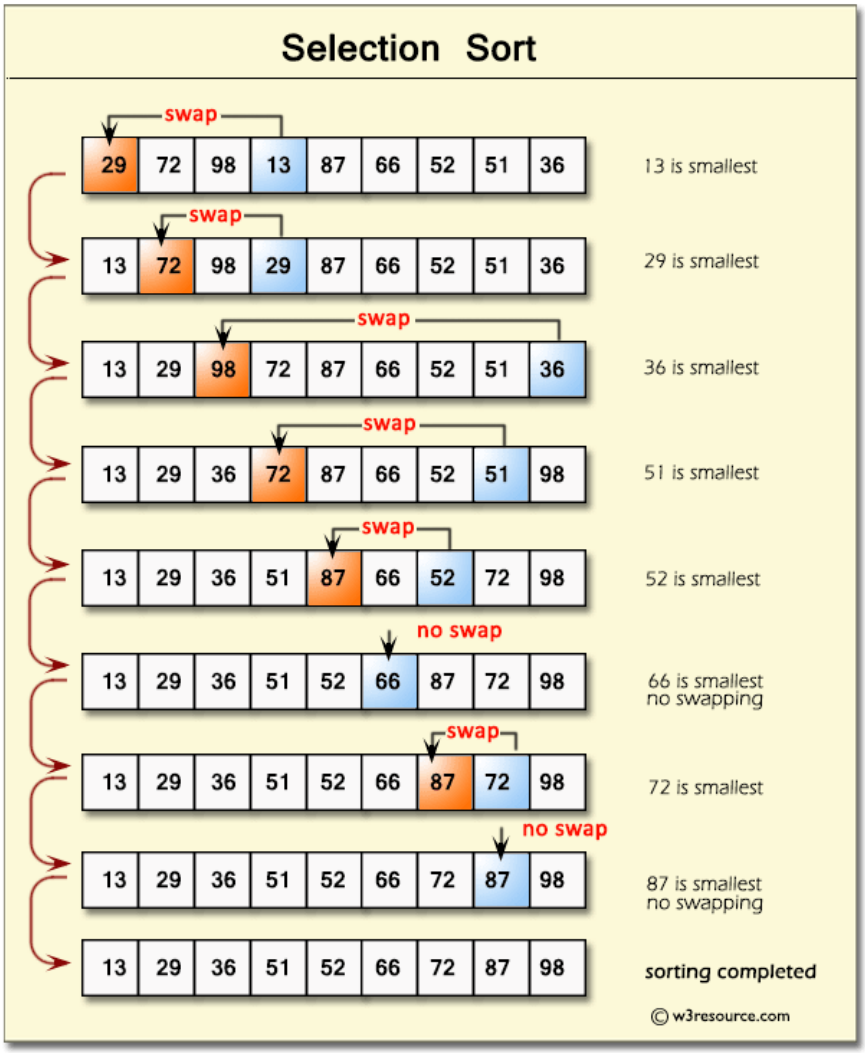
2. Vi gjør dette en gang til. Men nå finner vi den minste i resten av tabellen, dvs. i den delen av tabellen som starter i posisjon 1. Den byttes med verdien i posisjon 1 slik at den nest minste verdien kommer nest først i tabellen

3. Da er det bare å fortsette. Den minste verdien fra og med posisjon 2 og utover.

Kode

```
public static void bytt(int[] a, int i, int j)
  {
    int temp = a[i]; a[i] = a[j]; a[j] = temp;
  }

  public static int min(int[] a, int fra, int til) {
    if (fra < 0 || til > a.length || fra >= til)
      throw new IllegalArgumentException("Illegalt intervall!");

    int m = fra;             // indeks til minste verdi i a[fra:til>
    int minverdi = a[fra];   // minste verdi i a[fra:til>

    for (int i = fra + 1; i < til; i++) if (a[i] < minverdi) {
      m = i;               // indeks til minste verdi oppdateres
      minverdi = a[m];     // minste verdi oppdateres
    }

    return m;  // posisjonen til minste verdi i a[fra:til>
  }


public static void utvalgssortering(int[] a) {
    for (int i = 0; i < a.length - 1; i++)
      bytt(a, i, min(a, i, a.length));  // to hjelpemetoder
  }
```

Selection sort illustrasjon



Effektivitet

Hvor effektiv er utvalgssortering? La n være antall verdier. Det trengs n − 1 sammenligninger for å finne den minste verdien. Deretter leter vi etter den minste blant resten. Til det trengs n − 2 sammenligninger. Osv. Til sammen blir det:

$$n - 1 + n - 2 + \cdot \cdot \cdot \cdot + 3 + 2 + 1 = n(n - 1)/2$$

Resultatet n(n – 1)/2 er summen av en aritmetisk rekke. Hvis f.eks. n = 1000, blir det totalt 499.500 sammenligninger. Antallet kan også skrives som n2/2 – n/2, dvs. et kvadratisk uttrykk i n. Algoritmen er derfor av kvadratisk orden eller av orden n^2. Det betyr at hvis vi dobler antall verdier, vil antall sammenligninger bli fire ganger så stort. Legg merke til at utvalgssortering er av kvadratisk orden i alle tilfeller - også om tabellen allerede er sortert.

Utvalgssortering er ineffektiv – i hvert fall hvis det er mange verdier. En fordel er imidlertid at algoritmen har en klar idé og har enkel kode. Vi forstår umiddelbart at den gir korrekt resultat. Den har også få ombyttinger. Dette er faktisk den sorteringsmetoden som har færrest ombyttinger, dvs. n − 1 stykker i alle tilfeller. Dette er gunstig sammenlignet f.eks. med boblesortering som i gjennomsnitt har n(n − 1)/4 ombyttinger.

Oppsummering - utvalgssortering:

- **Navn:** Algoritmen kalles utvalgssortering fordi det fortløpende «velges ut» en verdi som så blir plassert på rett sortert plass. I den vanlige versjonen av utvalgssortering blir fortløpende den minste blant de usorterte «valgt ut» og så plassert først blant dem.

- **Effektivitet:** Det trengs n − 1 sammenligninger for å finne den minste, så n − 2 stykker for å finne den nest minste (den minste av de øvrige), osv. Til sammen blir det n(n − 1)/2 = n2/2 − n/2 sammenligninger og det uansett fordelingen av verdiene. Det betyr at algoritmen har kvadratisk orden i alle tilfeller (verst, gjennomsnittlig og best).

- **Ombyttinger:** Det gjøres én ombytting i hver iterasjon og dermed n − 1 til sammen. Dette kan reduseres noe.

- **Konklusjon:** Utvalgssortering er som alle sorteringsalgoritmer av kvadratisk orden, ikke egnet til å sortere store tabeller. Den er imidlertid bedre enn boblesortering. Se Oppgave 4. I noen situasjoner kan det være ekstra kostbart å bytte om verdier (gjelder ikke i Java). I slike tilfeller kan denne algoritmen brukes for tabeller av moderat størrelse. Dette er den algoritmen som har færrest ombyttinger.

Insertion sort (Innsettingssortering)

Hvis en tabell skal holdes sortert, kan vi ikke legge inn nye verdier på vilkårlige plasser. De må legges inn på rett sortert plass. Hvis en verdi ikke er der fra før, har den et veldefinert innsettingspunkt. Hvis verdien allerede er der, kan den settes inn foran, bak eller eventuelt mellom dem som er der fra før. Det må også være plass i tabellen til en ny eller nye verdier.

Gitt tabellen:

3	5	6	10	10	11	13	14	16	20	12	4	7	2	15
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

10 verdier sortert (hvit del) - 5 verdier usortert (grå del)

I innsettingssortering er tabellen todelt. Første del inneholder sorterte og andre del (med grå bakgrunn) usorterte verdier. Fortsettelsen går ut på at den første verdien i den usorterte delen settes inn på rett plass i den sorterte delen. Vi legger den (her 12) midlertidig til side i en hjelpevariabel. Dermed får den «hvite» delen en ledig plass (indeks 10):

12	3	5	6	10	10	11	13	14	16	20		4	7	2	15
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Første usorterte verdi (dvs. 12) er flyttet til en hjelpevariabel

Neste skritt er å finne hvor verdien 12 skal inn. Én og én verdi forskyves mot høyre inntil vi finner rett plass. Først flyttes 20 én mot høyre, så 16, osv. inntil indeks 6 der 12 skal inn:

3	5	6	10	10	11	12	13	14	16	20	4	7	2	15
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Tallet 12 er nå på rett sortert plass

Dette fortsetter: Den første av de usorterte (dvs. 4) legges til side og dens plass blir ledig:

4	3	5	6	10	10	11	12	13	14	16	20		7	2	15
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Første usorterte verdi (dvs. 4) er flyttet til en hjelpevariabel

Så forskyves én og én verdi. Vi ser at rett plass er nest først (indeks 1). Dermed:

3	4	5	6	10	10	11	12	13	14	16	20	7	2	15
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Tallet 4 har kommet på rett sortert plass

Vi er ferdige når hele tabellen har blitt «hvit». I utgangspunktet er hele tabellen usortert. Men vi kan se på det første elementet som sortert. Det betyr at når innsettingssorteringen starter, utgjør første element den «hvite» delen og resten den «grå» delen.

Insertion sort kode

```
public static void innsettingssortering(int[] a) {
  for (int i = 1; i < a.length; i++) {  // starter med i = 1
    int verdi = a[i], j = i - 1;      // verdi er et tabellelemnet, j er en indeks
    for (; j >= 0 && verdi < a[j]; j--) a[j+1] = a[j];  // sammenligner og flytter
    a[j + 1] = verdi;                 // j + 1 er rett sortert plass
  }
}
```

Algoritmeanalyse

Vi skal finne ut hvor mange ganger sammenligningen verdi < a[j] utføres i en tabell med n verdier der alle er forskjellige. Anta at de i første verdiene er sortert, dvs. a[0], a[1], · · · a[i - 1]. Tallet verdi skal inn på rett plass blant dem. Det er i + 1 forskjellige muligheter: foran a[0], mellom a[0] og a[1], mellom a[1] og a[2], osv. eller etter a[i - 1]. Alle er like sannsynlige siden verdiene er forskjellige.

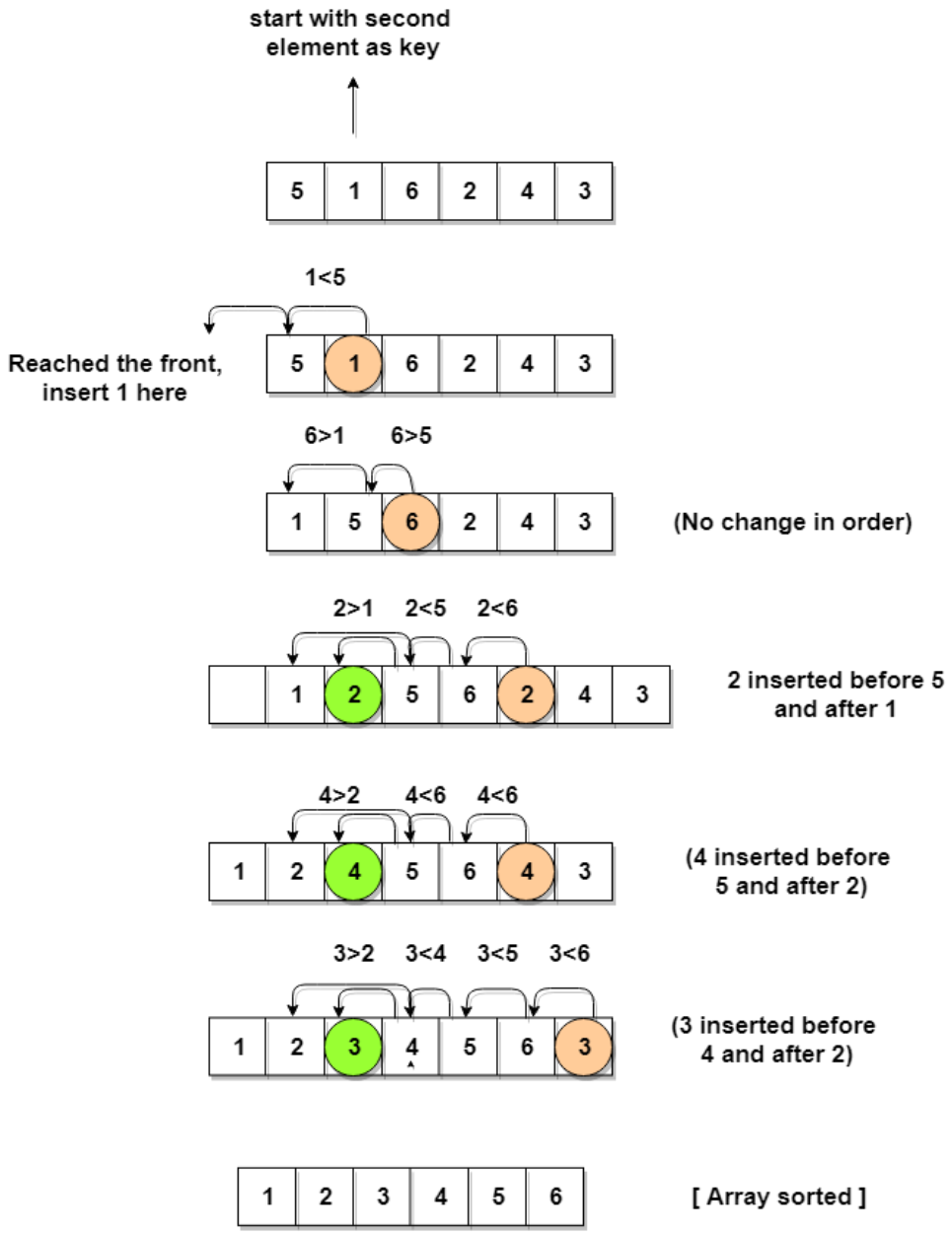
Vi trenger én sammenligning for å avgjøre om verdi skal bak a[i - 1], to for å avgjøre om den skal mellom a[i – 1] og a[i – 2], osv. Til slutt trengs i stykker både for å avgjøre om verdi skal mellom a[0] og a[1] eller foran a[0]. Sum: 1 + 2 + · · · + i + i = i(i+1)/2 + i. Gjennomsnittet får vi ved å dele med i + 1, dvs. lik i/2 + 1 - 1/(i + 1) Vi summerer fra 1 til n - 1 og får det gjennomsnittlige antallet ganger verdi < a[j] utføres:

n(n − 1)/4 + n − Hn = n(n + 3)/4 − Hn

Det verste tilfellet er en sortert avtagende tabell. Da er verdi < a[j] sann for hver j. Dermed i sammenligninger i indre løkke, og totalt 1 + 2 + · · · + i = n (n − 1)/2 sammenligninger. Det beste tilfellet er når tabellen er sortert stigende. Da er verdi < a[j] aldri sann og antallet blir 1 + 1 + · · · + 1 = n − 1.

Gjennomsnittlig antall sammenligninger i innsettingssortering med n forskjellige verdier er: n(n + 3)/4 − Hn. Det verste: n(n − 1)/2 og det beste: n − 1. Dermed av kvadratisk orden både i gjennomsnitt og i det verste tilfellet.

Insertion sort illustrasjon



Oppsummering innsettingssortering

- Navn: Det kalles innsettingssortering fordi tabellen underveis er todelt. Venstre del er sortert. Én og én verdi fra høyre del settes inn på rett sortert plass i venstre del. Ved start består venstre del av det første elementet og høyre del av resten. Når algoritmen slutter, består venstre del av hele tabellen (og høyre del er tom).

- Effektivitet: Det kan vises (se algoritmeanalyse) at det i gjennomsnitt (hvis alle verdiene er forskjellige) utføres $n(n + 3)/4 - H_n$ sammenligninger. Det betyr at den er av kvadratisk orden i gjennomsnitt. Hvis tabellen allerede er sortert, utføres det $n - 1$ sammenligninger. Dermed er den av lineær orden i det beste tilfellet.

Ombyttinger: Det er ingen ombyttinger. Det er isteden forskyvninger (tilordninger) som er litt mindre kostbare, og av dem er det i gjennomsnitt $n(n - 1)/4$ stykker.

- Konklusjon: Av de tre av kvadratisk orden som vi har sett på, er innsettingssortering best. Vi skal se på algoritmer av orden $n \log(n)$ og de er langt bedre for store tabeller. Men innsettingssortering er best hvis tabellen er liten eller stor, men delvis sortert. Det er kvikksortering som er best av de «avanserte». Den arbeider på intervaller og når de har blitt små nok, er det der vanlig å skifte over til innsettingssortering.

Shell sort

Shellsort er en forbedring av innsettingssortering. Idéen er at tabellen fortløpende deles opp i mindre og mindre grupper av verdier og de sorteres hver for seg ved hjelp av innsettingssortering.

Gitt følgende tabell:

8	14	16	19	17	9	3	4	1	15	5	18	13	11	12	2	6	20	7	10
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Vi deler opp i 10 grupper à to verdier. Første gruppe består av a[0] og a[10], andre gruppe av a[1] og a[11], tredje gruppe a[2] og a[12], osv. til a[9] og a[19] som tiende gruppe. Hver gruppe sorteres for seg. Med kun to verdier kan det gjøres ved at de to verdiene bytter plass hvis de er i utakt. Det betyr f.eks. at a[0] = 8 må bytte plass med a[10] = 5. Osv.

5	14	13	11	12	2	3	4	1	10	8	18	16	19	17	9	6	20	7	15
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

I 6 av de 10 parene har verdiene skiftet plass

Nå deler vi opp i 4 grupper à fem verdier. Første gruppe skal bestå av a[0], a[4], a[8], a[12] og a[16], dvs. av verdiene 5, 12, 1, 16 og 6. For lesbarhetens skyld får de grå bakgrunn. Andre gruppe: a[1], a[5], a[9], a[13] og a[17], dvs. verdiene 14, 2, 10, 19 og 20. Osv. Disse 4 gruppene skal sorteres hver for seg ved hjelp av innsettingssortering.

5	14	13	11	12	2	3	4	1	10	8	18	16	19	17	9	6	20	7	15
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

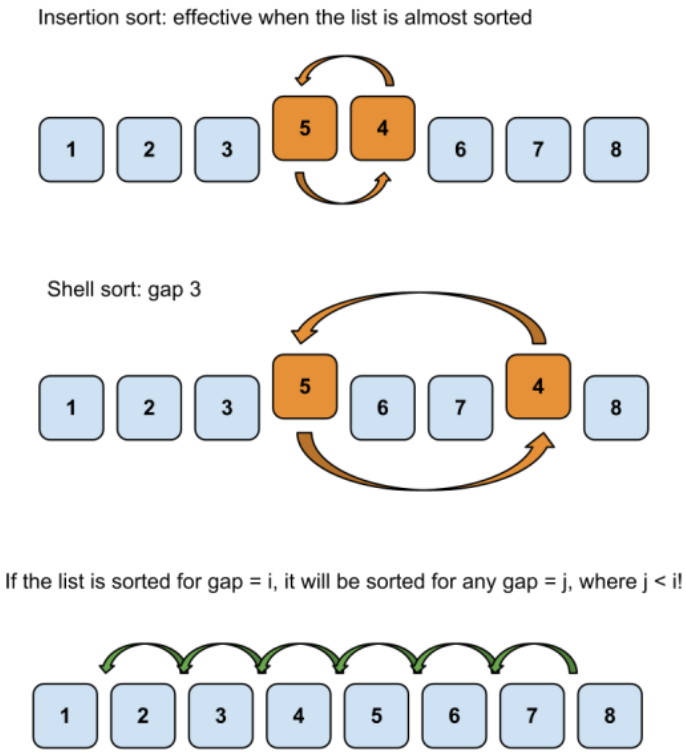
Verdiene i første gruppe har fått grå bakgrunn

Innsettingssortering brukes normalt på en hel tabell eller på et intervall. Men her har vi ingen av delene. Men verdiene har en fast avstand eller gap. Den er forskjellen mellom indeksene for to naboverdier. Mellom a[0] og a[4] er det 4 - 0 = 4, mellom a[4] og a[8] lik 8 - 4 = 4. Osv. Vanligvis økes (eller reduseres) en indeks med 1, men her blir det 4.

Kode

```
public static void shell(int[] a, int k) {
   for (int i = k; i < a.length; i++) {
     int temp = a[i], j = i - k;
     for ( ; j >= 0 && temp < a[j]; j -= k) a[j + k] = a[j];
     a[j + k] = temp;
   }
 }
```

Shell sort illustrasjon



Oppsummering - shellsortering

- Navn: Kalles shellsortering til ære for «oppdageren» Donald Shell. Den ble kjent i 1959 og vakte stor interesse siden man på den tiden ikke kjente til generelle metoder med en (gjennomsnittlig) orden vesentlig bedre enn n^2.

- Effektivitet: Det er kjent at den er av kvadrastisk (n^2) orden i det verste og av orden n log(n) i det beste tilfellet. Men metoden er sterkt avhengig av sekvensen av gap-verdier og det er ikke kjent hvilken orden den har i gjennomsnitt.

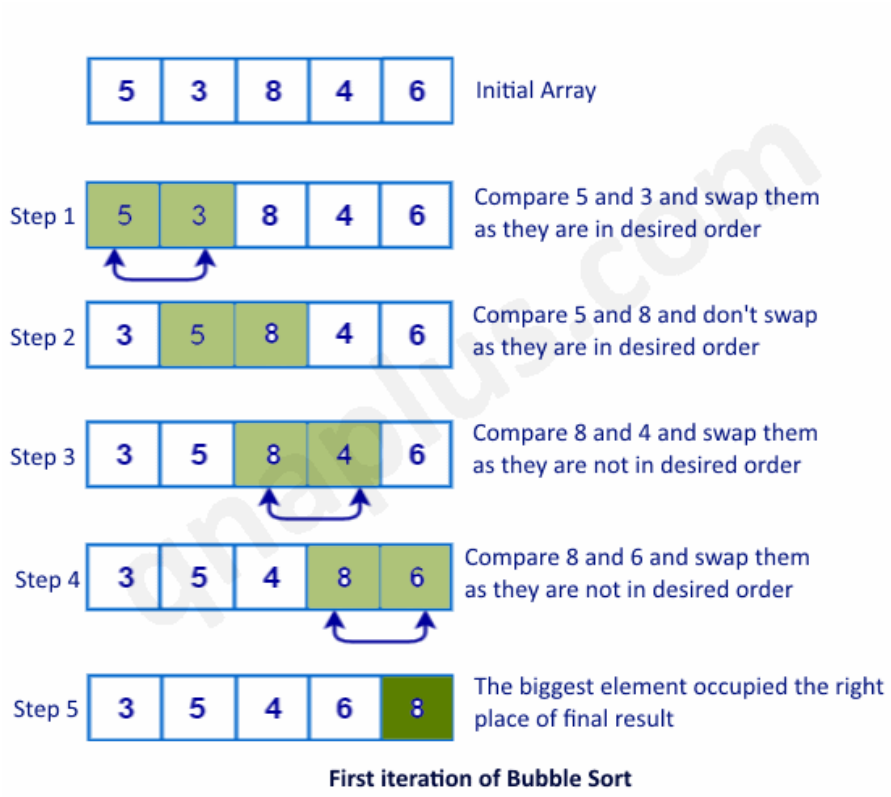
- Konklusjon: Metoden brukes ikke lenger i praksis siden det nå finnes bedre metoder - f.eks. kvikksortering. Men den er interessant siden den på en systematisk måte utnytter de beste sidene ved innsettingssortering.

Bubble sort

Gitt at vi har en tabell a som inneholder en permutasjon av tallene fra 1 til n. Den vil (hvis den ikke er sortert stigende) inneholde et antall inversjoner. I flg. metode sammenlignes to og to naboverdier og de bytter plass hvis de danner en **inversjon** *(Gitt en rekkefølge av heltall. Et par (x , y) av tall fra rekkefølgen der x ligger til venstre for y, kalles en inversjon hvis x og y er i utakt, dvs. hvis tallet x er større enn tallet y.)*. Til slutt returneres antallet ombyttinger. Hvis vi tenker oss at tabellen står vertikalt, vil store verdier «bobles» oppover (bobler som i en vannkjele som koker). Derav navnet.

```
public static void bytt(int[] a, int i, int j)
  {
    int temp = a[i]; a[i] = a[j]; a[j] = temp;
  }

  public static void boblesortering(int[] a) {
    for (int n = a.length; n > 1; n--)           // n reduseres med 1 hver gang
    {
      for (int i = 1; i < n; i++)                // går fra 1 til n
      {
        if (a[i - 1] > a[i]) bytt(a, i - 1, i);  // sammenligner/bytter
      }
    }
  }
```

Bubble sort illustrasjon



First iteration of Bubble Sort

Algoritmeanalyse

La a inneholde en permutasjon av tallene fra 1 til n. Vi må finne ut hvor mange sammenligninger av typen a[i - 1] > a[i] som utføres og så hvor mange kall på bytt() (dvs. ombyttinger) det er. I første gjennomgang blir det $n - 1$ sammenligninger. Så reduseres n. Det betyr at i neste gjennomgang blir det $n - 2$ stykker, osv. til 1 i siste. Summen blir (den aritmetiske rekken $1 + 2 + \ldots + n - 1$) lik $n(n - 1)/2$. Dette er helt uavhengig av hva tabellen inneholder. Med andre er denne versjonen av boblesortering av kvadratisk orden. I algoritmeanalyse skiller vi normalt mellom hvilken orden en metode har i det verste tilfellet og i gjennomsnitt. Noen ganger ser vi også på det beste tilfellet. Men her blir det kvadratisk orden i alle tilfellene siden det alltid blir $n(n - 1)/2$ sammenligninger

Det utføres nøyaktig like mange ombyttinger som det er inversjoner. Det betyr at i det beste tilfellet (sortert stigende) blir det ingen ombyttinger. I det verste tilfellet (sortert avtagende) blir det $n(n - 1)/2$ og i gjennomsnitt $n(n - 1)/4$ stykker.

Oppsummering - boblesortering:

- **Navn:** I en «loddrett» tabell, vil ombyttinger føre til at store verdier flytter seg oppover og den største havner øverst. Navnet boble er hentet fra det som skjer når vannet i en vannkjele koker. Da går det bobler oppover fra bunnen av kjelen.
- **Effektivitet:** Den «forbedrede» versjonen av boblesortering (Programkode 1.3.3 f) er (mhp. sammenligninger) av lineær orden (orden n) i det beste (en sortert tabell) og av kvadratisk orden (orden n^2) i gjennomsnitt og i det verste tilfellet.
- **Ombyttinger:** Antallet vil variere fra 0 (en sortert tabell) til $n(n - 1)/2$ (sortert motsatt vei). I gjennomsnitt blir det $n(n - 1)/4$ ombyttinger.
- **Konklusjon:** Av de mest kjente sorteringsmetodene er boblesortering den som er minst effektiv. Den brukes normalt ikke. I hvert fall ikke på store tabeller.

Generics og java

I programmering brukes begrepet generisk både om metoder og klasser. En generisk metode burde virke eller kunne brukes for alle datatyper av samme «art». F.eks. kunne vi si at alle datatyper som er «sammenlignbare» (dvs. at instanser kan sammenlignes innbyrdes og ordnes i en bestemt rekkefølge) utgjør samme «art».

• **compareTo:** Hvis instanser av en klasse X har en «naturlig» ordning, så er det som nevnt over, vanlig å la X implementere Comparable<X>. Da må det være flg. sammenheng mellom compareTo og ordningen (x og y er to vilkårlige instanser av klassen X):

1. x er «mindre enn» y hvis og bare hvis x.compareTo(y) < 0

2. x er «mindre enn eller lik» y hvis og bare hvis x.compareTo(y) <= 0

3. x er lik y hvis og bare hvis x.compareTo(y) = 0

4. x.compareTo(y) < 0 hvis og bare hvis y.compareTo(x) > 0

• **compare:** Comparator har metoden compare, men når vi lager en komparator ved hjelp av et lambda-uttrykk, ser vi ikke den. Lambda-uttrykket er på formen (x,y) -> f(x,y) der f er en funksjon som returnerer et heltall. Komparatorordningen behøver ikke være en total ordning. Det holder at den er en total preordning. Flg. krav til sammenhengen mellom f og ordningen må være oppfylt:

1. x er «mindre enn» y hvis og bare hvis f(x,y) < 0

2. x er «mindre enn eller lik» y hvis og bare hvis f(x,y) <= 0

3. hvis x er lik y, så er f(x,y) = 0

4. f(x,y) < 0 hvis og bare hvis f(y,x) > 0

Type	Navn i kompendiet	Navn i java.util
Grensesnitt	Komparator	Comparator
Grensesnitt	Funksjon	Function
Abstrakt metode	*compare*	*compare*
Statisk metode	*naturligOrden*	*naturalOrder*
Statisk metode	*omvendtOrden*	*reverseOrder*
Statisk metode	*orden*	*comparing*
Metodeargument	velger	keyExtractor
Default metode	*deretter*	*thenComparing*
Default metode	*omvendt*	*reversed*

De grunnleggende typene kan sammenlignes vha. ulikhetstegnene (<, <=, >, >=), men det går ikke for referansetypene. De kan kun sammenlignes ved hjep av en metode. Derfor er det ikke mulig å lage en felles maks-metode for alle «sammenlignbare» typer. Men vi kan få det til for alle referansetyper T som er «comparable», dvs. er en subtype til grensesnittet Comparable<T>.

T står for type (eller datatype) og vinkelparentesen <T> forteller at T er en typeparameter. Slik som det nå står kan T bety hvilken som helst referansetype (men ikke en grunnleggende type). Men det er kun for «sammenlignbare» referansetyper at en slik maks-metode er aktuell. Derfor må typen T ha en metode som sammenligner. Dette ordnes ved hjelp av et grensesnitt (eng: interface).

Dette kalles et generisk grensesnitt fordi det inneholder en typeparameter T. Vi kan bruke det til å definere hva det skal bety at instanser av en referansetype kan sammenlignes: Instanser av referansetypen T kan sammenlignes hvis T implementerer Comparable<T>. Typen T sies da å være sammenlignbar.

I generiske metoder, klasser og grensesnitt må det inngå én eller flere typeparametere. Da er det ofte gunstig å bruke en stor bokstav som navn. T er et vanlig navn siden det er første bokstav i ordet Type. Hvis det trengs flere, kunne en bruke S eller kanskje U. Men alle bokstaver kan selvfølgelig brukes. Noen eksempler er E for element, V for verdi (eng: value) og K for nøkkelverdi (eng: key). Noen liker også å bruke korte ord med stor forbokstav, f.eks. ordet Type.

Omslagsklasser

Hver grunnleggende datatype kan «gjøres om» til en referansetype ved hjelp av en såkalt omslagsklasse (eng: wrapper class). For eksempel er klassen Integer en omslagsklasse for datatypen int. Klassen Integer har kun et heltall, dvs. en int, som instansvariabel. Navnet omslagsklasse kommer av at klassen fungerer som et omslag rundt datatypen. I biblioteket java.lang starter klassedefinisjonen av Integer slik:

```
public final class Integer extends Number implements Comparable<Integer>
```

Klassen Integer har vært med i Java siden starten, men senere har den fått mer innhold. Legg merke til at Integer implementerer Comparable<Integer>. Det betyr at Integer er sammenlignbar.

Vi har blant annet Integer, Double, Boolean, Character, Byte, Short, Long og Float.

Subtyper

Typebegrensningen <T extends Comparable<? super T>> av typen T (eng: bounded type parameter) vil gjøre at alle typer T og subtyper av slike typer, kan brukes. For eksempel kan vi lage subtype Student til Person. Det gjør at Student kan arve fra Person.

Ternary if

```
variable = Expression1 ? Expression2: Expression3
```

Lambda

Moderne teknikk: Et lamda-uttrykk lager implisitt en instans av en (anonym) klasse som implementerer Komparator. Denne nye Javakonstruksjonen fører til vesentlig kortere kode:

```
class FornavnKomparator implements Komparator<Person> {
      public int compare(Person p1, Person p2) {       // to personer
        return p1.fornavn().compareTo(p2.fornavn());  // sammenligner fornavn
      }
    }

    Komparator<Person> c = new FornavnKomparator();   // en instans av klassen
    Tabell.innsettingssortering(p, c);
```

Blir til:

```
Komparator<Person> c = (p1,p2) -> p1.fornavn().compareTo(p2.fornavn());
```

(parameter1, parameter2) -> expression

Fra-til kontroll

```
 public static void fratilKontroll(int tablengde, int fra, int til) {

        if (fra < 0)                                  // fra er negativ

            throw new ArrayIndexOutOfBoundsException ("fra(" + fra + ") er negativ!");

        if (til > tablengde)                          // til er utenfor tabellen

            throw new ArrayIndexOutOfBoundsException

                    ("til(" + til + ") > tablengde(" + tablengde + ")");

        if (fra > til)                                // fra er større enn til

            throw new IllegalArgumentException

                    ("fra(" + fra + ") > til(" + til + ") - illegalt intervall!");

        if(tablengde == 0)

            throw new NullPointerException("Tabellen er tom!");

        if (fra == til)

            throw new NoSuchElementException

                    ("fra(" + fra + ") = til(" + til + ") - tomt tabellintervall!");

    }
```

Komperator interface kode

```
@FunctionalInterface
public interface Komparator<T> {     // et funksjonsgrensesnitt
  // Den abstrakte metoden:

  int compare(T o1, T o2);

  // Statiske metoder:

  public static <T extends Comparable<? super T>> Komparator<T> naturligOrden() {
    return (x,y) -> x.compareTo(y);
  }

  public static <T extends Comparable<? super T>> Komparator<T> omvendtOrden() {
    return (x, y) -> y.compareTo(x);
  }

  public static <T, R extends Comparable<? super R>>
  Komparator<T> orden(Funksjon<? super T, ? extends R> velger) {
    return (x, y) -> velger.anvend(x).compareTo(velger.anvend(y));
  }

  public static <T> Komparator<T> intorden(IntFunksjon<? super T> velger) {
    return (x, y) -> {
      int a = velger.anvend(x);
      int b = velger.anvend(y);
      if (a < b) return -1;
      else if (a == b) return 0;
      else return 1;
    };
  }

  public static <T, R> Komparator<T> orden
  (Funksjon<? super T, ? extends R> velger, Komparator<? super R> c) {
    return (x, y) -> c.compare(velger.anvend(x), velger.anvend(y));
  }

  public static <T extends Comparable<? super T>> Komparator<T> orden(IntFunksjon<? super T> velger) {
    return (x,y) -> {
      int a = velger.anvend(x);
      int b = velger.anvend(y);
      if (a < b) return -1;
      else if (a == b) return 0;
      else return 1;
    };
  }

  // Default-metoder:

  default Komparator<T> deretter(Komparator<? super T> c) {
    return (x,y) -> {
      int cmp = compare(x,y);
      return cmp != 0 ? cmp : c.compare(x, y);
    };
  }
```

```
default <R extends Comparable<? super R>> Komparator<T>
deretter(Funksjon<? super T,? extends R> velger) {
    return (x,y) -> {
      int cmp = compare(x,y);
      if (cmp != 0) return cmp;
      return velger.anvend(x).compareTo(velger.anvend(y));
    };
  }

  default <R> Komparator<T>
  deretter(Funksjon<? super T, ? extends R> velger, Komparator<? super R> c) {
    return (x, y) ->
    {
      int k = compare(x, y);
      return k != 0 ? k : c.compare(velger.anvend(x), velger.anvend(y));
    };
  }

  default Komparator<T> omvendt() {

    return (x, y) -> compare(y, x);  // bytter x og y
  }

}
```

Rekursjon

En rekursiv metode er en metode som kaller seg selv. Det betyr at metodens kode inneholder et eller flere kall på den samme metoden.

Krav:

1. Kallene må bli enklere hver gang det kalles. I tilegg må kallene være slik at ting ikke gjentas, dvs. at noe som allerede er løst ikke løses på nytt.

2. Metodekallene må nå ett tilfelle som kan behandles uten et nye kall på metoden. Dette kalles et basistilfelle (eller en basissituasjon).

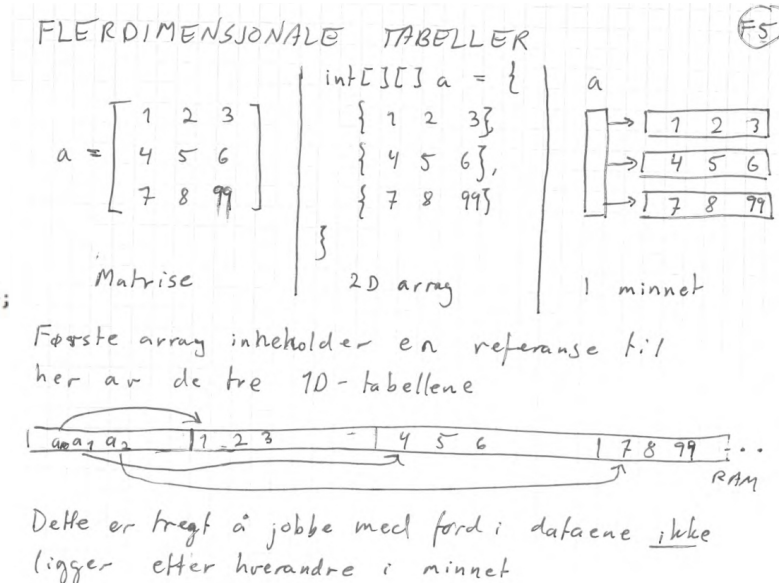
Multidimensjonale tabeller

- To dimensjoner

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15

```
int[][] a = {{1,2,3,4,5}, {6,7,8,9,10}, {11,12,13,14,15}};
```

- a[j][i] gir oss rad j, kolonne i
- Eksempel: a[3][2] = 12



Lineære datastrukturer

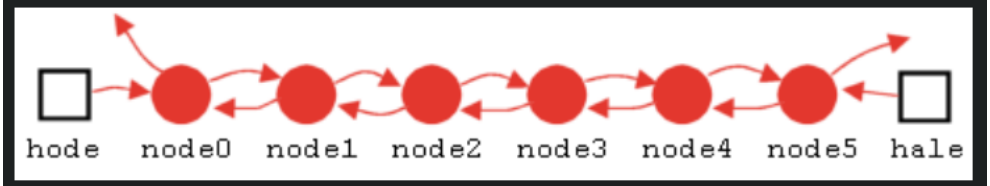
En beholder er noe vi kan legge ting i.

En metode som kan endre tilstanden (dvs. endre på noen av instansvariablene) til en instans av en klasse, kalles en mutator.

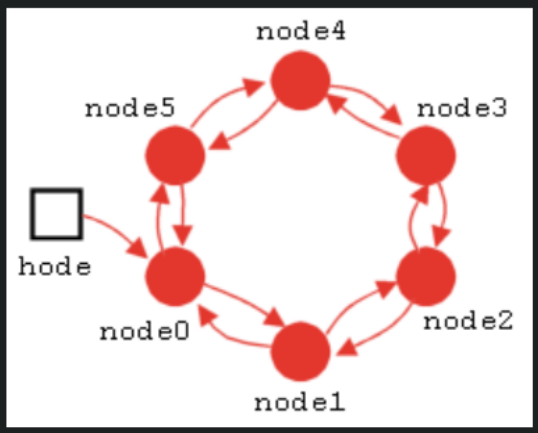
Lenkede lister

En lenket liste (eller referansekjede/pekerkjede som det også kalles) består av en samling noder som er lenket sammen (i en kjede). Vi kan ha en enkeltlenket liste med hode, en dobbeltlenket liste med hode og hale, en sirkulær dobbeltlenket liste og andre varianter.

Figur 3.3.1 a) : En enkeltlenket liste med 6 noder



Figur 3.3.1 b) : En dobbeltlenket liste med 6 noder



Figur 3.3.1 c) : En dobbeltlenket sirkulær liste med 6 noder

I en enkeltlenket liste har vi en referanse (hode) som går til den første (indeks 0) noden i listen. I en dobbeltlenket liste har vi to referanser - en (hode) som går til den første i listen og en (hale) som går til den siste. I en sirkulær dobbeltlenket liste kan vi gå både forover og bakover fra node 0. En referanse kan være null. På figurene er det markert ved piler som ikke går til en node, men går til «ingenting».

Fordelen med en dobbeltlenket liste er at når vi skal lete etter en node med oppgitt indeks, kan vi lete forfra hvis indeks hører hjemme i første halvpart av indekser og starte letingen bakfra hvis indeks hører til andre halvpart. Det vil i gjennomsnitt halvere arbeidet i forhold til en enkeltlenket liste. I en enkeltlenket liste må vi alltid lete fra venstre ende (hodet).

Ulempen med en dobbeltlenket liste er selvfølgelig at vi må reservere plass til to referanser i hver node - en som går til neste node og en som går til forrige. Med andre ord må vi bruke dobbelt så mye minne til referanser som det vi behøver å bruke i en enkeltlenket liste.

Enkeltlenket liste

Kode for enkeltlenket liste:

```
public class EnkeltLenketListe<T> implements Liste<T> {
   private static final class Node<T> {       // en «nøstet» klasse
     private T verdi;
     private Node<T> neste;

     private Node(T verdi, Node<T> neste) {   // konstruktør
       this.verdi = verdi; this.neste = neste;
     }
   }

    private Node<T> hode;         // referanse til første node i listen

    private int antall;           // antall verdier/noder i listen

    public EnkeltLenketListe() {   // standardkonstruktør

    hode = hale = null;                // hode og hale er null
    antall = 0;                // ingen verdier - listen er tom
    }

 }
```

Det skal lite arbeid til å sette inn en ny node helt forrest i en enkeltlenket liste med et hode. Det kan gjøres ved at det 1) opprettes en ny node med gitt verdi, 2) nestereferansen i den nye noden settes til den første i listen og 3) hode settes til den nye noden.

Men generelt gjelder at hvis en ny node skal inn på en bestemt plass i listen, må vi starte ved hodet og lete oss frem til denne plassen. Dermed øker letearbeidet jo lenger ut i listen vi skal. Grensesnittet Liste krever at metoden leggInn(T verdi) skal legge verdi bakerst. Hvis vi lar listen også ha en hale, dvs. en referanse til den siste noden, får vi direkte aksess til den siste noden. Da vil leggInn(T verdi) kunne bli effektiv.

Legg inn bakerst

```
public boolean leggInnBakerst(T verdi) {   // verdi legges bakerst
   Objects.requireNonNull(verdi, "Ikke tillatt med null-verdier!");

   if (antall == 0)  hode = hale = new Node<>(verdi, null);  // tom liste
   else hale = hale.neste = new Node<>(verdi, null);         // legges bakerst

   antall++;                  // en mer i listen
   return true;               // vellykket innlegging
 }
```

Legg inn på indeks

I innleggingsmetoden leggInn(int indeks, T verdi) skal verdi legges slik at posisjonen blir lik indeks. Det betyr at hvis indeks er lik 0, skal verdi legges først. Hvis indeks er lik 1, skal den legges nest først, osv. Hvis indeks er lik antall, skal den legges bakerst. Hvis indeks er negativ eller er større enn antall, er den ulovlig. Hvis indeks er lovlig og ulik både 0 og antall, må vi finne noden rett foran der den nye skal inn. Det gjøres i en løkke der en hjelpevariabel (en nodereferanse) p starter på første node og flyttes indeks - 1 ganger.

```
public void leggInn(int indeks, T verdi) {    // verdi til posisjon indeks
    Objects.requireNonNull(verdi, "Ikke tillatt med null-verdier!");

    indeksKontroll(indeks, true);  //true: indeks = antall er lovlig

    if (indeks == 0) {                      // ny verdi skal ligge først
      hode = new Node<>(verdi, hode);    // legges først
      if (antall == 0) hale = hode;      // hode og hale går til samme node
    }
    else if (indeks == antall) {            // ny verdi skal ligge bakerst
      hale = hale.neste = new Node<>(verdi, null);  // legges bakerst
    }
    else {
      Node<T> p = hode;                  // p flyttes indeks - 1 ganger
      for (int i = 1; i < indeks; i++) p = p.neste;

      p.neste = new Node<>(verdi, p.neste);  // verdi settes inn i listen
    }

    antall++;                            // listen har fått en ny verdi
  }
```

Finn node

```
private Node<T> finnNode(int indeks) {
    Node<T> p = hode;
    for (int i = 0; i < indeks; i++) p = p.neste;
    return p;
  }
```

Hent node

```
public T hent(int indeks) {
    indeksKontroll(indeks, false);  //false: indeks = antall er ulovlig
    return finnNode(indeks).verdi;
  }
```

Oppdater verdi

```
public T oppdater(int indeks, T verdi) {
    Objects.requireNonNull(verdi, "Ikke tillatt med null-verdier!");

    indeksKontroll(indeks, false);  //false: indeks = antall er ulovlig

    Node<T> p = finnNode(indeks);
    T gammelVerdi = p.verdi;

    p.verdi = verdi;
    return gammelVerdi;
```

```
}
```

Fjern node ved indeks

Metoden fjern(int indeks) er litt mer komplisert. Hvis den første noden skal fjernes, dvs. indeks = 0, flytter vi rett og slett hode til den neste noden. Hvis vi skal fjerne en node lenger ut i listen, må vi først finne noden som ligger rett foran den som skal fjernes.

Anta at det er node3 som skal fjernes. Da flyttes p til noden rett foran og en hjelpvariabel q settes til den noden som skal fjernes.

Noden q (node3) forsvinner fra listen når p.neste dirigeres forbi q (se figuren). Det gjøres ved setningen: p.neste = q.neste; Vi må passe på spesialtifellene. Det er når listen kun har én verdi og når den siste skal fjernes. I begge tilfeller må hale oppdateres.

```
public T fjern(int indeks) {
   indeksKontroll(indeks, false);  //false: indeks = antall er ulovlig

   T temp;                              // hjelpevariabel

   if (indeks == 0) {                    // skal første verdi fjernes?
     temp = hode.verdi;                 // tar vare på verdien som skal fjernes
     hode = hode.neste;                 // hode flyttes til neste node
     if (antall == 1) hale = null;      // det var kun en verdi i listen
   }
   else {
     Node<T> p = finnNode(indeks - 1);  // p er noden foran den som skal fjernes
     Node<T> q = p.neste;               // q skal fjernes
     temp = q.verdi;                    // tar vare på verdien som skal fjernes

     if (q == hale) hale = p;           // q er siste node
     p.neste = q.neste;                 // "hopper over" q
   }

   antall--;                            // reduserer antallet
   return temp;                         // returner fjernet verdi
 }
```

Indre iteratorklasse

Iteratorklassen EnkeltLenketListeIterator skal ligge som en privat indre klasse i EnkeltLenketListe. Internt i klassen trenger vi en referansevariabel p som kan flyttes fortløpende fra en node og til dens neste. Videre må vi ha en variabel som avgjør om det er tillatt å kalle remove() eller ikke.

```
private class EnkeltLenketListeIterator implements Iterator<T> {
    private Node<T> p = hode;         // p starter på den første i listen
    private boolean fjernOK = false;  // blir sann når next() kalles

    // Metoden hasNext() skal sjekke om p har gått ut av listen eller ikke.

       public boolean hasNext() {
        return p != null;  // p er ute av listen hvis den har blitt null
       }


    // Metoden next() skal returnere «denne» (eng: current) verdien, dvs. verdien i noden p og samtidig
       flytte p til neste node eller til null hvis p var den siste.

       public T next() {
          if (!hasNext()) throw new NoSuchElementException("Ingen verdier!");

          fjernOK = true;            // nå kan remove() kalles
          T denneVerdi = p.verdi;    // tar vare på verdien i p
          p = p.neste;               // flytter p til den neste noden

        return denneVerdi;         // returnerer verdien
       }


        public void remove() {


          if (!fjernOK) throw new IllegalStateException("Ulovlig tilstand!");


          fjernOK = false;               // remove() kan ikke kalles på nytt
          Node<T> q = hode;              // hjelpevariabel

          if (hode.neste == p) {          // skal den første fjernes?

          hode = hode.neste;           // den første fjernes
          if (p == null) hale = null;  // dette var den eneste noden
           }
          else {
           Node<T> r = hode;            // må finne forgjengeren
                                   // til forgjengeren til p
           while (r.neste.neste != p) {
            r = r.neste;               // flytter r
           }

           q = r.neste;                // det er q som skal fjernes
           r.neste = p;                // "hopper" over q
           if (p == null) hale = r;    // q var den siste
          }

          q.verdi = null;               // nuller verdien i noden
          q.neste = null;               // nuller nestereferansen

          antall--;                     // en node mindre i listen
         }

  }
```

Array vs. enkeltlenket liste

Vi kan gjøre flg. sammenligninger mellom de to listeimplementasjonene, dvs. TabellListe og EnkeltLenketListe:

- TabellListe bruker tilsynelatende kun halvparten av plassen til EnkeltLenketListe. Det kommer av at hver node i EnkeltLenketListe har en nestereferanse i tillegg til en verdi. Men der brukes nøyaktig den plassen som trengs, mens den interne tabellen i TabellListe normalt lages en del større enn det som trengs i øyeblikket. Det betyr at de to likevel ikke adskiller seg så veldig mye med hensyn på plassbehov.

- Søking og oppdatering (metodene hent og oppdater) er av konstant orden i TabellListe og av orden n (der n er antallet verdier i listen) i EnkeltLenketListe.

- Det å legge en verdi bakerst i listen er av konstant orden for både TabellListe og for EnkeltLenketListe.

- Det å legge en verdi forrest i listen er av konstant orden i EnkeltLenketListe, men av orden n i TabellListe. Det kommer av at i TabellListe må alle verdiene flyttes før det kan legges noe forrest i tabellen.

- Det å legge en verdi på et vilkårlig sted i listen er av orden n for både TabellListe og for EnkeltLenketListe. I TabellListe er det å finne plassen av konstant orden, men det å flytte på verdier slik at plassen blir ledig, er av orden n. I EnkeltLenketListe er det å finne plassen av orden n, men det å flytte på verdier slik at plassen blir ledig, er av konstant orden.

- Metodene indeksTil, inneholder og fjern er av orden n både i TabellListe og i EnkeltLenketListe. I TabellListe er det å finne den som skal slettes av konstant orden, men det å «tette» igjen «hullet» i tabellen er av orden n. I EnkeltLenketListe er det å finne verdien av orden n, men det å ta vekk en node er av konstant orden.

Konklusjonen er avhengig av hvilke operasjoner som skal brukes mest. Hvis f.eks. hver verdi som skal legges inn, skal legges foran de som allerede er der, så vil EnkeltLenketListe være best. Hvis derimot hver verdi som skal legges inn, skal legges bak de som allerede er der, så vil begge være like gode. Hvis det ofte er behov for å hente eller å oppdatere verdier på bestemte plasser, så er TabellListe best. Dette betyr at normalt vil en TabellListe (eller en ArrayList fra java.util) være det beste valget.

Dobbeltlenket liste

En dobbeltlenket liste er en lenket liste som har en peker til både neste og forrige node.

Kode

```
public class DobbeltLenketListe<T> implements Liste<T> {
	// instansvariabler
	private Node<T> hode;		// peker til den første i listen
	private Node<T> hale;		// peker til den siste i listen
	private int antall;		// antall noder i listen
	private int endringer;		// antall endringer i listen

	public DobbeltLenketListe() {
		hode = null;
		hale = null;
		antall = 0;
		endringer = 0;
	}
	public DobbeltLenketListe(T[] a) {
		if (a == null) {
			throw new NullPointerException("Tabellen a er null!");
		}
		//Iterer gjennom listen a
		for (T t : a) {
			//Hvis elementet er null skriver vi ikke til det
			if (t != null) {
				//Lag ny node
				Node<T> nynode = new Node<>(t);
				if (tom()) {	//Hvis vi er på starten av listen sett første node
					hode = nynode;
				} else {	//Ellers må vi peke på hale sin neste
					hale.neste = nynode;
					nynode.forrige = hale;
				}
				hale = nynode;
				antall++;
				endringer++;
			}
		}
```

```
}
```

Sjekke om liste er tom

```
public boolean tom() {//Returnerer at hale og hode er null (altså tom tabell)
        //Hvis både halen og hodet er null så har vi en tom liste
        return this.hale == null && this.hode == null;
}
```

Legg inn node

```
public boolean leggInn(T verdi) {
        //Hvis verdien er null, kast en melding
        Objects.requireNonNull(verdi, "Verdien er null!");

        //Deklarer ny node med verdien som gis
        Node<T> nynode = new Node<>(verdi);
        if (tom()) {    //Hvis vi er på starten av listen sett første node
                hode = nynode;
        } else {    //Ellers må vi peke på hale sin neste
                hale.neste = nynode;
                nynode.forrige = hale;
        }
        //Noden blir satt som hale
        hale = nynode;
        //Oppdaterer antall og endringer etter å ha kjørt
        antall++;
        endringer++;
        return true;
}
```

Legge inn node med indeks

```
public void leggInn(int indeks, T verdi) {
        if (indeks < 0 || indeks > antall) {        //Sjekker om indeksen er gyldig
                throw new IndexOutOfBoundsException("Indeksen kan ikke være negativ eller større
                                                     enn antall noder");
        }
        //Hvis verdien er null, skal vi ikke skrive denne
        if (verdi == null) {
                throw new NullPointerException("Verdien er tom");
        }
        if (tom() || indeks == antall) { //Hvis vi har en tom liste eller skal legge elementet
                                         sist brukes metoden leggInn()
                leggInn(verdi);
                return;
        }
        //Deklarerer den nye noden som skal inn med verdien som gis
        Node<T> nyNode = new Node<>(verdi);

        if (indeks == 0) {    //Hvis indeks er 0 så setter den ny verdi til hode
                nyNode.neste = hode;
                hode.forrige = nyNode;
                hode = nyNode;
        } else {    //Verdien skal legges mellom to verdier

                //Deklarerer nodene p, q og r, hvor q er noden som skal settes mellom p og r
                Node<T> p = finnNode(indeks - 1);    //Node p skal være før q (nyNode) og får
                                                       derfor indeks-1
                Node<T> r = p.neste;  //node r starter som noden etter p

                //Setter q mellom node p og r
                p.neste = nyNode;
                r.forrige = nyNode;
                nyNode.forrige = p;
                nyNode.neste = r;
        }
        //Oppdaterer antall og endringer etter å ha kjørt
        antall++;
        endringer++;
}
```

Finn indeks til node

```
public int indeksTil(T verdi) {      //Sjekker nodene for å finne indeksen til en gitt verdi
        if (verdi == null) {  //Returnerer -1 hvis det ikke gis en verdi
                return -1;
        }
        //Vi starter på starten av listen
        Node<T> n = hode;

        //Vi starter på starten av listen og itererer til vi finner verdien
        for (int i = 0; i < antall; i++) {
                if (n.verdi.equals(verdi)) {
                        return i;
                }
                n = n.neste;
        }
        return -1;     //Returnerer -1 hvis verdien ikke er i lista
}
```

Oppdater verdi basert på indeks

```
public T oppdater(int indeks, T nyverdi) {
        //Vi sjekker om verdien vi har fått er null
        Objects.requireNonNull(nyverdi, "Verdien er null!");

        //Sjekk om indeksen er gyldig
        indeksKontroll(indeks, false);

        //Finn noden, og lagre verdien midlertidig og øk endringer
        Node<T> nynode = finnNode(indeks);
        T tidligverdi = nynode.verdi;

        nynode.verdi = nyverdi;

        endringer++;
        return tidligverdi;
}
```

Fjerne node basert på verdi

```
public boolean fjern(T verdi) {
        if (verdi == null) {
                return false;
        }

        Node <T> n = hode;
        //Går gjennom lenken for å se om verdien finnes
        while (n != null) {
                if (n.verdi.equals(verdi)) {
                        break;
                }
                n = n.neste;
        }
        if (n == null) {              //Tilfelle der verdi ikke ligger i liste
                //Oppgaven ber om at vi da returnerer false
                return false;
        }
        if (antall == 1) {            //Tilfelle der vi kun har én node, nuller både hode og hale
                hode = hale = null;
        }
        else if (n == hode) {         //Der hvor verdi ligger på indeks 0
                hode = hode.neste;    //Setter nytt hode til neste verdi
                hode.forrige = null;  //Nuller forrige til det nye hode
        }
        else if (n == hale) {         //Der hvor verdi ligger på siste indeks
                hale = hale.forrige;  //Setter ny hale til forrige verdi
                hale.neste = null;    //Nuller neste til den nye halen
        }
        else {                        //HVor verdi ligger mellom indeks 0 og siste indeks
                n.forrige.neste = n.neste;
                n.neste.forrige = n.forrige;
        }
        antall--;
        endringer++;
        //Fjerning har skjedd og vi returnerer true
        return true;
}
```

Fjern node basert på indeks

```
public T fjern(int indeks) {
        if (indeks < 0) {     //Feilhåndteringer
                throw new IndexOutOfBoundsException("Indeks mindre enn null finnes ikke");
        }
        if (antall == 0) {
                throw new IndexOutOfBoundsException("Tabellen er tom");
        }
        if (indeks >= antall) {
                throw new IndexOutOfBoundsException("Indeks høyere enn antall");
        }

        Node<T> n = hode;
        if (antall == 1) {            //Tilfelle der vi kun har én node
                hode = hale = null;
        } else if (indeks == 0) {     //Tilfelle der vi skal fjerne første element
                hode = hode.neste;    //Setter nytt hode til neste verdi
                hode.forrige = null;  //Nuller forrige til det nye hode
        } else if (indeks == antall - 1) {   //Tilfelle hvor vi skal fjerne siste element
                hale = hale.forrige;  //Setter ny hale til forrige verdi
                hale.neste = null;    //Nuller neste til den nye halen
        } else {              //Finner verdi for node og fjerner samme node
                n = finnNode(indeks); //Bruker metoden til å finne noden med gitt indeks
                n.forrige.neste = n.neste;
                n.neste.forrige = n.forrige;
        }
        antall--;
        endringer++;
        return n.verdi;
}
```

Nullstill liste

```
public void nullstill() {
        hode = hale = null;
        setAntall(0);
        endringer = 0;
}
```

Finn node basert på indeks

```
private Node<T> finnNode(int indeks) {
        //Hvis indeksen ligger før midten så begynner vi fra hode og itererer oppover
        if (indeks < antall / 2) {
                Node<T> gjeldene = hode;
                for (int i = 0; i < indeks; i++) {
                        //Bytt til neste node
                        gjeldene = gjeldene.neste;
                }
                return gjeldene;
        }
        //Hvis indeksen ligger etter midten så begynner vi fra hale og itererer nedover
        Node<T> gjeldene = hale;
        for (int i = antall - 1; i > indeks; i--) {
                //Bytt til forrige node
                gjeldene = gjeldene.forrige;
        }
        return gjeldene;
}
```

Antall noder av gitt verdi

```
public int antall(T verdi) {
    //Gir feilmelding om det kommer inn en nullverdi
    Objects.requireNonNull(verdi, "Ulovlig med nullverdier!");

    Node<T> p = rot;               // p starter i roten
    int cmp;                            // hjelpevariabel
    int antall = 0; //Variabel for å telle antall
    while (p != null) { // fortsetter til p er ute av treet
        cmp = comp.compare(verdi,p.verdi);     // bruker komparatoren
        if (cmp == 0) { //Hvis verdien er lik p sin verdi (compare gir da 0)
            antall++;   //Øker antall
        }
        p = cmp < 0 ? p.venstre : p.høyre;     // flytter p
    }
    return antall;  //Returnerer antall
}
```

Effektivitet av lenkede lister

- `TabellListe` bruker tilsynelatende kun halvparten av plassen til `EnkeltLenketListe`. Det kommer av at hver node i `EnkeltLenketListe` har en nestereferanse i tillegg til en verdi. Men der brukes nøyaktig den plassen som trengs, mens den interne tabellen i `TabellListe` normalt lages en del større enn det som trengs i øyeblikket. Det betyr at de to likevel ikke adskiller seg så veldig mye med hensyn på plassbehov.
- Søking og oppdatering (metodene *hent* og *oppdater*) er av konstant orden i `TabellListe` og av orden *n* (der *n* er antallet verdier i listen) i `EnkeltLenketListe`.
- Det å legge en verdi bakerst i listen er av konstant orden for både `TabellListe` og for `EnkeltLenketListe`.
- Det å legge en verdi forrest i listen er av konstant orden i `EnkeltLenketListe`, men av orden *n* i `TabellListe`. Det kommer av at i `TabellListe` må alle verdiene flyttes før det kan legges noe forrest i tabellen.
- Det å legge en verdi på et vilkårlig sted i listen er av orden *n* for både `TabellListe` og for `EnkeltLenketListe`. I `TabellListe` er det å finne plassen av konstant orden, men det å flytte på verdier slik at plassen blir ledig, er av orden *n*. I `EnkeltLenketListe` er det å finne plassen av orden *n*, men det å flytte på verdier slik at plassen blir ledig, er av konstant orden.
- Metodene *indeksTil*, *inneholder* og *fjern* er av orden *n* både i `TabellListe` og i `EnkeltLenketListe`. I `TabellListe` er det å finne den som skal slettes av konstant orden, men det å «tette» igjen «hullet» i tabellen er av orden *n*. I `EnkeltLenketListe` er det å finne verdien av orden *n*, men det å ta vekk en node er av konstant orden.

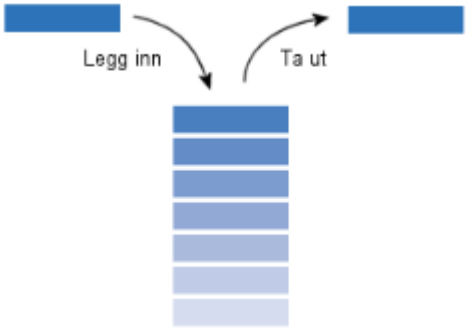
Konklusjonen er avhengig av hvilke operasjoner som skal brukes mest. Hvis f.eks. hver verdi som skal legges inn, skal legges foran de som allerede er der, så vil *EnkeltLenketListe* være best. Hvis derimot hver verdi som skal legges inn, skal legges bak de som allerede er der, så vil begge være like gode. Hvis det ofte er behov for å hente eller å oppdatere verdier på bestemte plasser, så er `TabellListe` best. Dette betyr at normalt vil en `TabellListe` (eller en `ArrayList` fra `java.util`) være det beste valget.

Køer

Køer er det samme konseptet som køer i virkeligheten. Som en butikkkø. Vi har flere ulike typer køer.

OBS: Når man snakker om forrerst eller bakerst er det opp til oss å definere hva det betyr. Forran kan godt være siste element i en liste, hvis man velger å definere det som først.

Stakker (stack)



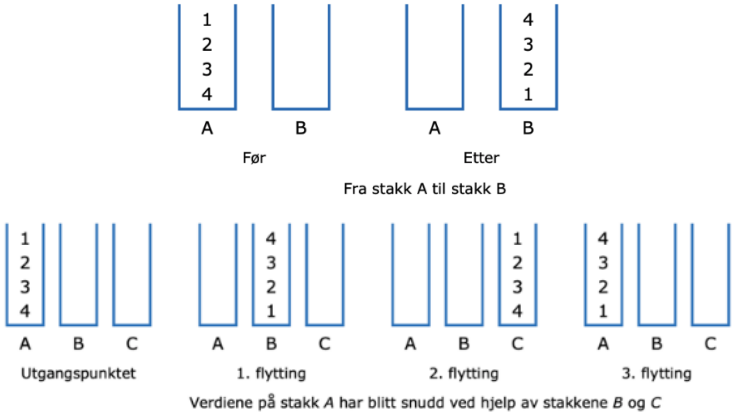
Tenk på en stabel med pannekaker. Det er forklarende for konseptet stakk.

Elementer (pannekaker) legges oppå hverandre og vi tar ut fra toppen. Stakk er en Last In, First Out (LIFO) kø.

En stakk er en datastruktur der det alltid er den verdien som ble lagt på sist som står for tur til å bli tatt ut. Vi kan tenke på en stakk som en stabel av verdier der en ny verdi alltid legges øverst (på toppen) og der vi alltid (så sant stakken ikke er tom) tar den øverste hver gang en verdi skal tas ut.

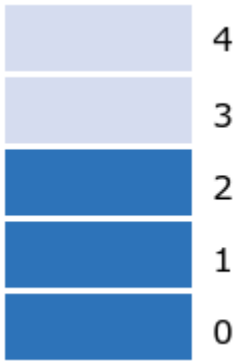
First in - first out (FIFO)

Som en "vanlig kø". Den som stilte seg først i kø er den som får hjelp først. Det er også det samme som en stakk.



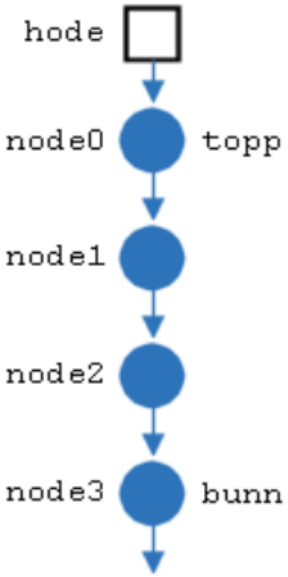
Det første elementet som ble lagt inn i lista er den første som går ut av lista.

Det er ganske enkelt å implementere en stakk ved hjelp av en tabell. Da tenker vi oss at tabellen står på høykant - med indeks 0 nederst og med indeksene 1, 2, 3 osv. oppover. I figuren til venstre er det en tabell med plass til fem verdier. De tre nederste radene har mørk farge. Det skal indikere at der ligger det verdier. Med andre ord er antall verdier på «stakken» lik 3. De to øverste radene har lys farge. Det indikerer ledige plasser. Øverst på stakken er dermed den øverste av de «mørke».



FIFO som enkeltlenket liste

En stakk kan på en enkel måte implementeres ved hjelp av en enkeltlenket liste . Hvis listen tegnes vertikalt, vil den ligne på en stakk. Den første noden i listen blir da øverst av nodene (toppen av stakken) og den siste noden blir nederst (bunnen av stakken).
I figuren til høyre er en liste med fire noder tegnet vertikalt. Når et nytt objekt skal legges på toppen av stakken, representert ved hjelp av denne listen, må den tilhørende noden legges inn først, dvs. som en ny *node* 0. Å legge inn vil få konstant orden siden arbeidet med å legge inn noe først i en liste er uavhengig av antallet i listen. I en enkeltlenket liste er det vanlig å ha en halepeker (peker til den siste noden). Da vil det å legge inn et objekt bakerst i listen kunne gjøres effektivt.



Last in - first out (LIFO)

"Omvendt kø". Den som stilte seg sist i køen får først hjelp.

Dvs. det siste elementet som ble lagt inn i lista er den første som går ut av lista.

	FIFO	**LIFO**
Legge til element	Bakerst i lista addLast() / add()	Foran i lista addFirst() / push()
Fjerne element	Foran i lista removeFirst() / pop() / remove()	

Sirkulær kø

La f.eks. A, B, C, D, E, F og G fortløpende stille seg opp i køen. Det gir flg. tabell:

A	B	C	D	E	F	G									
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Når den første i køen betjenes (og dermed forlater køen) er det vanlig «køkultur» at de andre beveger seg et skritt eller så fremover. I tabellen betyr det at når den første (her er det A) tas ut av køen, blir posisjon 0 ledig. Dermed må de andre fortløpende flytte seg én posisjon mot venstre. Det vil gi følgende resultat:

B	C	D	E	F	G										
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Hvis vi kjenner posisjonen til første ledige plass, kan innleggingen skje direkte - dvs. konstant orden. Det å «kikke» på den første innebærer å hente den som ligger i posisjon 0 - også av konstant orden. Men det å ta ut fra køen er «problematisk». Hvis det er n stykker i køen, krever et uttak at de n − 1 andre må flyttes én og én mot venstre for å «tette igjen» hullet som oppstod. Det betyr at uttaksoperasjonen blir av lineær orden eller av orden n.

Innlegging skal være som før, men etter et uttak skal køen forbli i ro. Altså ingen forflytning mot venstre. Hvis vi på nytt tar utgangspunkt i køen i første tabell, får vi dette resultatet når den første tas ut av køen:

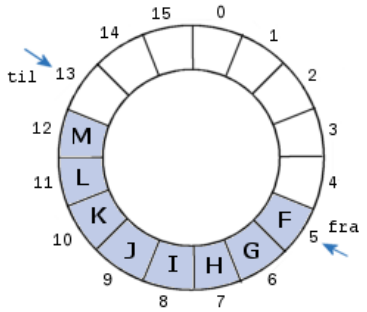
	fra ↓						til ↓								
	B	C	D	E	F	G									
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Hvis tabellen heter a, utgjør køen nå det halvåpne intervallet a [fra : til >. Dette betyr at fra er posisjonen til den første i køen og at til er posisjonen til den første ledige plassen i tabellen, dvs. én posisjon forbi den siste i køen. Videre ser vi at antallet i køen er lik differensen mellom til og fra, dvs. til − fra. I tabellen over blir det 7 − 1 = 6.

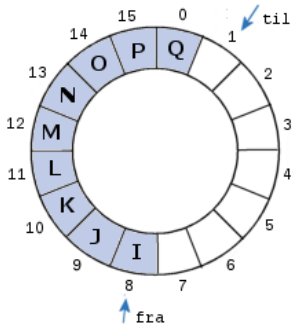
Legg først fortløpende inn H, I, J, K, L og M, og ta deretter ut fortløpende B, C, D og E. Det vil gi flg. resultat:

					fra ↓								til ↓		
					F	G	H	I	J	K	L	M			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Det er nå tre ledige plasser lengst til høyre i tabellen over. Det betyr at vi kan legge inn f.eks. N, O og P uten problemer. Vi kan «utvide» tabellen slik vi har gjort før når det ikke er plass i en tabell. Men det er dumt når det er ledige plasser lengst til venstre. Vi kan isteden tenke oss at tabellen er «sirkelformet». Hold fast i venstre ende (posisjon 0) og bøy høyre ende nedover på en slik måte at de to endene møtes:



Når tabellen er «sirkelformet», får vi et område på 8 fortløpende ledige plasser. Vi legger inn som før i posisjon til og til økes så med 1. Men hvis den blir 16 (lik a.length), må den isteden settes til 0. Vi kan nå legge inn N, O, P og Q fortløpende i den «sirkulære» tabellen i tabellen over. Deretter tas F, G og H fortløpende ut. Det gir tabellen:

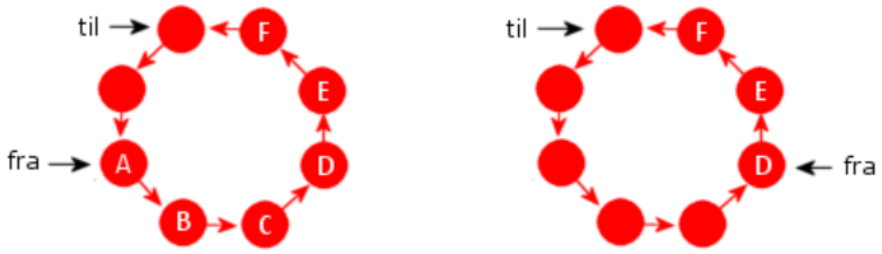


Legg merke til at vi tar ut fra posisjon fra og så økes den med 1. Også her må vi passe på at hvis fra blir 16 (eller a.length), må den isteden settes til 0. Normalt har vi en antall-variabel med verdi til enhver tid er lik antallet i køen. Det trengs egentlig ikke her. Antallet kan regnes ut ved hjelp av fra og til. Hvis fra < til (slik som i første sirkeltabell), blir antall lik til − fra (= 13 − 5 = 8). Men dette vil ikke stemme hvis fra > til slik som i tabellen over). Da blir til − fra negativ. Men da blir istedet antall lik a.length + til − fra (= 16 + 1 − 8 = 9). Vi får et spesialtilfelle hvis fra og til er like. Køen ligger i a [fra : til > og det er tomt hvis fra er lik til. Men det blir annerledes i vår «sirkulære» tabell. Der kan a [fra : til > utgjøre hele tabellen. Se hva som skjer hvis R, S, T, U, V, W og X (7 stykker) legges inn i tabellen over). Da ender til opp med å bli lik 8. Dette problemet kan vi løse ved å sørge for at tabellen aldri er full. Dvs. hvis en innlegging fører til at den blir full, «utvider» vi den med en gang. Dermed vil fra og til være like kun når køen er tom. Antallet kan da også alltid regnes ut.

- Køen skal ligge i en «sirkulær» tabell a .
- En variabel fra skal markere den første i køen (starten på køen). Den settes til 0 hvis en økning har gjort at den har blitt lik a.length.
- En variabel til skal markere første ledige, dvs. én posisjon forbi den siste i køen. Den settes til 0 hvis en økning har gjort at den har blitt lik a.length.
- Tabellen må utvides med en gang hvis den har blitt full etter en innlegging.
- Hvis punktene foran er oppfylt, vil køen være tom hvis og bare hvis fra og til er like.

Lenket kø

Det er mulig å implementere en kø ved hjelp av en enkeltlenket liste med hode og hale. Køen starter ved hodet og slutter ved halen. Det å legge inn en verdi på slutten vil være av konstant orden. Det samme for det å ta ut (eller kikke på) den første i køen. Med andre ord vil både leggInn(), kikk() og taUt() da bli effektive operasjoner.



1) Seks kall på leggInn(). 2) Tre kall på taUt().

Ideene over kan oppsummeres slik:

• Køen skal organiseres som en sirkelformet pekerkjede med en fast startstørrelse.

• En peker fra skal gå til den første i køen.

• En peker til skal gå til den første ledige noden, dvs. én forbi den siste i køen.

• Hvis til == fra, er køen tom.

• Køen skal aldri være full. Ved en innlegging skal det alltid være en ledig node. Hvis det er kun én, utvides pekerekjeden med en ekstra node mellom til og fra.

• En antall-variabel holder orden på antallet verdier i køen.

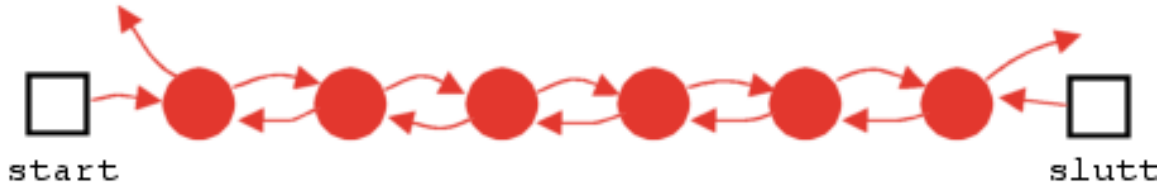
Lenket kø kode

```
public class LenketKø<T> implements Kø<T> {
   private static final class Node<T> {  // en indre nodeklasse
     private T verdi;        // nodens verdi
     private Node<T> neste;  // peker til neste node

     Node(Node<T> neste) {     // nodekonstruktør
       verdi = null; this.neste = neste;
     }
   }

   private Node<T> fra, til;  // fra: først i køen, til: etter den siste
   private int antall;        // antall i køen

   private static final int START_STØRRELSE = 8;

   public LenketKø(int størrelse) {  // konstruktør

  public LenketKø(int størrelse) {   // konstruktør
   // Hvis størrelse <= 1, blir det en node.
   // Ellers blir antall noder lik størrelse.

   til = fra = new Node<>(null);  // lager den første noden

   Node<T> p = fra;               // en hjelpevariabel
   for (int i = 1; i < størrelse; i++) {
     p = new Node<>(p);           // lager resten av nodene
   }
   fra.neste = p;                 // for å få en sirkel

   antall = 0;                    // ingen verdier foreløpig
 }

}
```

Toveiskø (deque)

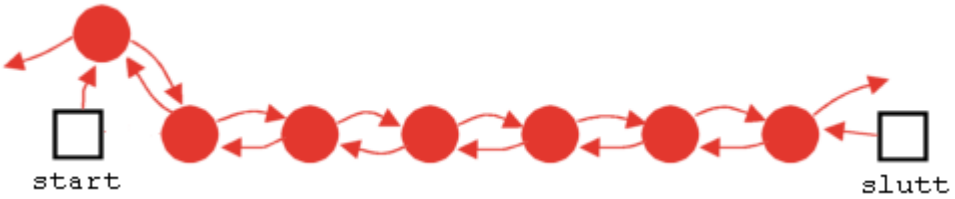
Vi kan slå sammen konseptene FIFO og LIFO. En kø der vi både kan legge inn i begge ender og ta ut fra begge ender. En slik kø skal vi her kalle en toveiskø. Det engelsk navnet for dette er deque og ordet er en forkortelse for double ended queue. En toveiskø må ha metoder med navn som klart indikerer i hvilken ende av køen vi opererer. Vi skal her bruke ordene først og sist til dette formålet.

Lenket toveiskø



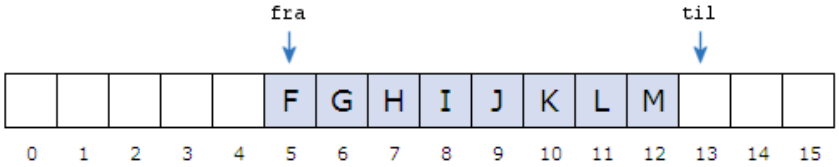
Innlegging i en tom kø gir en kø med kun én node og både start og slutt må dermed peke på denne noden. Nodens forrige- og nestepeker settes til null:

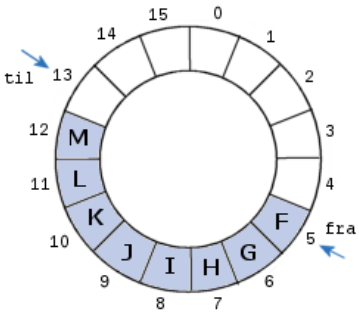
En ny node først i en ikke-tom kø får vi til ved å la forrigepeker i den første noden peke til den nye. Nestepeker i den nye må så peke til det som var den første og forrigepeker i den nye må peke til null. Det hele blir korrekt når pekeren start settes til å peke på den nye:



Sirkulær toveiskø

En sirkulær toveiskø bruker samme idé som i den sirkulære køen.





Det å legge inn en ny verdi først betyr at den skal legges på plassen til venstre for fra eller hvis vi ser på sirkelen, på plassen vi får ved å gå én enhet mot klokken. På begge figurene blir det posisjon 4.

Her må vi imidlertid passe på tilfellet at fra er 0. Én enhet mot klokken blir da posisjon 15, eller generelt lik a.length - 1 hvis tabellen heter a.

Vi må tenke på samme måte når vi skal se på eller ta ut den siste verdien. Den siste verdien ligger i posisjon til - 1 eller posisjon a.length - 1 hvis til er 0. Det er tomt hvis fra og til er like.

Prioriteringskø

En prioritetskø er en kø der de som «står» i køen har en prioritet. Da er regelen at den som har «best» eller «høyest» prioritet står først for tur til å bli tatt ut. En konsekvens er at den som har «lav» prioritet kan oppleve å måtte vente lenge. Tenk legevakta, hvor barn og eldre blir prioritert, mens brudd blant unge voksne som blir nedprioritert. Hva som er prioritert er opp til oss å definere.

Det kan ofte hende at to eller flere elementer har samme prioritet. I mange anvendelser spiller dette ingen rolle og dermed kan man lage mer effektive prioritetskøer. I noen tilfeller er det viktig å kunne ha en bestemt rekkefølge på de som har samme prioritet. Et krav kunne f.eks. være at de som har samme prioritet ordnes som i en vanlig kø, dvs. den som har ventet lengst står først for tur til å bli tatt ut. Det kan man få til f.eks. ved å innføre et «kønummer» i tillegg til en prioritet. Den med lavest «kønummer» blant dem med samme prioritet skal da tas ut først.

Prioriteringskø med usortert tabell

Prioritetskøen kan ha en usortert dynamisk tabell som intern datastruktur. Vi bestemmer at det objektet som en komparator sier er minst, er det som har best/høyest prioritet. I flg. eksempel inneholder køen heltall og det minste heltallet har dermed høyest prioritet:

7	12	3	8	11					
0	1	2	3	4	5	6	7	8	9

I tabellen over består selve prioritetskøen av den grå delen av tabellen. Som vi ser er den usortert. Når tabellen er usortert kan nye verdier legges bakerst. Innlegging får derfor konstant orden. Hvis f.eks. tallet 9 legges inn, blir dette resultatet:

7	12	3	8	11	9				
0	1	2	3	4	5	6	7	8	9

Men det å finne den som har best/høyest prioritet, dvs. det minste tallet, krever at vi må lete gjennom hele tabellen. Til det kan vi f.eks. bruke en metode som finner posisjonen til den minste verdien i et tabellintervall. Det betyr at det å ta ut en verdi får lineær orden. Hvis den verdien som her er tallet 3, tas ut av køen, tetter vi igjen «hullet» ved at det bakerste tallet (tallet 9) flyttes dit. Da får vi dette resultatet:

7	12	9	8	11					
0	1	2	3	4	5	6	7	8	9

UsortertTabellPrioritetsKø

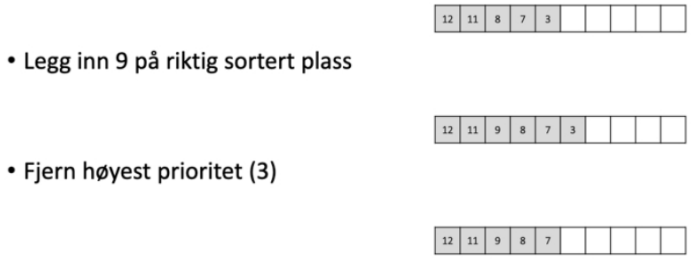
Metode	**Operasjoner**	**Orden**
leggInn	Et nytt element legges alltid bakerst.	*konstant*
taUt	Hvis det er *n* elementer, trengs *n* - 1 sammenligninger for å finne den minste.	*lineær*

Prioriteringskø med sortert tabell

I en prioritetskø tas verdiene ut etter prioritet, dvs. at ved hvert uttak er det den med best eller høyest prioritet som velges. Det er verdienes ordning som bestemmer prioriteten. Den som etter ordningen er «minst», har best eller høyest prioritet. Dette betyr at hvis vi legger inn en samling verdier i en prioriteskø, får vi verdiene ut i stigende sortert rekkefølge. Bruker vi en UsortertTabellPrioritetsKø vil taUt-metoden hver gang finne den minste i tabellen, dvs. at dette er samme teknikk som i utvalgssortering. Hvis vi isteden bruker den andre prioritetskøen, dvs. en SortertTabellPrioritetsKø, vil leggInn-metoden hver gang legge nye verdier på rett sortert plass i tabellen, mens taUt-metoden tar ut den bakerste. Med andre ord er dette samme teknikk som i innsettingssortering.

12	11	8	7	3					
0	1	2	3	4	5	6	7	8	9

I tabellen over består prioritetskøen av den grå delen av tabellen og er, som vi ser, sortert avtagende. Det betyr at den minste verdien ligger bakerst. I metodene kikk og taUt er det bare å hente denne verdien og til det trengs én operasjon og dermed konstant orden. I metoden leggInn blir det derimot mer arbeid. Vi må lete oss frem til rett sortert plass og samtidig rydde plass til den nye verdien. Hvis køen har n verdier trengs i gjennomsnitt ca. n/2 sammenligninger og n/2 forflyttninger. Antall sammenligninger kan imidlertid reduseres hvis vi bruker binærsøk, men det vil fortsatt bli n/2 forflyttninger. Det betyr at leggInn-metoden blir av orden n eller av lineær orden.



SortertTabellPrioritetsKø

Metode	**Operasjoner**	**Orden**
leggInn	Det må letes etter rett sortert plass og de til høyre må forskyves.	*lineær*
taUt	Den minste ligger alltid bakerst.	*konstant*

Prioriteringskø kode

```
public class SortertTabellPrioritetsKø<T> implements PrioritetsKø<T> {
  private T[] a;                        // en usortert tabell
  private int antall;                   // antall verdier i køen
  private Comparator<? super T> c;      // en komparator

  public SortertTabellPrioritetsKø(int størrelse, Comparator<? super T> c) {
    a = (T[])new Object[størrelse];   // tabellens startstørrelse
    antall = 0;
    this.c = c;
  }

  public SortertTabellPrioritetsKø(Comparator<? super T> c) {
    this(8,c);  // bruker 8 som startstørrelse
  }

  public static <T extends Comparable<? super T>> PrioritetsKø<T> naturligOrdenKø() {
    return new SortertTabellPrioritetsKø<>(Comparator.naturalOrder());
  }


public void leggInn(T verdi) {
    if (antall == a.length) a = Arrays.copyOf(a, 2*antall+1);

    int i = antall - 1;   //  vi sammenligner og flytter
    for (; i >= 0 && c.compare(verdi,a[i]) > 0; i--) a[i+1] = a[i];
    a[i+1] = verdi;
    antall++;
  }

  public int antall() {
    return antall;
  }

  public boolean tom() {
    return antall == 0;
  }

  public T kikk() {
    if (tom()) throw new NoSuchElementException("Køen er tom!");
    return a[antall-1];
  }

  public T taUt() {
    if (antall == 0) throw new NoSuchElementException("Køen er tom!");

    T minverdi = a[--antall];      // tar vare på den bakerste
    a[antall] = null;              // klargjør for resirkulering
    return minverdi;               // returnerer den største
  }

  public void nullstill() {
    while (antall > 0) a[--antall] = null;
  }
```

Binære trær

Et binærtre består av en samling noder (eng: node/nodes) (muligens en tom samling) og en samling kanter (eng: edge/edges) som forbinder par av noder:

- Hvis treet ikke er tomt, har det en rotnode. Kalles også treets rot (eng: root).

- Til enhver node Y, unntatt rotnoden, hører det nøyaktig én node X som vi kaller dens foreldernode eller bare forelder (eng: parent). Det går en kant mellom noden Y og dens forelder X. Omvendt sier vi at Y er et barn (eng: children) til X. En node kan ha to, ett eller ingen barn. Det er kun mellom nodepar av typen barn/forelder det går kanter.

- Hvis en node har to barn er det ene venstre barn og det andre høyre barn. Hvis noden har bare ett barn defineres det enten som venstre barn eller som høyre barn.

Begreper

Nivå (eng: level) brukes vanligvis i forbindelse med høyder og høydeforskjell. Rotnoden er på nivå 0 (En bør være klar over at i enkelte fremstillinger sies rotnoden å være på nivå 1.) Barna til rotnoden er på nivå 1, barnebarna på nivå 2, osv. Det er vanlig å tegne alle nodene som hører til samme nivå på en og samme (vannrette) rad.

Begrepet generasjon henter vi fra slektstrær. Der kalles gjerne rotnoden for stamforelder (stammor eller stamfar) og nivåene svarer til generasjoner. Forskjellen er at stamforelder vanligvis kalles 1. generasjon. Dermed vil generasjon k være det samme som nivå $k - 1$.

Slektskap Det er ikke bare begrepet generasjon vi låner fra slektstreet. Vi bruker også begreper som barn og forelder, barnebarn og besteforelder, oldebarn og oldeforelder, etterkommer (eng: descendant, successor) og forgjenger (eng: ancestor, predecessor). To noder kalles søsken (eng: sibling) hvis de har samme forelder. En node (forskjellig fra rotnoden) kalles enebarn hvis den ikke har søsken. En node kalles barnløs hvis den ikke har barn.

Subtrær Enhver node kan ses på som rotnode i sitt eget tre, dvs. det treet som består av noden og alle dens etterkommere (barn, barnebarn, osv). En node har alltid to subtrær - et venstre subtre og et høyre subtre - der ett eller begge kan være tomme. Venstre subtre til en node er (hvis det ikke er tomt) det treet som har venstre barn som sin rotnode. Det blir tilsvarende for høyre subtre.

Bladnoder og indre noder Nodene kan deles opp i to typer - bladnoder og indre noder (eng: leaf node, inner node). En bladnode (eller et blad) er en node som ikke har barn, eller som har to tomme subtrær om en vil. Alle andre noder er indre noder. Det betyr at en indre node har ett eller to barn.

En vei Vi kan orientere alle kantene i treet ved å si at en kants retning er fra forelder til barn, dvs. nedover. Vi sier at det går en vei (eng: path) mellom to noder X og Y hvis det er mulig å komme fra X til Y ved å følge kanter. Spesielt får vi at Y er en etterkommer av X hvis det går en vei fra X til Y, eller omvendt at X er en forgjenger til Y. Veilengden er antallet kanter på veien.

Avstand Hvis det går en vei fra noden X til noden Y (eller fra Y til X) sier vi at avstanden mellom dem er lik veilengden. Hvis det ikke går en vei mellom dem, må de ha en nærmeste felles forgjenger Z. Da sier vi at avstanden mellom X og Y er lik avstanden mellom Z og X pluss avstanden mellom Z og Y.

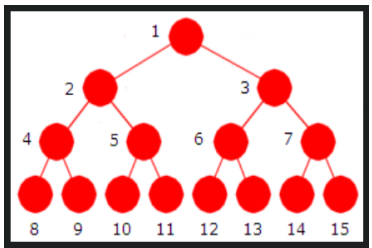
Høyden til et binærtre er lengden på den lengste veien i treet. Den lengste veien må nødvendigvis starte i rotnoden. En annen måte å si det på er at høyden er det samme som det største nivået i treet. Et binærtre med bare én node har høyde 0 og spesielt skal vi si at et tomt tre har høyde −1.

Dybden til en node X er avstanden mellom rotnoden og noden X. Dermed kan vi si at høyden til et binærtre er dybden til den «dypeste» noden. Det er alle nodene på den nederste nivået.

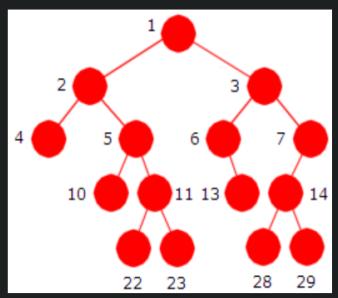
Retninger Vi tegner som sagt et binærtre opp-ned. Dermed vil oppover og nedover bli det omvendte av det normale med tanke på et botanisk tre. Nedover betyr nå i retning vekk fra roten. Tilsvarende blir oppover retning mot roten. Et uttrykk som «langt nede i treet» vil nå bety langt fra roten. Bunnen av et tre betyr så langt ned en kan komme.

En gren i treet består av alle nodene fra rotnoden og ned til en bladnode. Det betyr at det er like mange grener i treet som det er bladnoder.

Nodeposisjon



Figur 5.1.3 a) : Nodeposisjoner i et perfekt binærtre



Figur 5.1.3 b) : Posisjoner i et generelt binærtre. Barna til *k* er 2*k* og 2*k* + 1.

Til venstre i Figur 5.1.3 a) har vi et perfekt binærtre med 15 noder. Ved siden av hver node er det satt opp et heltall. Tallet kalles nodens posisjon (eller posisjonstall) og er bestemt av hvor i treet noden ligger. Når treet er perfekt slik som i Figur 5.1.3 a), er det lett å finne posisjonene til alle nodene. Rotnoden har alltid posisjon 1, og for de andre er posisjonene fortløpende heltall, dvs. forløpende for hvert nivå - fra venstre mot høyre.

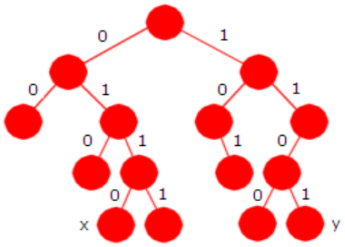
I Figur 5.1.3 b) til venstre har vi et mer generelt binærtre. Vi kan finne posisjoner som i et perfekt tre. Roten har alltid posisjon 1. Men vi må hoppe over de tallene som representerer manglende noder. Nodene har posisjoner 1, 2, 3, 4, 5, 6, 7, 10, 11, 13, 14, 22, 23, 28 og 29. Tallene 8, 9 og 12 (og enda flere) mangler fordi nodene som hører til de posisjonene ikke er i treet.

Flg. regel er enklere: Hvis en node har posisjon k, så har de to barna posisjonene 2k (venstre) og 2k + 1 (høyre). Dette gjelder fra og med rotnoden (posisjon 1). Ta f.eks. noden med posisjon 5. De to barna har posisjoner $2 \cdot 5 = 10$ og $2 \cdot 5 + 1 = 11$.

Det er en tilsvarende regel motsatt vei: Hvis en node har posisjon k ($k \neq 1$), så er $\lfloor k/2 \rfloor$ posisjonen til foreldernoden. Ta f.eks. noden med posisjon 5. Det gir posisjon $\lfloor 5/2 \rfloor = 2$ for forelderen. Obs: Vi kan skrive k/2 istedenfor $\lfloor k/2 \rfloor$ hvis vi tolker divisjonen som heltallsdivisjon.

Konklusjon Vi kan bruke flg. regel for å bestemme nodeposisjoner: Rotnoden har posisjon 1. Deretter brukes «barnereglen»: Barna til en node med posisjon k har henholdsvis 2 k (venstre) og 2 k + 1 (høyre) som posisjonstall.

Nodeposisjon og binære tall



Binære tall henger sammen med binære trær. Treet i Figur 5.1.4 a) til venstre er det samme som det i Figur 5.1.3 b), men uten posisjonstall. Isteden er det satt på binære siffer på kantene: 0 på hver venstrekant og 1 på hver høyrekant.

Noden med posisjon 22 er markert med x. Hvis vi starter med et 1-tall (et 1-tall for rotnoden) og så går nedover ved å følge kantene fra rotnoden ned til x-noden, får vi følgende binære sekvens: 10110. Denne sekvensen er nettopp tallet 22 på binærform.

Noden med posisjon 29 er markert med en y. Binærformen til 29 er 11101 og det får vi (bortsett fra den første 1-eren) ved å følge kantene nedover fra rotnoden og ned til y-noden.

Det blir nå enkelt å finne frem til en node eller sette inn en node på en oppgitt posisjon hvis posisjonstallet er gitt. La k være et posisjonstall. Hvis k er 1, svarer det til rotnoden. Hvis ikke, finner vi først de binære sifrene til k, ser bort fra den ledende 1-eren, starter i rotnoden, følger de binære sifrene, dvs. går til venstre når det er 0 og til høyre når det er 1.

Vi får flg. oppsummering:

-Hver node i et binærtre har en entydig posisjon (eller et posisjonstall) og det er nodens plassering i treet som bestemmer dette tallet.

- Rotnoden har alltid posisjon 1.

- Hvis k, k > 1, er posisjonen til en node, så er $\lfloor k/2 \rfloor$ posisjonen til nodens forelder.

- Hvis en node med posisjon k har et venstre barn, så er ventrebarnets posisjon lik 2k.

- Hvis en node med posisjon k har et høyre barn, så er høyrebarnets posisjon lik 2k + 1.

- Hvis k, k > 1, er en posisjon i binærtreet, så gir de binære sifrene til k oss veien fra rotnoden ned til denne posisjonen. Vi hopper over det ledende 1-tallet. Deretter går vi til venstre når det er 0 og til høyre når det er 1.

- Høyden i et tre er én mindre enn antallet binære siffer i treets største posisjonstall.

Binærtre kode

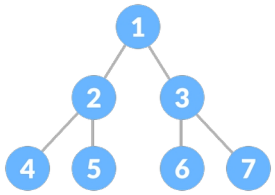
En klasse for et generisk binærtre må ha en indre nodeklasse med tre instansvariabler (en verdi og to nodereferanser), konstruktører og metoder som gjør det mulig å bygge opp et tre:

```
public class BinTre<T> {           // et generisk binærtre
   private static final class Node<T> {  // en indre nodeklasse
     private T verdi;            // nodens verdi
     private Node<T> venstre;    // referanse til venstre barn/subtre
     private Node<T> høyre;      // referanse til høyre barn/subtre

     private Node(T verdi, Node<T> v, Node<T> h) {    // konstruktør
       this.verdi = verdi; venstre = v; høyre = h;
     }

     private Node(T verdi) { this.verdi = verdi; }  // konstruktør

   }

   private Node<T> rot;      // referanse til rotnoden
   private int antall;       // antall noder i treet

   public BinTre() { rot = null; antall = 0; }          // konstruktør

   public final void leggInn(int posisjon, T verdi) {}  // kode utelatt

   public int antall() { return antall; }               // returnerer antallet

   public boolean tom() { return antall == 0; }         // tomt tre?

 }
```

Typer binære trær

Fire ulike typer binære trær og definisjonen på hver av de:

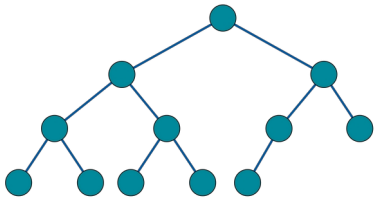
Perfekt tre



Alle nivåer er fulle. Et perfekt tre er derfor også et komplett og fullt tre.

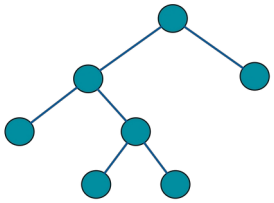
Høyde: log2(n+1) = x Antall noder: (2^h-1)

Komplett tre



Alle nivåer er fulle, bortsett fra evt. siste, som er <u>fylt opp fra venstre</u>. Treet i illustrasjonen er ikke et perfekt eller fullt tre. Høyde: ⌊log 2 n⌋

Fullt tre



Alle noder har null eller to barn. Treet i illustrasjonen er ikke et perfekt eller komplett tre. Høyde: ⌊log 2 n⌋

Turneringstre

Et turneringstre blir gjerne brukt i en fotballturnering, el. Et turneringstre er både et komplett og fullt tre.

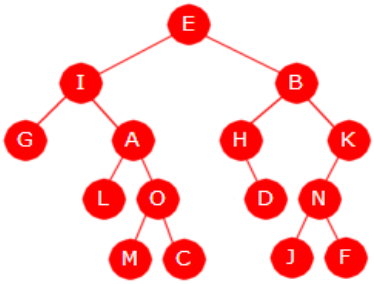
https://www.cs.hioa.no/~ulfu/appolonius/kap1/2/kap12.html#1.2.12

Maksimumstre: Hver node bortsett fra roten har en verdi mindre eller lik verdien til nodens forelder.
Maksimumsheap: Komplett maksimumstre

Traverseringer

Vi har to hovedtyper av traverseringer. Vi kan for hver node gå til siden før vi går nedover, dvs. **«bredde først» (eng: breadth-first)**. Alternativt kan vi gå nedover før vi går til siden, dvs. **«dybde først» (eng: depth-first)**. Rekkefølgen nivåorden er av typen «bredde først» og preorden, inorden og postorden er alle av typen «dybde først».

Nivåorden (bredde først)



Nivåorden er av typen **«bredde først»**. Det kalles nivåorden når nodene «besøkes» nivå for nivå, fra rotnoden og nedover, og for hvert nivå fra venstre mot høyre. I figuren til venstre vil en oppramsing av nodeverdiene i nivåorden gi oss denne rekkefølgen: E, I, B, G, A, H, K, L, O, D, N, M, C, J og F. Dette blir en «bredde først»-traversering.

Traversering av typen «bredde først» utføres ved hjelp av en kø. I «dybde først» brukes enten rekursjon eller en stakk.

```
public void nivåorden() {
    if (tom()) return;                    // tomt tre

    Kø<Node<T>> kø = new TabellKø<>();    // Se avsnitt om kø
    kø.leggInn(rot);                      // legger inn roten

    while (!kø.tom())                     // så lenge som køen ikke er tom
    {
      Node<T> p = kø.taUt();              // tar ut fra køen
      System.out.print(p.verdi + " ");    // skriver ut

      if (p.venstre != null) kø.leggInn(p.venstre);
      if (p.høyre != null) kø.leggInn(p.høyre);
    }
  }
```

Først legges rotnoden (*E*-noden) i køen. Så går while-løkken så lenge som køen ikke er tom. Tabellen viser de 6 første iterasjonene.

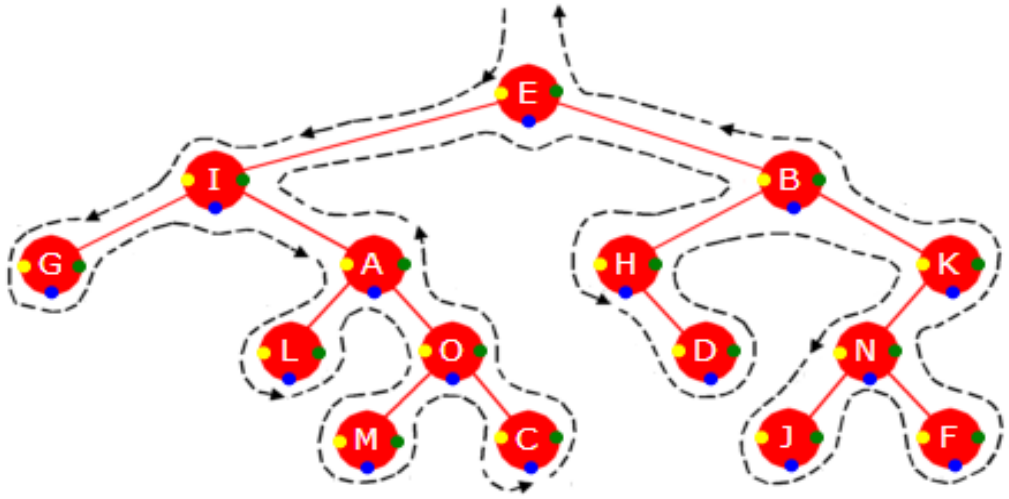
Runde	Ut av køen	Inn i køen	Køens innhold
1	E	I , B	I , B
2	I	G , A	B , G , A
3	B	H , K	G , A , H , K
4	G		A , H , K
5	A	L , O	H , K , L , O
6	H	D	K , L , O , D

Hvis vi for hver runde lagrer i en tabell de tallene antall-metoden gir oss, kan den tabellen etterpå gi oss informasjon om treet, f.ek. både treets bredde og treets høyde.

```
public int[] nivåer() {   // returnerer en tabell som inneholder nivåantallene
    if (tom()) return new int[0];       // en tom tabell for et tomt tre

    int[] a = new int[8];               // hjelpetabell
    Kø<Node<T>> kø = new TabellKø<>();  // hjelpekø
    int nivå = 0;                       // hjelpevariabel

    kø.leggInn(rot);    // legger roten i køen

    while (!kø.tom()) {  // så lenge som køen ikke er tom
      // utvider a hvis det er nødvendig
      if (nivå == a.length) a = Arrays.copyOf(a,2*nivå);

      int k = a[nivå] = kø.antall();  // antallet på dette nivået

      for (int i = 0; i < k; i++)  // alle på nivået
      {
        Node<T> p = kø.taUt();

        if (p.venstre != null) kø.leggInn(p.venstre);
        if (p.høyre != null) kø.leggInn(p.høyre);
      }

      nivå++;  // fortsetter på neste nivå
    }

    return Arrays.copyOf(a, nivå);  // fjerner det overflødige
  }
```

Dybde først

Dybde først vil si at vi traverserer nedover i binærtreet før vi går til siden.



Figuren viser et binærtre med konturkurve. Hver prikk med farge representerer de ulike ordnene.

Hvis vi starter ved rotnoden, følger konturkurven og skriver ut nodeverdiene ved passering av en farget «prikk», får vi flg. tre tilfeller:

1) Skriver vi ut nodeverdien når den gule «prikken» passeres, får vi verdiene i preorden. For treet i figuren over blir det E, I, G, A, L, O, M, C, B, H, D, K, N, J, F. Rotnoden kommer alltid først i preorden!

2) Skriver vi ut nodeverdien når den blå «prikken» passeres, får vi verdiene i inorden og dermed G, I, L, A, M, O, C, E, H, D, B, J, N, F, K for treet i figuren. Noden nederst til venstre kommer alltid først i inorden!

3) Skriver vi ut nodeverdien når den grønne «prikken» passeres, får vi dem i postorden. Det blir da G, L, M, C, O, A, I, D, H, J, F, N, K, B, E. Rotnoden kommer alltid sist i postorden!

<u>Samlet kode for dybde først</u>

```
void printTree(Node node) {
    if (node == null) {
        return;
    }
    else {
      println("Preorden " + node.value);
      printTree(node.left_child);
      println("Inorden " + node.value);
      printTree(node.right_child);
      println("Postorden " + node.value);
    }
}
```

Preorden

Det er lettest å definere denne traverseringsrekkefølgen rekursivt:

- Vi starter i rotnoden.

- Videre gjelder for alle noder at først «besøker» vi noden, så dens venstre barn hvis den har et venstre barn og så dens høyre barn hvis den har et høyre barn.

En enkel huskeregel for preorden er: *node, venstre, høyre*

Både nivåorden og preorden har den egenskapen at når en node (forskjellig fra rotnoden) besøkes er nodens forelder allerede besøkt.

<u>Preorden kode</u>

```
private static <T> void preorden(Node<T> p, Oppgave<? super T> oppgave)
  {
    if (p != null) { // metoden returnerer hvis p == null

      oppgave.utførOppgave(p.verdi);//Kaller på et oppgave interface. Utfører en oppgave, feks skriv ut

      preorden(p.venstre,oppgave);
      preorden(p.høyre,oppgave);
    }
  }
```

<u>Finne første preorden:</u>

```
public T førstPreorden() {
    if (tom()) throw new NoSuchElementException("Treet er tomt!");

    Node<T> p = rot;
    while (true) {
      if (p.høyre != null) p = p.høyre;
      else if (p.venstre != null) p = p.venstre;
      else return p.verdi;
    }
  }
```

<u>Finne neste preorden:</u>

- Hvis p har et venstre barn, så er det barnet den neste.

- Hvis p ikke har et venstre barn, men et høyre barn, så er det barnet den neste.

- Hvis p ikke har barn, dvs. p er en bladnode, så må vi først til den nærmeste (oppover mot roten) noden q som har et høyre barn og som har p i sitt venstre subtre. Den neste til p er da det høyre barnet til q.

- Hvis det ikke finnes noen slik q, er p den siste i preorden.

Inorden

Inorden Dette definerer vi også rekursivt:

- Vi starter i rotnoden.

- Videre gjelder for alle noder at først «besøker» vi nodens venstre barn hvis den har et venstre barn, så noden og så dens høyre barn hvis den har et høyre barn.

En enkel huskeregel for inorden er: *venstre, node, høyre*

Koden for en inordentraversering er den samme som for preorden. Det er bare å bytte ut metodenavnene og flytte oppgavekallet mellom de to rekursive kallene. Husk at pre betyr foran, dvs. oppgavekallet skal stå foran de to rekursive kallene, mens in betyr mellom, dvs. at oppgavekallet skal være mellom de to kallene.

Inorden kode

```
private static <T> void inorden(Node<T> p, Oppgave<? super T> oppgave) {
   if (p.venstre != null) {
   inorden(p.venstre,oppgave);

   oppgave.utførOppgave(p.verdi); //Kaller på et oppgave interface. Utfører en oppgave, feks skriv ut

   if (p.høyre != null)
      inorden(p.høyre,oppgave);
 }
```

Finne første inorden:

```
public T førstInorden() {
   if (tom()) throw new NoSuchElementException("Treet er tomt!");

   Node<T> p = rot;
   while (p.venstre != null) p = p.venstre;

   return p.verdi;
 }
```

Finne neste inorden:

- Hvis p har et ikke-tomt høyre subtre, så er den neste den noden som kommer først i inorden i det subtreet.

- Hvis p har et tomt høyre subtre, er den neste den nærmeste noden oppover mot roten som har p i sitt venstre subtre.

- Hvis det ikke finnes noen slik node, er p den siste i inorden.

Postorden

Postorden En rekursiv definisjon:

- Vi starter i rotnoden.

- Videre gjelder for alle noder at først «besøker» vi nodens venstre barn hvis den har et venstre barn, så dens høyre barn hvis den har et høyre barn og så «besøker» vi noden.

En enkel huskeregel for postorden er: *venstre, høyre, node*

I postorden vil en nodes to subtrær allerede være «besøkt» når noden selv skal «besøkes». Dette kan utnyttes til å lage korte og enkle algoritmer for f.eks. det å finne høyden og antall noder i et tre og for metoden nullstill(). Kode for en traversering i postorden lages på samme måte som for preorden og inorden.

Postorden kode

```
 private static <T> void postorden(Node<T> p, Oppgave<? super T> oppgave) {
    if (p.venstre != null) {
    postorden(p.venstre,oppgave);

    if (p.høyre != null)
       postorden(p.høyre,oppgave);

    oppgave.utførOppgave(p.verdi); //Kaller på et oppgave interface. Utfører en oppgave, feks skriv ut
  }
```

Finne første postorden:

```
public T førstPostorden() {
    if (tom()) throw new NoSuchElementException("Treet er tomt!");

    Node<T> p = rot;
    while (true) {
      if (p.venstre != null) p = p.venstre;
      else if (p.høyre != null) p = p.høyre;
      else return p.verdi;
    }
  }
```

Finne neste postorden:

- Hvis p ikke har en forelder (p er rotnoden), så er p den siste i postorden.

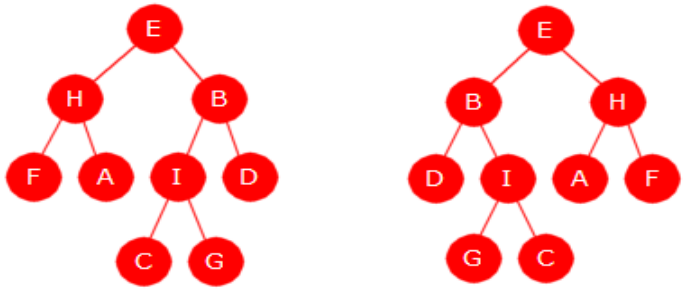
- Hvis p er høyre barn til sin forelder f, er forelderen f den neste.

- Hvis p er venstre barn til sin forelder f, gjelder:

    - Hvis p er enebarn (f.høyre er null), er forelderen f den neste.

    - Hvis p ikke er enebarn (dvs. f.høyre er ikke null), så er den neste den noden som kommer først i postorden i subtreet med f.høyre som rot.

Speilvendt og omvendt orden



Et binærtre speilvendes ved å rotere det om en vertikal linje gjennom roten, dvs. venstre og høyre barn bytter plass i alle noder. Trærne i figuren til venstre er speilvendte av hverandre.

En speilvendingsmetode kan vi lage ved å traversere treet og bytte om barna til nodene.

```
private static <T> void speilvend(Node<T> p) {
    if (p == null) return;   // tomt subtre

    Node<T> q = p.venstre; p.venstre = p.høyre; p.høyre = q;  // bytter

    speilvend(p.venstre); speilvend(p.høyre);
  }

  public void speilvend()
  {
    if (antall() > 1) speilvend(rot);
  }
```

Speilvendt nivåorden er det samme som nivåorden i det speilvendte treet eller at nodene «besøkes» nivå for nivå, fra rotnoden og nedover, og for hvert nivå fra høyre mot venstre. En nivåorden utskrift i det speilvendte treet vil gi speilvendt nivåorden.

```
public void speilvendtNivåorden(Oppgave<? super T> oppgave) {
    if (tom()) return;  // tomt tre
    Kø<Node<T> kø = new TabellKø<>();   // Se notater om kø
    kø.leggInn(rot);

    while (!kø.tom()) {
      Node<T> p = kø.taUt();
      oppgave.utførOppgave(p.verdi);     // den generiske oppgaven

      if (p.høyre != null) kø.leggInn(p.høyre);      // tar høyre først
      if (p.venstre != null) kø.leggInn(p.venstre);
    }
  }
```

Omvendt nivåorden er det samme som nivåorden i omvendt (eller baklengs) rekkefølge. Det er ikke mulig å lage en algoritme som starter i den siste noden i nivåorden og som så beveger seg gjennom treet nivå for nivå oppover mot roten. Men vi kan få det til ved at vi traverserer treet i nivåorden og legger verdiene (eller nodene) fortløpende i en datastruktur som det er mulig å traversere baklengs.

Speilvendt preorden, inorden og postorden er det samme som henholdsvis preorden, inorden og postorden i det speilvendte treet. Men slike traverseringer kan vi selvfølgelig få til uten å måtte speilvende treet først. Det gjøres rett og slett ved å bytte om venstre og høyre. For preorden blir det: node, høyre, venstre. Koden blir slik:

```
private static <T> void speilvendtPreorden(Node<T> p, Oppgave<? super T> oppgave) {
  oppgave.utførOppgave(p.verdi);
  if (p.høyre != null) speilvendtPreorden(p.høyre,oppgave);      // høyre først
  if (p.venstre != null) speilvendtPreorden(p.venstre,oppgave);  // så venstre
}

public void speilvendtPreorden(Oppgave<? super T> oppgave) {
  if (!tom()) speilvendtPreorden(rot,oppgave);
}
```

Det omvendte av preorden, inorden og postorden får vi rett og slett ved å gå omvendt vei i forhold til den ordinære veien. Omvendt preorden og speilvendt postorden blir alltid det samme.

Generelt:

- Omvendt preorden er det samme som speilvendt postorden

- Omvendt postorden er det samme som speilvendt preorden

- Omvendt inorden er det samme som speilvendt inorden

Traversering uten rekursjon

I et første forsøk på å lage en iterativ preorden, legger vi først p.høyre på stakken og deretter p.venstre. Dermed vil p.venstre ligge høyere opp enn p.høyre og vil bli tatt ut først av de to.

Iterativ inorden er også mulig. Den første i inorden får vi ved å starte i roten og gå mot venstre så langt det går.

1. En peker p må starte i rotnoden (K-noden) og flyttes ned mot venstre så langt det går (til H-noden). De nodene vi passerer på veien (K-noden , C-noden og F-noden) legges på stakken. Dermed vil p peke på den første noden i inorden.

2. En løkke settes i gang. Først utføres oppgaven på verdien til p (dvs. på H). Deretter skal p flyttes til den neste i inorden. Da sjekkes først om p har et høyre barn. Hvis ja, flyttes p dit og videre mot venstre så langt det går hvis det er flere noder den veien. På veien legges de passerte nodene på stakken. Hvis nei, dvs. at p ikke har høyre barn, er den neste rett og slett den vi henter fra stakken. Osv.

Binære søketrær

Binære søketre ligger i navnet. Det er binære trær som kan søkes i/brukes til søk. Konseptet er å opprette et binærtre og legge inn tallene fortløpende. Første tall i rot, deretter legges hvert tall inn fortløpende. Et binært søketre trenger ikke være perfekt, komplett eller fullt tre.

Binærsøk er effektiv siden den er av logaritmisk orden. Det er imidlertid kostbart å vedlikeholde en ordnet tabell. Innlegging på rett sortert plass er av orden *n*. Også fjerning er av orden *n* siden «hullet» må tettes igjen. Et binært søketre er en datastruktur der søkingen i gjennomsnitt er omtrent like effektiv som binærsøk. I tillegg er også innlegging og fjerning i gjennomsnitt av logaritmisk orden. Prisen er imidlertid at vi bruker mer plass enn det en tabell trenger.

Definisjon: For hver node p gjelder:
1) hvis p har et ikke-tomt venstre subtre, er alle verdiene der mindre enn verdien i p
2) hvis p har et ikke-tomt høyre subtre, er alle verdiene der større enn eller lik verdien i p.
Et tomt tre og et tre med kun én node er per definisjon et binært søketre.

Gjennomsnittlig nodedybde for binære søketrær uten duplikatverdier

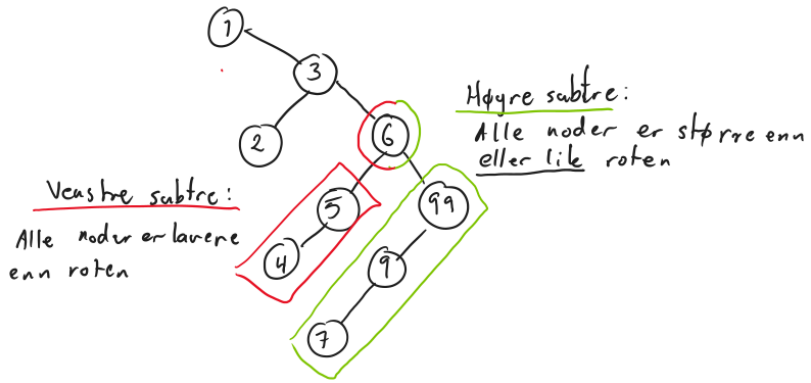
	Balansert tre	**Gjennomsnitt for alle binære søketrær**	**Ekstremt skjevt tre**
***n* noder**	$\log_2(n + 1) - 2$	$1{,}386 \log_2(n) - 2{,}846$	$(n - 1)/2$
***n* = 100**	4,7	6,4	49,5
***n* = 1000**	8,0	11,0	499,5
***n* = 100.000**	14,6	20,2	49999,5

Obs: Analysen over forutsetter at vi har binære søketrær uten duplikatverdier. Hvis det er like verdier, kan det bli helt annerledes. Det mest ekstreme er at alle verdiene er like. Da vil treet bli ekstremt høyreskjevt, dvs. ingen noder har venstre barn. Hvis treet har *n* verdier og alle er like, vil gjennomsnittlig nodedybde bli $(n - 1)/2$.

Innlegging

Alle noder **lavere** enn roten/bladnoden legges til **venstre subtre**, mens alle noder større enn eller lik roten/bladnoden legges til høyre subtre.

Gitt tallrekka [1, 3, 6, 99, 9, 7, 2, 5, 4]



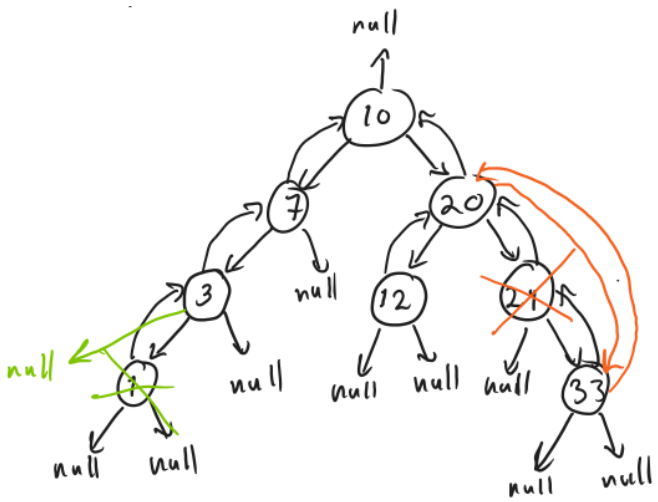
Treet over er et ugunstig søketre, fordi det er ubalansert. Treet har 5 nivåer. Skal vi søke etter tallet 7 må vi da gjøre 6 sammenlikninger før vi finner tallet. Et balansert tre vil da være mer effektivt fordi vi får færre nivåer å søke gjennom, og da også færre sammenlikninger.

Som sortert array

Se kapittel om binærsøk nærmere start

I likhet med binærsøk så finner vi venstre, høyre og midten av tabellen. Vi finner midtverdien og lager den om til en node. Så finner vi midtverdien mellom midt og høyre og legger inn den noden i treet. På lik linje som binærsøk fortsetter vi med dette til vi har alle nodene inn i treet. Da får vi til slutt midtverdien av hele tabellen som rot og alt til venstre i venstre del av subtreet og visa versa.

Fjerning av node

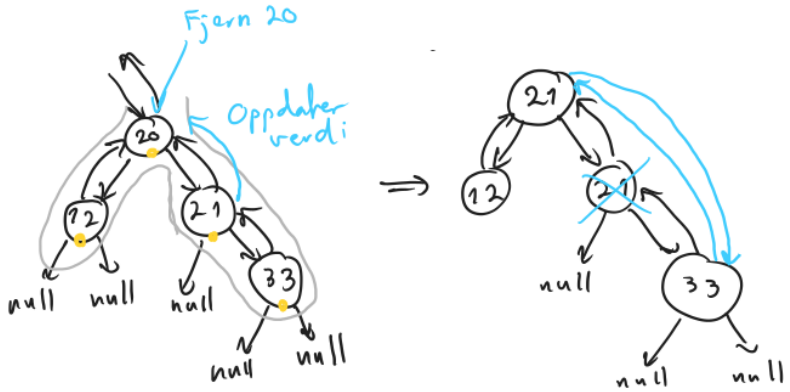


Vi kan ha tre tilfeller når vi skal fjerne noder i et binærtre:

1. Bladnode (ingen barn): Sett foreldre- og barnepeker til null

2. Ett barn: Sett r sin barnepeker til q og q sin foreldrepeker til r:

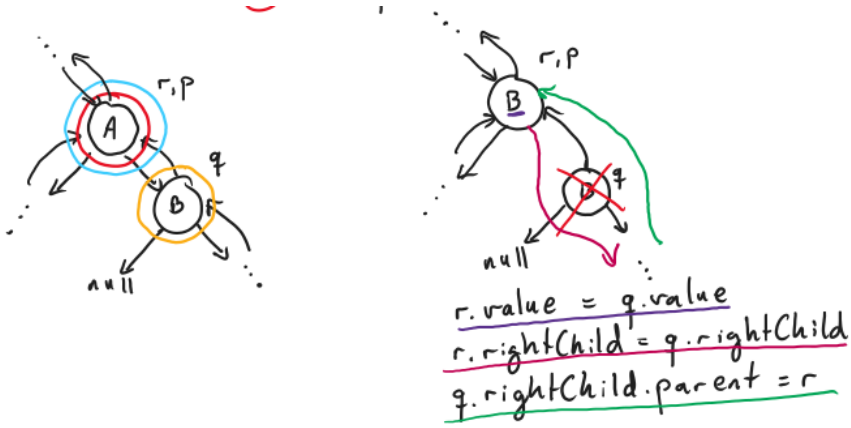
3. To barn: Bytt ut med neste inorden:



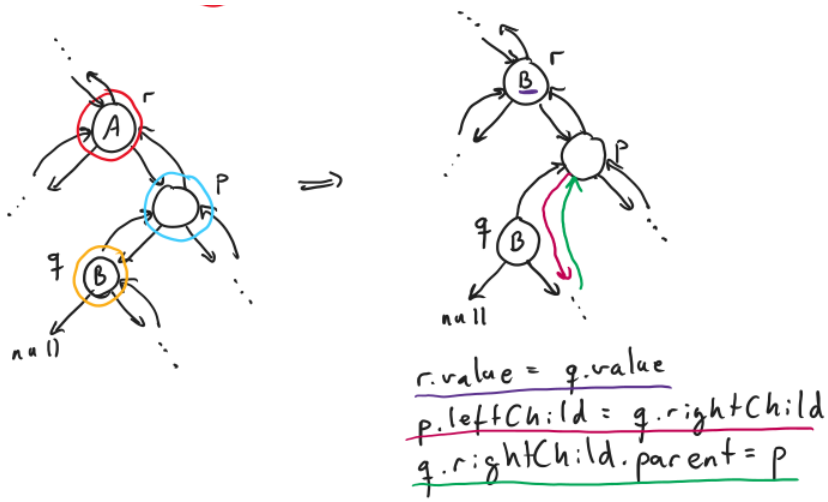
Generelt for fjerning med to barn:

For noden som skal fjernes (r), finn neste node i inorden (q) og dens forelder (p)

Tilfelle 1: r = p



Tilfelle 2: r ≠ p



Binærtre kode

```
public class BinTre<T> implements Iterable<T> {  // et generisk binærtre
  private static final class Node<T> {  // en indre nodeklasse

    private T verdi;            // nodens verdi
    private Node<T> venstre;    // peker til venstre barn/subtre
    private Node<T> høyre;      // peker til høyre barn/subtre

    private Node(T verdi, Node<T> v, Node<T> h) {    // konstruktør
      this.verdi = verdi;
      venstre = v;
      høyre = h;
    }

    private Node(T verdi) {  // konstruktør
      this.verdi = verdi;
    }

  } // class Node<T>

  private Node<T> rot;      // peker til rotnoden
  private int antall;       // antall noder i treet

  public BinTre() {          // standardkonstruktør
    rot = null;
    antall = 0;
  }

  public BinTre(int[] posisjon, T[] verdi) {  // konstruktør
    if (posisjon.length > verdi.length) throw new
      IllegalArgumentException("Verditabellen har for få elementer!");

    for (int i = 0; i < posisjon.length; i++) {
      leggInn(posisjon[i],verdi[i]);
    }
  }
```

Kopiere

```
  public BinTre(final BinTre<T> tre) {  // kopieringskonstruktør
    rot = kopi(tre.rot);  // kaller den rekursive kopieringsmetoden
    antall = tre.antall;
  }

  private static <T> Node<T> kopi(Node<T> p) {
    if (p == null) return null;    // ingenting å kopiere
    return new Node<>(p.verdi,kopi(p.venstre),kopi(p.høyre));
  }
```

Legg inn

```
public final void leggInn(int k, T verdi) {
  if (k < 1) throw new
    IllegalArgumentException("Posisjon k(" + k + ") < 1!");

  Node<T> p = rot, q = null;               // hjelpepekere

  int n = Integer.highestOneBit(k >> 1);   // n = 100...00

  while (p != null && n > 0) {
    q = p;
    p = (n & k) == 0 ? p.venstre : p.høyre;
    n >>= 1;  // bitforskyver n
  }

  if (n > 0) throw new
    IllegalArgumentException("Posisjon k(" + k + ") mangler forelder!");
  else if (p != null) throw new
    IllegalArgumentException("Posisjon k(" + k + ") finnes fra før!");

  p = new Node<>(verdi);  // ny node

  if (q == null) rot = p;   // tomt tre - ny rot
  else if ((k & 1) == 0)    // sjekker siste siffer i k
    q.venstre = p;
  else
    q.høyre = p;

  antall++;  // en ny verdi i treet
}
```

Antall og tom

```
public int antall() {
  return antall;   // returnerer antallet
}

public boolean tom() {
  return antall == 0;
}
```

Finn node

```
private Node<T> finnNode(int k) {    // henter noden i posisjon k
  if (k < 1) return null;

  Node<T> p = rot;                         // hjelpepeker
  int n = Integer.highestOneBit(k >> 1);   // n = 100...00

  for (; p != null && n > 0; n >>= 1) {
    p = (k & n) == 0 ? p.venstre : p.høyre;
  }

  return p;   // p blir null hvis k ikke er i treet
}
```

Finnes og hent

```
public boolean finnes(int k) {
  return finnNode(k) != null;
}

public T hent(int k) {
  Node<T> p = finnNode(k);

  if (p == null) throw new
    IllegalArgumentException("Posisjon k(" + k + ") er ukjent!");

  return p.verdi;
}
```

Oppdater

```
public T oppdater(int k, T nyverdi) {
  Node<T> p = finnNode(k);

  if (p == null) throw new
    IllegalArgumentException("Posisjon k(" + k + ") er ukjent!");

  T gammelverdi = p.verdi;
  p.verdi = nyverdi;
  return gammelverdi;
}
```

Inneholder

```
private static <T> boolean inneholder(Node<T> p, T verdi) {
  if (p == null) return false;  // kan ikke ligge i et tomt tre
  return verdi.equals(p.verdi) || inneholder(p.venstre,verdi)
         || inneholder(p.høyre,verdi);
}

public boolean inneholder(T verdi) {
  return inneholder(rot,verdi); // kaller den private metoden
}
```

Posisjon

```
private static <T> int posisjon(Node<T> p, int k, T verdi) {
  if (p == null) return -1;                  // ligger ikke i et tomt tre
  if (verdi.equals(p.verdi)) return k;       // verdi ligger i p
  int i = posisjon(p.venstre,2*k,verdi);     // leter i venstre subtre
  if (i > 0) return i;                       // ligger i venstre subtre
  return posisjon(p.høyre,2*k+1,verdi);      // leter i høyre subtre
}

public int posisjon(T verdi) {
  return posisjon(rot,1,verdi);  // kaller den private metoden
}
```

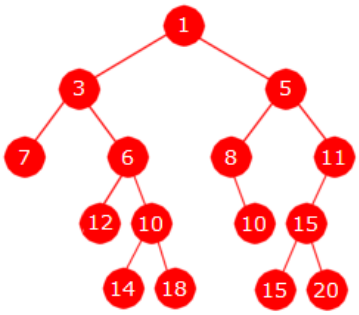
Fjern

```
public T fjern(int k) {
  if (k < 1) throw new
    IllegalArgumentException("Posisjon k(" + k + ") < 1!");

  Node<T> p = rot, q = null;               // hjelpepekere
  int n = Integer.highestOneBit(k >> 1);   // binært siffer

  while (p != null && n > 0) {
    q = p;
    p = (n & k) == 0 ? p.venstre : p.høyre;
    n >>= 1;
  }

  if (p == null) throw new
    IllegalArgumentException("Posisjon k(" + k + ") er utenfor treet!");

  if (p.venstre != null || p.høyre != null) throw new
    IllegalArgumentException("Posisjon k(" + k + ") er ingen bladnode!");

  if (p == rot) rot = null;
  else if (p == q.venstre) q.venstre = null;
  else q.høyre = null;

  antall--;  //
  return p.verdi;
}
```

Nullstill

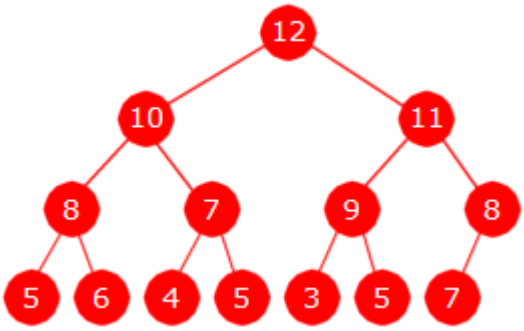
```
public void nullstill() {
  if (!tom()) nullstill(rot);  // nullstiller
  rot = null; antall = 0;      // treet er nå tomt
}

private void nullstill(Node<T> p) {
  if (p.venstre != null) {
    nullstill(p.venstre);      // venstre subtre
    p.venstre = null;          // nuller peker
  }
  if (p.høyre != null) {
    nullstill(p.høyre);        // høyre subtre
    p.høyre = null;            // nuller peker
  }
  p.verdi = null;              // nuller verdien
}
```

Minimums og maksimumstrær



Bildet under viser et binærtre med heltall som nodeverdier. Rotnoden har verdien 1. Begge barna har verdier som er større, dvs. 3 og 5. Hvis vi ser på alle de indre nodene (noder som har ett eller to barn) vil vi se at verdien i hver slik node alltid er mindre enn eller lik verdien i barnet eller barna. Hvis vi går fra roten og ned til en bladnode, kommer verdiene i sortert stigende rekkefølge.

Definisjon: Et binærtre kalles et minimumstre hvis verdien i hver indre node er mindre enn eller lik verdiene i nodens barn. For et maksimumstre er det det motsatte som gjelder.



Minimumstre:

- Et binærtre er et minimumstre hvis og bare hvis hver gren er sortert stigende.
- I et minimumstre er rotnodeverdien mindre enn eller lik alle de andre verdiene.
- Ethvert subtre i et minimumstre er selv et minimumstre.
- Hvis treet har minst to verdier ligger nest minste verdi i et av rotnodens barn.

Maksimumstre:

- Et binærtre er et maksimumstre hvis og bare hvis hver gren er sortert avtagende.
- I et maksimumstre er rotnodeverdien større enn eller lik alle de andre verdiene.
- Ethvert subtre i et maksimumstre er selv et maksimumstre.
- Hvis treet har minst to verdier ligger nest største verdi i et av rotnodens barn.

Kode for å sjekke minimumstre

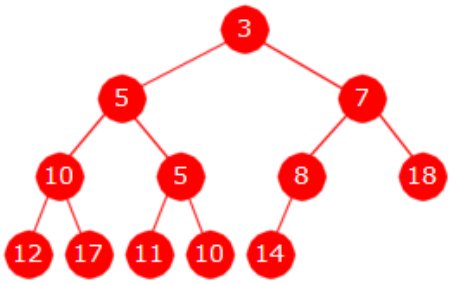
```
public boolean erMintre(Comparator<? super T> c) {
    if (rot == null) return true;    // et tomt tre er et minimumstre
    else return erMintre(rot,c);     // kaller den private hjelpemetoden
  }

  private static <T> boolean erMintre(Node<T> p, Comparator<? super T> c) {
    if (p.venstre != null) {
      if (c.compare(p.venstre.verdi,p.verdi) < 0) return false;
      if (!erMintre(p.venstre,c)) return false;
    }
    if (p.høyre != null) {
      if (c.compare(p.høyre.verdi,p.verdi) < 0) return false;
      if (!erMintre(p.høyre,c)) return false;
    }
    return true;
  }
```

Heap

En heap er en gruppe med ting som ligger oppå hverandre. På norsk kan det oversettes til haug.

Binære heaper

En binær minimums/maksimums heap er et binært og komplett minimums/maksimumstre.



Treet over viser et binærtre med 12 noder. Vi ser at hver gren er sortert stigende og dermed er det et minimumstre. Treet er komplett siden alle nivåer, bortsett fra det siste, er fulle av noder og det siste nivået er fylt opp fra venstre. Med andre ord en minimumsheap. Når verdier skal legges inn og tas ut, må det skje på en slik måte at treet bevares som en minimumsheap, dvs. at både minimumsegenskapen og komplettheten bevares.

Innlegging i minimumsheap

Hvis en verdi skal legges, må nødvendigvis treet få en ekstra node og for å bevare komplettheten må den legges på bunnen (nederste nivå rett til høyre for den siste). Hvis det nederste nivået er fullt, må den isteden legges som første node (lengst til venstre) på et nytt nivå. Siden den nye noden alltid blir en bladnode, blir den siste node på en ny gren.

Legg inn ***value*** i en minimumsheap:

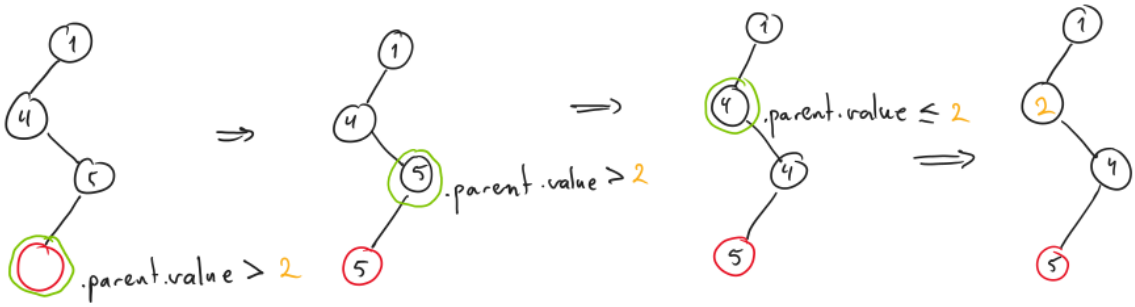
1. Legg til ny node **p** på første ledige plass i heapen

2. Sett node **q** = node **p**

3. while (**q**.parent ≠ null og **q**.parent.value > ***value***)

    q.value = **q**.parent.value

    q.value = ***value***



Uttak fra minimumsheap

Den minste verdien (som ligger i rotnoden) skal tas ut. Men rotnoden skal ikke fjernes. Isteden fjerner vi den siste noden (den lengst til høyre på nederste rad). Vi tar vare på verdien siden den skal inn på et annet sted i treet.

Uttak:

Vi tar alltid øverste node **p** (minste/største element) ved å sette det inn i siste node **q**.

1. Bytt om **p**.value og **q**.value

2. while (ingen av p sine barn == q og p.value > max(p.leftChild.value, p.rightChild.value))

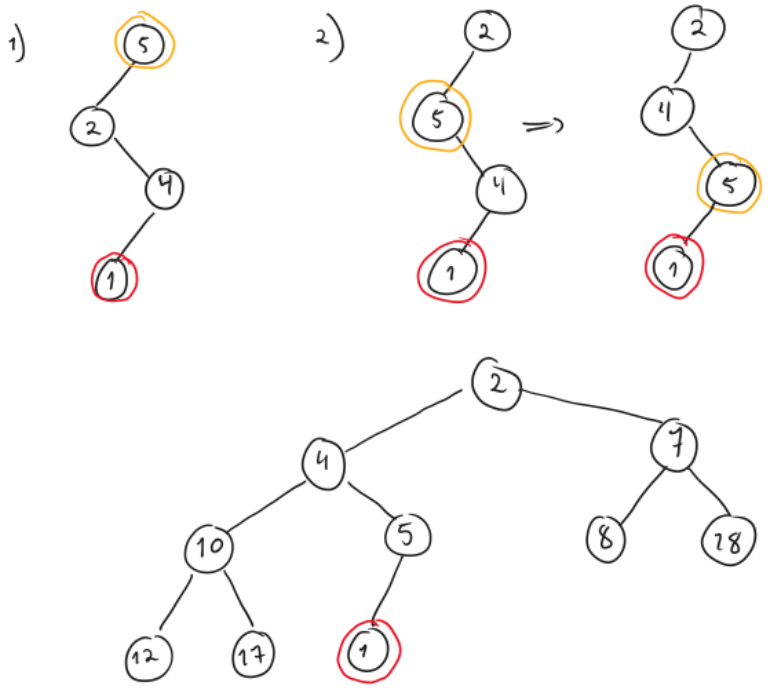
    if (p.leftChild.value < p.rightChild.value)

    Bytt om p.value og p.leftChild.value

    p = p.leftChild

    else

    som over, men, men med rightChild



Effektivitet - legge inn/uttak

Effektivitet (orden) i en minimumsheap

Metode	Gjennomsnittlig	Beste tilfellet	Verste tilfellet
leggInn	konstant	konstant	logaritmisk
taUt	logartimisk	konstant	logaritmisk

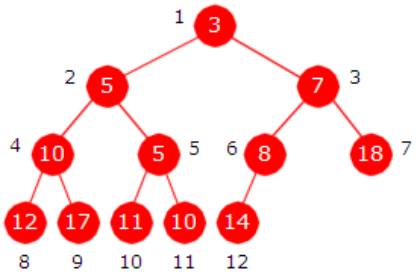
Hvor effektivt er det å legge inn og ta ut fra en minimumsheap? En ny verdi skal legges på rett sortert plass i den grenen som går ned til den nye noden i bunnen av treet. Men den grenen vil få, hvis det er n noder på forhånd, en lengde lik $\lfloor \log_2 (n + 1) \rfloor$. I verste fall skal den nye verdien legges øverst i grenen (i rotnoden) og innleggingen blir dermed av logaritmisk orden. Men i gjennomsnitt blir det annerledes. En minimumsheap er et komplett tre og i et slikt tre vil i gjennomsnitt ca. 2/3-deler (egentlig $1 - \frac{1}{2} \log 2$) av verdiene ligge på de to nederste radene. Hvis treet er perfekt ligger i gjennomsnitt andelen på 3/4. Alle grenene er sortert. Dermed vil verdiene på de to nederste radene utgjøre nær 2/3-deler av de største verdiene. En ny verdi har derfor i gjennomsnitt stor sannsynlighet for å havne nær en av de to nederste radene. Det viser seg at det i gjennomsnitt trengs kun litt over to sammenligninger for å finne den rette plassen siden sammenligningene starter fra bunnen. Innleggingen blir derfor i gjennomsnitt av konstant orden.

Det å ta ut den minste verdien vil være av logaritmisk orden både i gjennomsnitt og i det verste tilfellet. Når den minste, dvs. rotverdien, tas ut, fjernes den siste noden og dens verdi settes inn på rett sortert plass i minimumsgrenen. Denne verdien må relativt sett være stor siden den lå på bunnen av treet, og den vil derfor med stor sannsynlighet havne langt ned i minimumsgrenen. I verste tilfellet helt nederst. Det betyr at det i gjennomsnitt trengs nær like mange sammenligninger som grenen har lengde (som er lik $\lfloor \log_2 (n + 1) \rfloor$) for å finne rett plass i grenen siden sammenligningene starter ovenifra.

Heap som prioriteringskø

En prioritetskø kan kodes på en elegant og effektiv måte ved hjelp av en minimumsheap.

Minimumsheapen er satt opp med nodenes posisjonstall tatt med. Ved hjelp av dem kan vi gå oppover og nedover. Hvis k er posisjonen til en node, vil k/2 være posisjonen til forelderen, og $2 \cdot k$ og $2 \cdot k + 1$ posisjonene til venstre og høyre barn.



Heap som turneringstre

En minimumsheap kan (som et turneringstre) representeres ved hjelp av en tabell. Indekser svarer til nodeposisjoner. I algoritmene for å legge inn en verdi og å ta ut den minste verdien i en minimumsheap var det nødvendig å kunne gå i begge retninge i treet, og det gjør vi ved hjelp av indeksene.

Treet over er lagt inn i en tabell med 20 posisjoner. Det første tabellelementet er ikke i bruk siden det ikke er noen node som har posisjon 0:

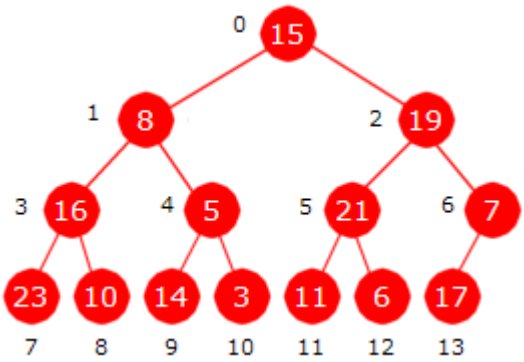
	3	5	7	10	5	8	18	12	17	11	10	14							
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Heapsortering

En tabell kan sorteres ved at den først gjøres om til en maksimumsheap og så gjøres den om til en sortert tabell. En slik algoritme kalles heapsortering.

Når verdiene allerede ligger i en tabell, må også tabellelementet med indeks lik 0 være med. Det betyr at når tabellen skal «tolkes» som et komplett binærtre, må roten ha posisjon 0:

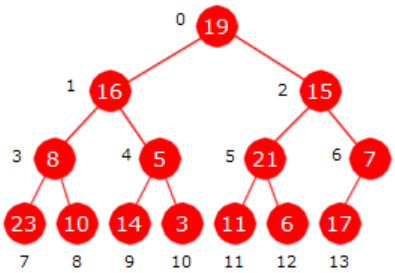
15	8	19	16	5	21	7	23	10	14	3	11	6	17
0	1	2	3	4	5	6	7	8	9	10	11	12	13



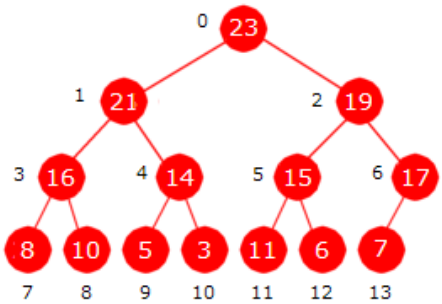
Når rotnoden har posisjon 0, må vi bruke en annen regel enn før når det gjelder forelder og barn. Hvis en node nå har posisjon k, så vil venstre barn ha posisjon 2k + 1, høyre barn posisjon 2k + 2 og forelder posisjon (k − 1)/2. Dermed får en verdi samme indeks i tabellen som den har posisjon i treet.

Treet over er i utgangspunktet ingen maksimumsheap. For at den skal være det må alle grenene være sortert avtagende. Første del av heapsortering går ut på å gjøre om treet til en maksimumsheap.

Vi starter med verdien i posisjon 1, dvs. tallet 8. Men det ligger allerede riktig i forhold til forelderverdien. Tallet i posisjon 2 (dvs. tallet 19) er større enn forelderverdien. Dermed må de bytte plass. Med andre ord blir 19 ny rotnodeverdi. Tallet 16 i posisjon 3 er større enn forelderverdien 8. De to må bytte plass, dvs. posisjon 1 vil da inneholde 16 og posisjon 3 vil inneholde 8. Går vi så videre til posisjon 4 (tallet 5), ser vi at den ligger riktig. Treet i figuren under viser dette så langt:



I treet over forstetter vi med verdien 21 i posisjon 5. Den er større enn verdiene i både forelder og besteforelder. Den skal derfor inn i posisjon 0 og de to andre (verdiene 19 og 15) trekkes nedover. Verdien 7 i posisjon 6 ligger riktig i forhold til forelderverdi. Verdien 23 i posisjon 7 er større enn rotnodeverdien. Den skal dit og de på veien oppover må forskyves nedover. Osv. til vi får flg. maksimumsheap:

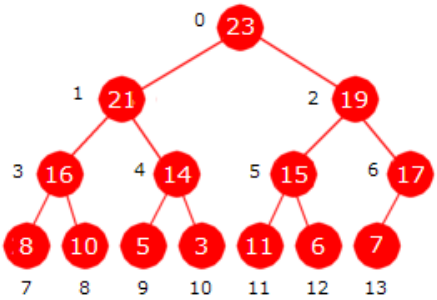


Kode for å lage maksimumsheap:

```
public static void lagMaksimumsheap(int[] a) {  // fra tabell til maksimumsheap
   for (int i = 1; i < a.length; i++) {         // starter i posisjon 1
     int k = i;                                // k er en hjelpevariabel
     int verdi = a[i];                         // verdi er en hjelpevariabel
     int forelder = (k - 1)/2;                 // forelder til k

     while (k > 0 && verdi > a[forelder]) {      // sammenligner med forelder
       a[k] = a[forelder];                     // trekker ned fra forelder
       k = forelder;                           // oppdaterer k
       forelder = (k - 1)/2;                   // oppdaterer forelder
     }
     a[k] = verdi;                             // rett sortert plass for verdi
   }
 }
```

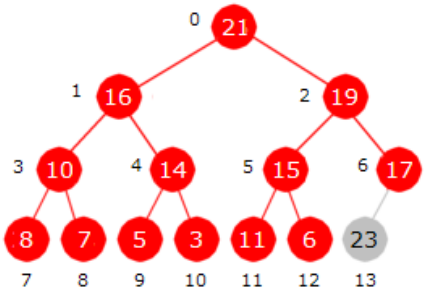
Neste skritt i heapsortering er å gjøre om maksimumsheapen til en sortert tabell. Vi tar utgangspunkt i flg. maksimumsheap satt opp både som et tre og som en tabell:

23	21	19	16	14	15	17	8	10	5	3	11	6	7
0	1	2	3	4	5	6	7	8	9	10	11	12	13



Siste verdi (7 i posisjon 13) tas vekk. Største verdi (rotnodeverdien 23) flyttes til posisjon 13. Så settes 7 inn på rett sortert plass i maksimumsgrenen. Det får vi til ved å flytte alle verdiene i grenen en enhet oppover og så legge 7 nederst (i posisjon 8). Det gir flg. resultat:

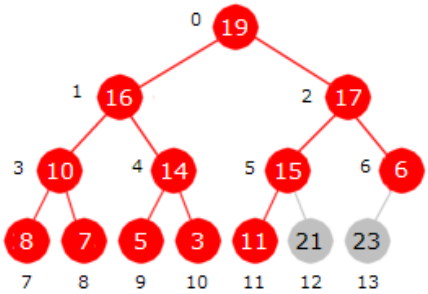
21	16	19	10	14	15	17	8	7	5	3	11	6	23
0	1	2	3	4	5	6	7	8	9	10	11	12	13



I treet over har den største verdien havnet bakerst og antallet i maksimumsheapen har blitt én mindre. Den delen som ikke hører til maksimumsheapen er markert med grått.

Dette gjentas. Siste verdi (6 i posisjon 12) tas vekk. 21 flyttes til posisjon 12 og 6 settes inn på rett sortert plass i maksimumsgrenen. Den vil nå ha verdiene 19, 17 og 6:

19	16	17	10	14	15	6	8	7	5	3	11	21	23
0	1	2	3	4	5	6	7	8	9	10	11	12	13



Kode for sortering av maksimumsheap:

```
public static void sorterMaksimumsheap(int[] a) {
   for (int n = a.length - 1; n > 0; n--) { // n er posisjonen til siste node
     int verdi = a[n];                      // den bakerste i maksimumsheapen
     a[n] = a[0];                           // den største flyttes bakerst
     int k = 0;                             // en nodeposisjon
     int m = (n - 1)/2;                     // stoppverdi

     while (k < m) {                         // så lenge k har to barn
       int barn = 2*k + 1;                  // venstre barn til k
       if (a[barn + 1] > a[barn]) barn++;   // finner maksimumsgrenen
       a[k] = a[barn];                      // forskyver oppover
       k = barn;                            // flytter k nedover
     }

     if (2*(k + 1) == n) {                  // k har kun et venstre barn
       k = 2*k + 1;                         // k går til venstre barn
       a[(k - 1)/2] = a[k];                 // forskyver oppover
     }

     int forelder = (k - 1)/2;              // forelder til k

     while (k > 0 && verdi > a[forelder]) {  // sammenligner med forelder
       a[k] = a[forelder];                  // trekker ned fra forelder
       k = forelder;                        // oppdaterer k
       forelder = (k - 1)/2;                // oppdaterer forelder
     }

     a[k] = verdi;                          // rett sortert plass for verdi
   }
 }
```

La n være posisjonen til den siste noden i et komplett binærtre der rotnoden har posisjon 0. La k være en vilkårlig nodeposisjon. Da har k to barn hvis 2*k + 1 < n og ett (venstre) barn hvis 2*k + 1 = n.

La n være posisjonen til siste node i maksimumsheapen. Dit skal den største verdien flyttes. Da forsvinner en node og dermed blir n − 1 siste posisjon i den delen hvor en verdi skal settes inn på rett sortert plass i maksimumsgrenen. Hvis 2*k + 1 < n - 1 (som er ekvivalent med 2*(k + 1) < n og med k < (n - 1)/2), vil dermed k ha to barn.

Heapsort blir av orden n log(n) siden første del er av orden n og den andre delen av n log(n). Den er i gjennomsnitt mindre effektiv enn f.eks. kvikksortering, men har den fordelen at den er av orden n log(n) i alle tilfeller, mens kvikksortering er av kvadratisk orden i de verste tilfellene.

Andrés kildekode for heapsort

```
public class HeapSort {
    public static void main(String[] args) {
        int values[] = {-99, 5, 9, 8, 3, 1};
        heapsort(values);
    }
    static void heapsort(int[] values) {
        System.out.println("Array før heapify: " + Arrays.toString(values));
        heapify(values);
        System.out.println("Array etter heapify: " + Arrays.toString(values));

        for (int i=1; i<values.length; ++i) {
            int first = 1;
            int last = values.length - i;
            System.out.println("Bytter " + first + " med " + last);
            int tmp = values[first];
            values[first] = values[last];
            values[last] = tmp;
            int current = first;
            int left_child = current*2;
            int right_child = current*2 + 1;
            while (true) {
                //Venstre barn er minst av barna, og mindre enn parent
                if (left_child < last
                        && values[left_child] < values[right_child]
                        && values[left_child] < values[current]) {
                    System.out.println("Bytter " + current + " med " + left_child);
                    int tmp2 = values[left_child];
                    values[left_child] = values[current];
                    values[current] = tmp2;
                    current = left_child;
                }
                //Høyre barn er minst av barna, og mindre enn parent
                else if (right_child < last
                        && values[right_child] < values[left_child]
                        && values[right_child] < values[current]) {
                    System.out.println("Bytter " + current + " med " + right_child);
                    int tmp2 = values[right_child];
```

```
                values[right_child] = values[current];
                values[current] = tmp2;
                current = right_child;
            }
            //Vi har funnet riktig plass til elementet.
            else {
                break;
            }
            left_child = current*2;
            right_child = current*2 + 1;
        }
    }
    System.out.println("Array etter uttak: " + Arrays.toString(values));
}

static void heapify(int[] values) {
    for (int i=1; i<values.length; ++i) {
        int current = i;
        int parent = i / 2;

        while (parent > 0 && values[parent] > values[current]) {
            System.out.println("Bytter " + current + " med " + parent);
            int tmp = values[parent];
            values[parent] = values[current];
            values[current] = tmp;
            current = parent;
            parent = current / 2;
        }
    }
}
}
```

Komprimering

Huffmantrær

Huffmantrær er en måte å tapsløst komprimere tekst i bit-format ved å benytte frekvensen av bokstaver i en tekst. Huffmantrær er basert på hvordan morsealfabetet fungerer. De mest høyfrekvente bokstavene i alfabetet har kortere/enklere morsekode enn de som er mindre frekvente.

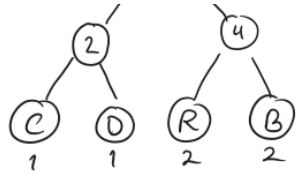
Vi lager et huffmantre først med å lage en frekvenstabell. Vi teller opp antall forekomster av en bokstav i en tekst. Man kan også bruke ferdiglagde standard frekvenstabeller.

Gitt ordet "abracadabra" får vi følgende frekvenstabell ved å telle opp antall forekomster av hver bokstav:

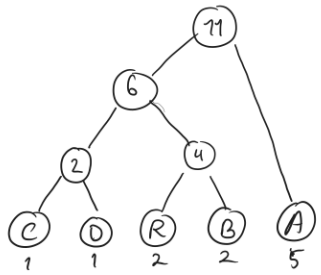
Symbol	a	b	c	d	r
Frequency	5	2	1	1	2

Neste steg er å lage noder av hver bokstav og sortere de med lavest frekvens fra lavest til venstre og fylle på mot høyre:

Deretter kobler man sammen de nodene med lavest verdi (frekvens) og gir foreldrenoden den sammenlagte verdien.



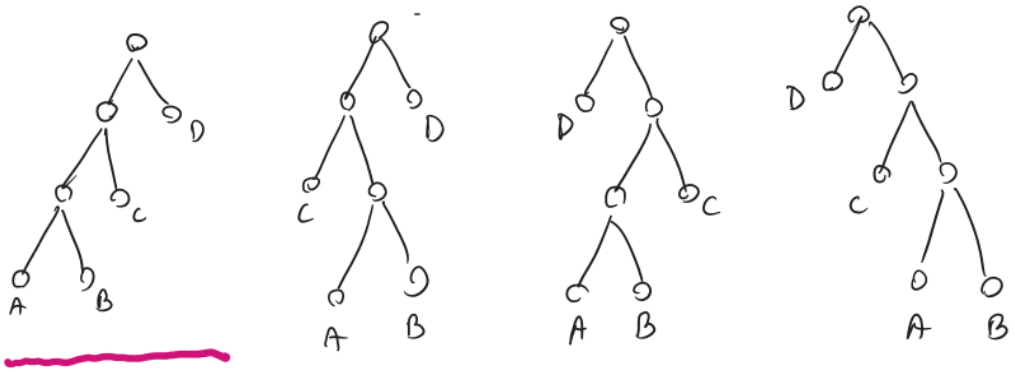
Vi fortsetter å koble de nodene med lavest verdi og gir foreldrenoden den sammenlagte verdien.



Til slutt ender man opp med en rotnode som blir den sammenlagte verdien av alle forekomster.

Kanonisk huffmantre

Vi kan ha mange ulike huffmantrær. Vi jobber med et kanonisk huffmantre. Det blir som en standard vi jobber utifra for å sørge for å et likt huffmantre å jobbe utifra, slik at vi får en standard komprimering.



Det kanoniske huffmantreet lager vi på følgende måte:

- Vi tar huffmantreet vi lagde over

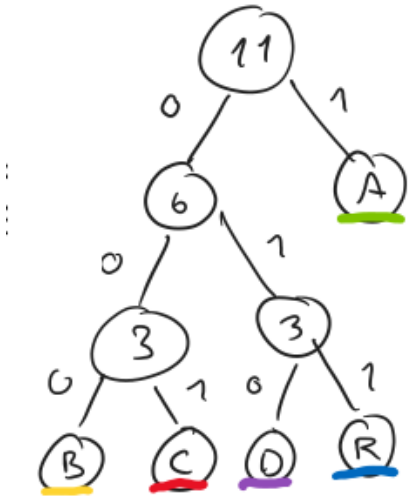
- Vi forskyver alle nodene til venstre

- Alle noder på samme nivå sorteres alfabetisk.

- Vi setter 1 på alle grener til høyre barn og 0 på alle grener til venstre barn

Frekvenssum legges til på samme måte, og vi ender opp med samme totalsum i rotnoden.

Fra eksempelet over får vi da følgende kanoniske tre:



Huffmankomprimering - koding/dekoding

Når vi har laget det kanoniske huffmantreet kan vi bruke det til å lage en komprimert kode. Istedenfor å gi hver bokstav en 8-bit ASCII verdi får vi nå en komprimert bitverdi, hvor de mest høyfrekvente bokstavene har en mindre bitverdi. Dette gjør at vi i snitt sparer mye plass. Istedenfor at A har bit-verdien 01000001 i ASCII vil den nå kun ha verdien 1.

Vi finner verdien til hver bokstav ved å følge grenene fra rotnoden og legge til 0 eller 1 i bitverdien alt ettersom om man går til høyre eller venstre i treet. Vi får da følgende kodetabell for "abracadabra":

	Frekvens	Kode
A	5	1
B	2	000
R	2	011
C	1	001
D	1	010

For å komprimere ordet benytter vi nå koden fra tabellen. Når vi skal dekode benytter vi samme tabell til å lese av hva koden representerer fra tabellen. Når vi bruker huffmankomprimering må vi da følgelig også lagre tabellen for å kunne komprimere og dekomprimere en tekst.

Ved å bruke tabellen vår vil da "abracadabra" bli til 1 000 011 1 001 1 010 1 000 011 1

LZW-komprimering

LZW-komprimering er en annen komprimeringsalgoritme for tapsløs komprimering av tekst. I motsetning til huffmankomprimering hvor man erstatter hvert symbol med kode, så tar man i LZW-komprimering og slår sammen flere symboler til "ett ord", og koder disse "ordene" istedenfor hvert enkelt symbol.

Hver gang LZW ser en ukjent symbolkombinasjon så lager algoritmen en ny kode for kombinasjonen. Fordelen med LZW-komprimering er at man ikke trenger å lagre hele ordboken med koder for å dekode, men kun enkeltsymbolene.

LZW algoritmen

1. Initialiser ordbok - Alle symboler med en tilhørende verdi (feks 0-99>)

2. Sett **w** = " " (tom streng)

3. Les neste bokstav inn i **k**

4. Lag strengen **wk** = **w** + **k**

    - Hvis vi har wk i ordbok:

        Sett **w** = **wk**

        Gå til punkt 3

    - Hvis ikke:

        Legg **wk** i ordboken

        Skriv ut koden for **w**

        Sett **w** = **k**

        Gå til punkt 3

5. Til slutt: Skriv ut kode for **wk**

LZW-komprimering eksempel

"ABRACADABRA". Startordbok:

A	B	C	D	R
0	1	2	3	4

Steg	**w**	**k**	**wk**	**i ordbok?**	**legg i ordbok**	**kode fra ordbok**
1	" "	"A"	"A"	JA		
2	"A"	"B"	"AB"	NEI	AB	0
3	"B"	"R"	"BR"	NEI	BR	1
4	"R"	"A"	"RA"	NEI	RA	4
5	"A"	"C"	"AC"	NEI	AC	0
6	"C"	"A"	"CA"	NEI	CA	2
7	"A"	"D"	"AD"	NEI	AD	0
8	"D"	"A"	"DA"	NEI	DA	3
9	"A"	"B"	"AB"	JA		
10	"AB"	"R"	"ABR"	NEI	ABR	5
11	"R"	"A"	"RA"	JA		7

Etter å ha lagt inn ordene fra tabellen over får vi ny ordbok:

A	**B**	**C**	**D**	**R**	AB	BR	RA	AC	CA	AD	DA	ABR
0	1	2	3	4	5	6	7	8	9	10	11	12

Kode som blir skrevet ut: 014020357

Ordbok: A=0, B=1, C=2, D=3, R=4

LZW-dekoding

1. Sett x = første tall
2. Sett w = tegnet for x -> skriv ut w
3. Les neste x

    Hvis x er i ordboka:

        Sett k = tegnet for x
        Skriv ut k
        Sett wk = w + k[0] og legg til i ordboka
        Sett w = k
        Gå til 3.

Eksempel:

Gitt eksempelet over med koden 014020357 og ordbok A=0, B=1, C=2, D=3, R=4

Steg	**x (kode)**	**Kode i ordbok?**	**w**	**k**	**wk**	**Utskrift**
1	0	JA	"A"			A
2	1	JA	"A"	"B"	AB	B
3	4	JA	"B"	"R"	BR	R
4	0	JA	"R"	"A"	RA	A
5	2	JA	"A"	"C"	AC	C
6	0	JA	"C"	"A"	CA	A
7	3	JA	"A"	"D"	AD	D
8	5	JA	"D"	"AB"	DA	AB
9	7	JA	"AB"	"RA"	ABR	RA

Ordbok:

A	**B**	**C**	**D**	**R**	AB	BR	RA	AC	CA	AD	DA	ABR
0	1	2	3	4	5	6	7	8	9	10	11	12

Utskrift: ABRACADABRA

Hashing (hashmap)

Verbet hash har betydningen: «to chop into small pieces». I databehandling er hashing en teknikk for å legge objekter i en tabell på en slik måte at både innlegging, søking og fjerning blir effektive operasjoner. Målet er at alle tre operasjonene skal være av konstant orden. Teknikken består normalt av tre trinn:

1. Objektet deles opp («hakkes i biter») og delene brukes til å konstruere et (stort) heltall. Dette heltallet kalles objektets hashverdi.

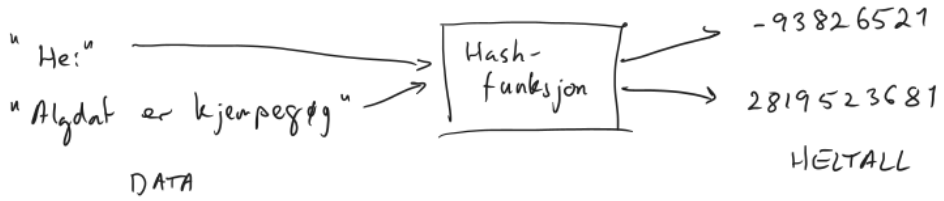
2. Hashverdien (heltallet) konverteres ved hjelp av en bestemt regel til en tabellposisjon, f.eks. ved å bruke resten når hashverdien heltallsdivideres med tabellengden.

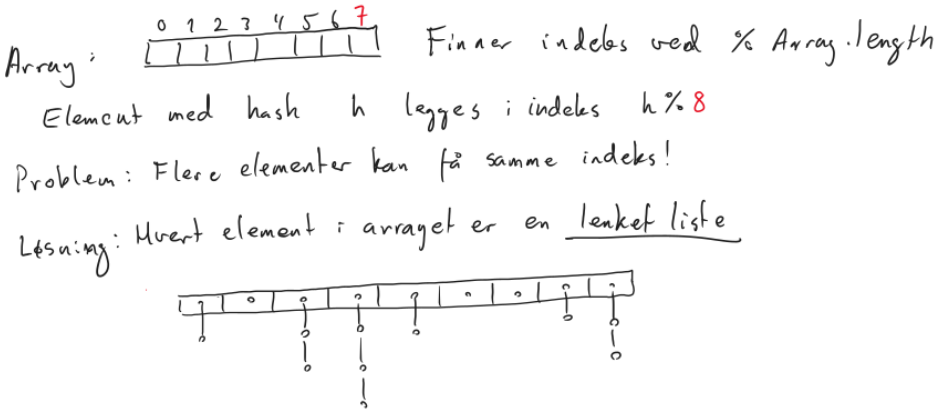
3. Hvis tabellposisjonen allerede er i bruk (kollisjon - et annet objekt ligger der), skal det likevel være mulig (ved hjelp av en fast regel) å lagre objektet og da på en slik måte at det lett kan gjenfinnes. Dette kalles kollisjonsbehandling.

Hashing har O(1) for innlegging og søking.

Hashing er en så viktig teknikk at de som konstruerte Java valgte å la basisklassen Object få metoden hashCode(). Metoden hashCode() er satt opp som native i basisklassen Object. Det betyr at koden ikke er synlig for oss. Metoden vil da normalt være kodet i et annet språk (f.eks. C eller C++) pga. effektivitet.

Kort fortalt sender man inn data i hashfunksjonen og får et heltall ut:





Hvis vi ikke skal bruke lenket liste, men kun ha en verdi på hver plass så gjør man det samme som i skrittene under, men istedenfor å legge i en lenket liste så "hopper" man en plass fremover og legger inn i nærmeste ledige plass. Om arrayet er fullt så lager man et nytt array og dobler vanligvis arrayet. For å søke så gjør man det samme som under, men istedenfor å søke i den lenkete listen så finner man heltallsrest, ser om verdien ligger der, og hvis ikke så "hopper" man fremover til man finner riktig verdi.

Hash innlegging

1. Beregn hash **O(1)**

2. Finn heltallsrest **O(1)**

3. Legg inn i lenket liste **O(1)**

Effektivitet: **O(1)**

Hash søking

1. Beregn hash O(1)

2. Finn heltallsrest O(1)

3. Søk i lenket liste O(N/n) = O(m)

N = Antall elementer i hashmapet
n = Array.length
m = Gjennomsnittlig antall elementer i hver lenkede liste

= LOAD FACTOR

Hvis O(m) = O(1)

Hash fjerning

1. Finn tabellindeksen.

2. Søk i den (eventuelt tomme) (lenkede) listen som hører til indeksen.

3. Fjern den (hvis den er der) på vanlig måte (ved å endre på en referanse).

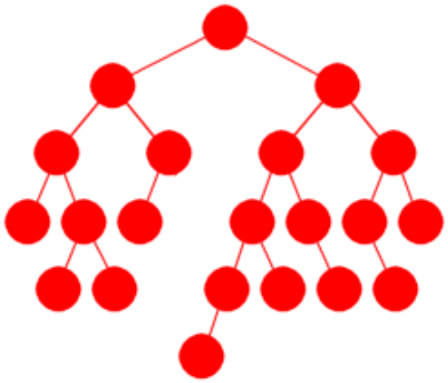
4. Reduser antallvariabelen.

Hash kode

```
public boolean legginn(T verdi) {
            Objects.requireNonNull(verdi, "verdi er null");
            if (antall >= grense) {
                utvid();
            }
            int hashverdi = verdi.hashCode() & 0x7ffffff;
            int indeks = hashverdi % hash.length;   //legger til første node som hører til indeks
            hash[indeks] = new Node<>(verdi, hashverdi, hash[indeks]);
            antall++;
            return true;
        }
    }

public boolean fjern(T verdi) {
        if (verdi == null) return false;
        int hashverdi = verdi.hashCode() & 0x7ffffff;
        int index = hashverdi % hash.length;
        Node<T> remove = hash[index];
        Node next = null;
        while (remove != null) {
            if (verdi.equals(remove.verdi)) break;
            remove = (next = remove).neste;
        }
        if (remove == null) return false;
        else if (remove == hash[index]) hash[index] = remove.neste;
        else next.neste = remove.neste;

        antall--;
        return true;
    }
```

Balanserte trær

Et binærtre kalles **høydebalansert** hvis det for hver node er slik at høydeforskjellen mellom nodens to subtrær ikke er mer enn 1. Se treet til høyre.



Et alternativ er **antallbalanse** eller vektbalanse (antall tolkes som vekt). Her må antallet noder i en vilkårlig nodes subtrær utgjøre en gitt minsteandel α av antallet i treet med noden som rot. Hvis p er en vilkårlig node, må antall(p.venstre) $\geq \alpha \cdot$ antall(p) og antall(p.høyre) $\geq \alpha \cdot$ antall(p). Her er det viktig å sette rett verdi på andelsfaktoren α.

Ubalanse

Et binærtre der ingen noder har to barn, er det mest ubalanserte av alle. Et slikt tre har høyde $n - 1$ og gjennomsnittelig nodedybde $(n - 1)/2$. Det er to ytterpunkter. Et binærtre kalles ekstremt høyreskjevt hvis ingen noder har venstre barn og ekstremt venstreskjevt hvis ingen noder har høyre barn.

Selvbalanserende trær

En naiv innlegging av tallrekka 1-99 vil føre til at vi får alle noder plassert til høyre i en lang rekke. Dette er verst tenkelig scenario når det kommer til søking. Selvbalanserende trær er trær som kontinuerlig balanseres når elementer blir lagt inn. Dette skjer ved ulike regler som sørger for at de alltid blir balanserte.

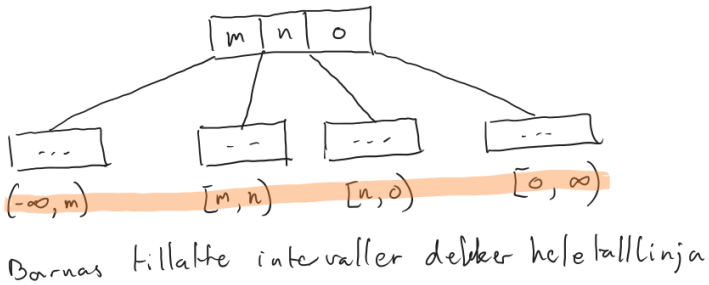
2-3-4 trær

- En node kan ha 1,2 eller 3 verdier.

- En indre node kan ha 2,3 eller 4 barn.

- Antallet verdier i en indre node er alltid én mindre enn antallet barn som noden har.

- Alle bladnoder ligger på samme nivå i treet



Sorteringsrekkefølge:

- Verdiene i hver node skal være sortert stigende.

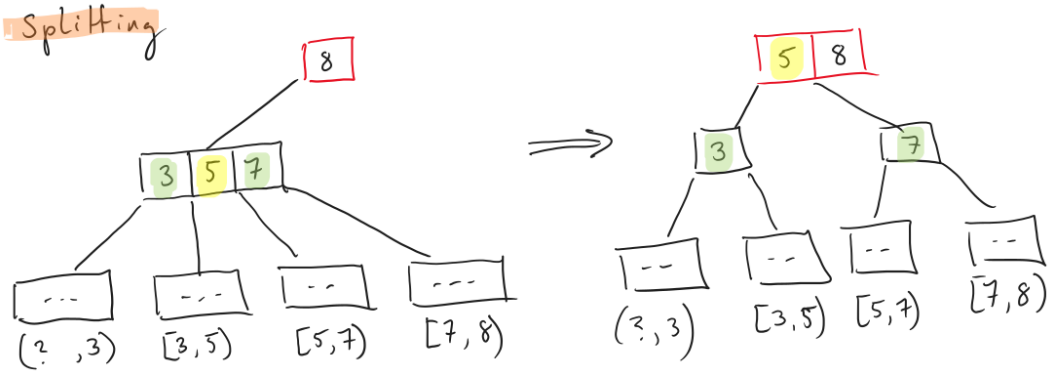
- Hvis x er en vilkårlig verdi i en indre node skal største verdi i det venstre subtreet til x være mindre enn x og x skal være mindre enn den minste verdi i høyre subtreet til x.

Innleggingsalgoritme (ovenifra og ned) for 2-3-4 trær

- Hvis treet er tomt, opprettes en bladnode og verdien legges inn i den.

- Finn den bladnoden som verdien sorteringsmessig hører til. Splitt alle 4-noder som passeres på veien ned dit (også bladnoden hvis den er en 4-node). En node splittes ved at midtverdien rykker opp til foreldernoden (på rett sortert plass) og ved at de to verdiene på hver siden inngår i sine egne noder. Hvis det er rotnoden som skal splittes, rykker midtverdien opp til en ny rotnode.

- Hvis bladnoden har plass, legges verdien inn på rett sortert plass.
- Hvis den ble splittet (var en 4-node), legges verdien inn i den noden som den etter splittingen hører til.



Røde og svarte trær

Røde og svarte trær er selvbalanserende trær tilsvarende 2-3-4 trær. De er ekvivalente. For å slippe å bruke farger sier vi gjerne at ***røde noder er runde*** og ***svarte noder er firkantet.***

Definisjon:

1. Rotnoden er alltid svart (firkantet)

2. Alle nye noder er alltid røde (runde)

3. En rød node (runde) kan ikke ha røde barn

4. Alle grener til bladnoder har like mange svarte noder (firkantet)

Innleggingsalgoritmen:

1) Ny node **X** (rød/rund) settes på rett plass

2) Sett **F** = forelder, **B** = besteforelder og **O** = onkel

3.1) **F** er svart (firkantet) -> ferdig

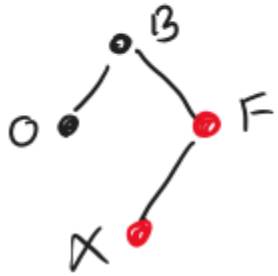
3.2) **O** er svart (firkantet) - hvis O er null så anser vi den som svart

3.2.1) Hvis **B - F - X** ligger på **rett linje** (x h. barn) -> Enkel rotasjon

3.2.2) Hvis **B - F - X** ligger på **knekt linje** (x v. barn) -> Dobbel rotasjon

- Fargeskift:

F (rød/rund) - **B** (svart/firkantet) ->
F (svart/firkantet) - **O** (svart/firkantet) - **B** (rød/rund)



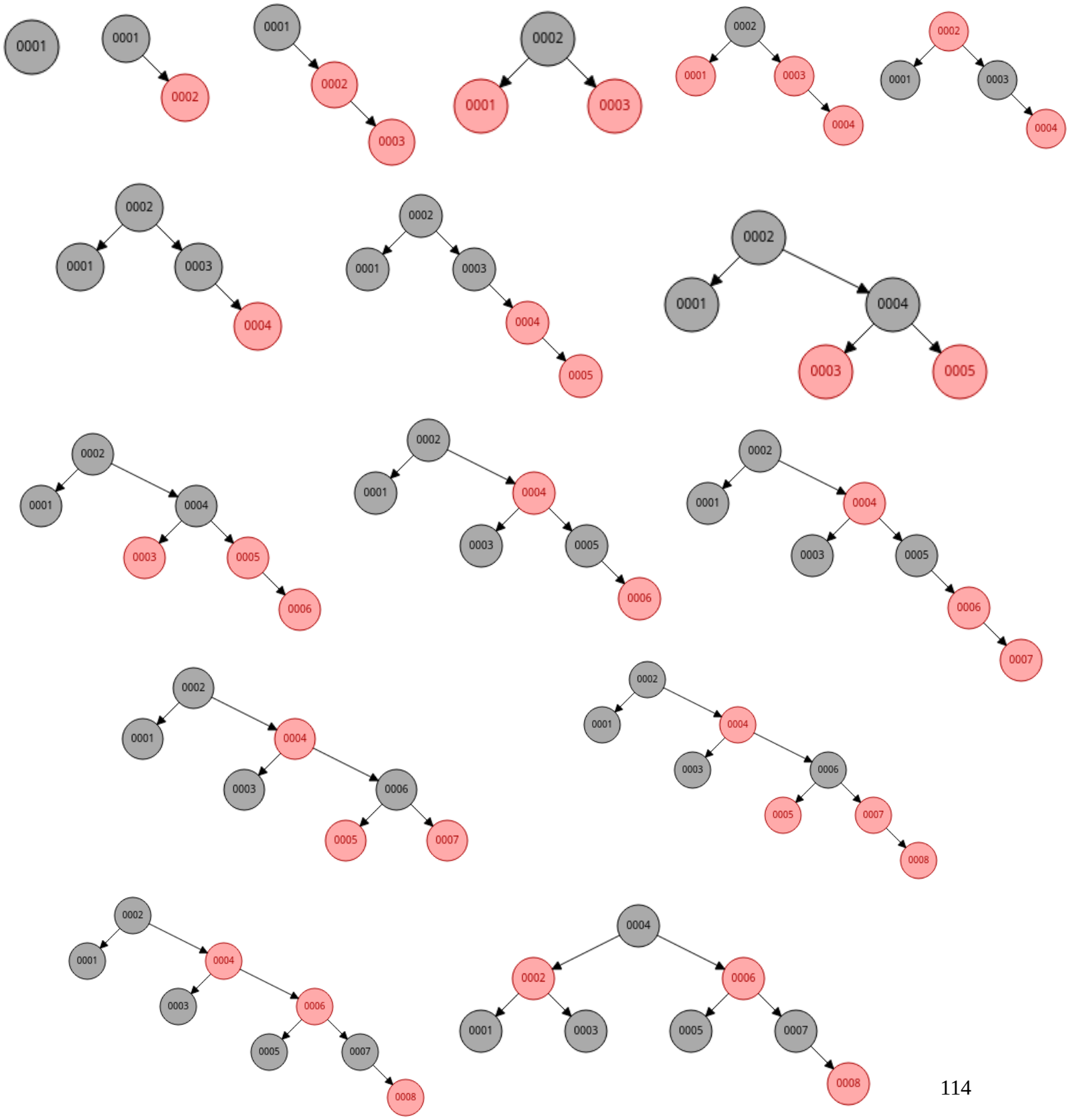
3.3) **O** er rød/rund

- Fargeskift:

F (rød) - **O** (rød) - **B** (svart) ->
F (svart) - **O** (svart) - **B** (rød)

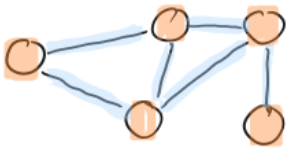
- Gå til 2) (Sett X = B???)

Røde og sorte trær eksempel: 1-8



Grafer

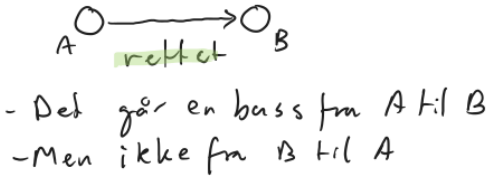
En graf G = (V,E) er en samling noder V (vertices) forbundet av kanter E (edges). Hver node har gjerne en bokstav som identifikator

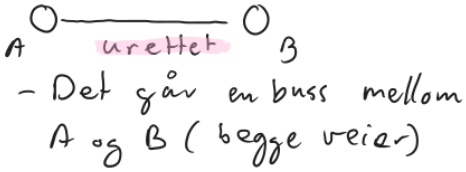


En bussrute kan tegnes som en graf. Nodene er stasjoner og kantene er ruten som går mellom stasjonene.

- To noder er ***naboer*** hvis det går en kant mellom de.

- En kant kan være ***rettet***, da har kanten en pil som viser retning. Eller ***urettet***, en kant uten pil.





- Kanter kan være ***vektede***, da skriver man en verdi på kanten. Vekten kan feks være tid en rute tar. De kan også være ***uvektede***. Da har de ingen verdi.

En kant er et nodepar. Hvis de to nodene i paret er like, kalles det en sløyfe (eng: loop). En graf uten sløyfer kalles sløyfefri. Det kan også være to eller flere like par, dvs. flere kanter mellom to noder. **Det gir flg. graftyper:**

- **Enkel urettet graf** − maksimalt én kant mellom to noder og sløyfefri.
- **Enkel rettet graf** − maksimalt én kant i samme retning mellom to noder og sløyfefri.
- **Multigraf** − det kan være flere kanter mellom to noder, men sløyfefri.
- **Pseudograf** − det kan være flere kanter mellom to noder og noder kan ha sløyfer.

Eksempler på grafer:

- Enkeltlenket liste (uvektet og rettet)
- Huffmantrær (vektet og rettet)
- Veinett i Google Maps (vektet og rettet)
- Sosiale grafer (Facebooks OpenGraph) - personer er noder og vennskap er kanter

Begreper

Urettet graf

- Nabo To noder X og Y kalles naboer hvis det går en kant mellom dem. På engelsk er to noder adjacent (tilstøtende) hvis de er naboer.

- Nodegrad Graden til en node er lik kantantallet. En sløyfe utgjør to kanter.

- Vei og veilengde Det går en vei mellom to noder X og Y hvis det er mulig å gå fra X til Y ved å følge kanter. Hvis grafen er uvektet, er veilengden lik antallet kanter og hvis den er vektet, er veilengden summen av kantvektene.

- Enkel vei En vei kalles enkel hvis alle kantene på veien er forskjellige.

- Sykel En enkel vei som starter i en node X, inneholder minst én kant og som ender der den startet (dvs. i X), kalles en sykel eller en krets (eng: circuit). En graf uten sykler kalles asyklisk.

- Korteste vei Hvis det går flere enn én vei mellom to noder X og Y, er den veien kortest som inneholder færrest kanter eller minst kantvektsum hvis grafen er vektet.

- Sammenheng Grafen kalles sammenhengende hvis det går en vei mellom hvert par av forskjellige noder X og Y. Hvis den ikke er sammenhengende, så består den av to eller flere sammenhengende komponenter.

- Subgraf En subgraf (eller en delgraf) S av en graf G er en graf som består av en eller flere noder fra G og med kanter som også er kanter i G.

- Trær Grafen kalles et tre hvis den er sammenhengende og uten sykler (asyklisk).

Rettet graf

- Direkte etterfølger En node Y kalles en direkte etterfølger til en node X hvis det er en kant med retning fra X til Y.

- Vei og veilengde Det går en vei fra en node X til en node Y hvis det er mulig å gå fra X til Y ved å følge kanter i kantenes retning. I en uvektet graf er veilengden lik antall kanter på veien og i en vektet graf er den summen av kantenes lengde/vekt.

- Enkel vei En vei kalles enkel hvis alle kantene på veien er forskjellige.

- Sykel En enkel vei som starter i en node X, inneholder minst én kant og som ender der den startet (dvs. i X), kalles en sykel eller en krets (eng: circuit). En rettet graf uten sykler kalles asyklisk.

- Korteste vei Hvis det går flere enn én vei fra en node X til en node Y, er den veien kortest som har minst lengde.

- Etterfølger En node Y kalles en etterfølger til en node X hvis det går en vei fra X til Y.

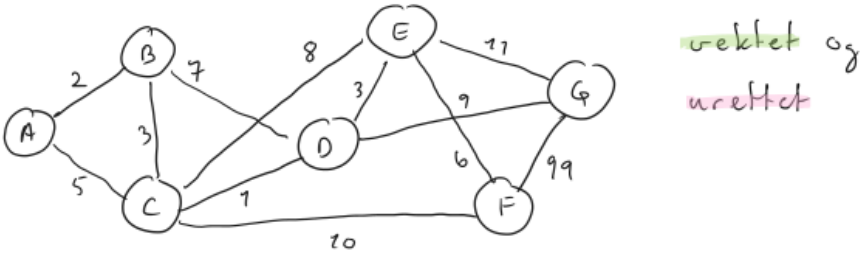
- Direkte forgjenger og forgjenger En node X kalles en direkte forgjenger til en node Y hvis det går en kant fra X til Y. Dvs. at Y er en direkte etterfølger til X. En node X kalles en forgjenger til en node Y hvis Y er en etterfølger til X.

- Kilde og sluk En node med kun utkanter er en kilde. En med kun innkanter er et sluk.

- Sterk sammenheng Grafen kalles sterkt sammenhengende hvis det finnes en vei fra enhver node til enhver annen node.

- Svak sammenheng Grafen kalles svakt sammenhengende hvis den urettede grafen vi får ved å fjerne retningen på alle kantene, blir sammenhengende.

Grafer i minnet



Grafer kan lagres i minne på mange ulike måter. Lenkede lister, trær og hashmap er kanskje vanligst.

Som tekstfil

Graf kan lagres som en tekstfil. Hver linje har en nodeidentifikator og alle dens naboer (med evt. kantvekter). Vi trenger ikke skrive den uretta kanten to ganger. Gitt grafen over, så får vi i tekstfil:

A, B2, C5
B, C3, D7
C, D1, E8, F10
D, G9, E3
E, G11, F6
F, G99
G

Grafer som matriser

En graf kan representeres som en todimensjonal tabell (matrise). Hvis grafen har ***n*** noder, vil tabellen få dimensjon ***n x n***. Hver rad tilhører en bestemt node og i raden er de nodene markert der hvor det går en kant fra denne noden. Hver kolonne hører også til en bestemt node. I kolonnen er det markert fra hvilke noder det går en kant til noden. Dette kalles en naboskapsmatrise. (vannrett til, loddrett fra)

	A	B	C	D	E	F	G
A		x	x	x			
B	x			x	x		
C	x			x		x	
D	x	x	x		x	x	x
E		x		x			x
F			x	x			x
G				x	x	x	

	A	B	C	D	E	F	G
A		x	x	x			
B				x	x		
C				x		x	
D					x	x	x
E							x
F							x
G							

	A	B	C	D	E	F	G
A		2	3	5			
B				2	4		
C				2		6	
D					2	3	6
E							3
F							2
G							

Bildet viser 1) urettet og uvektet graf 2) rettet og uvektet graf 3) rettet og vektet graf

Dijkstras algoritme

Dijkstras algoritme handler om å finne korteste/enkleste vei i en graf.

Lag en køtabell, skriv inn nodenavn, verdi og kryss av (og nummererer) de du har besøkt

1. Sett startnode som ***current*** og verdien til 0
2. Sett nodeverdi til alle ubesøkte noder til ∞
3. Gå til ubesøkt node med minst ***kantvekt***
    - Hvis ***current*** verdi + ***kantvekt*** er mindre enn nodeverdi -> oppdater verdi
    - Fortsett til alle verdier er oppdatert
4. Marker current som besøkt
5. Flytt ***current*** til noden med lavest verdi
6. Gå til punkt 3
    - Hvis ***current*** = sluttnode eller minste avstand til ubesøkte node er ∞ (finnes ingen vei) -> slutt

Finne korteste vei:

Traverser deg tilbake fra sluttnoden.

Sett ***current*** som sluttnoden, trekk kantvekt fra nodeverdien.

Hvis resultatet blir lik verdien til nabonoden -> flytt current til nabonoden

Fortsett til man har kommet til 0.

Da har man funnet veien "baklengs". Snu om denne for å vinne veien fra start til slutt.